SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA37 – Software Testing

ASSIGNMENT 1 – CO1

Apply, Analyze and Justify Software Testing Principles and Defect Management Strategies for a Hospital Outpatient Management System

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Student Name	TANUJA P	**Register Number**	192321052
Department / Section	IT	**Academic Year / Semester**	2025- 2026 / 7th sem
Faculty Name	Dr. Gnanajayaraman	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Testing as an Engineering Activity	4		
2	Role of Process in Software Quality	5		
3	Testing as a Process and Basic Definitions	4		
4	Software Testing Principles	4		
5	Tester's Role	5		
6	Origins of Defects	5		
7	Defect Classes	4		
8	Defect Repository	5		
9	Integrated Testing Practice	6		
10	Re-apply and Adapt the Testing Practice	8		
TOTAL		50		Percentage:

OVERALL REMARKS

TANUJA P		
Student Signature	**Faculty Signature**	**Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task.

Task	Evidence expected	Status	Note
1	A fault-detection and effort table for the three models, a comparison of the activities each contains, and a justified choice with its limits stated.	✓	Table 3, Table 5; Figure 1, Figure 2
2	A process-wise test-activity timeline, a defect-removal-efficiency and cost table, a comparison of the three processes, and a justified choice.	✓	Figure 3; Table 10; Table 11
3	A table of the three strategies with faults exposed, an error–fault–failure classification of real runs, and a justified choice of oracle strategy.	✓	Table 12, Table 13; Figure 5
4	An input-space calculation, a defect-clustering chart, a faults-found table for A, B and C, and a justified 25-test selection.	✓	Figure 6; Table 19, Table 22
5	A faults-found and effort table for the three arrangements, two contrasting defect reports, and a justified allocation of tester responsibility.	✓	Table 23, Table 24; Table 26
6	An artefact-versus-detection matrix, a fault tree and a Pareto chart, and a justified choice of method with preventive actions.	✓	Table 29; Figure 10, Figure 11, Figure 12
7	A classification table for D1–D10 under the three schemes, an agreement calculation, and a justified standard scheme.	P	Table 33; agreement sheet Table 35 to be filled with classmates' answers
8	Screenshots of the three tools with D1–D10, a life-cycle timing table, a defects-per-KLOC calculation, and a justified choice.	P	Table 37, Table 40; screenshots to be taken from the real tools
9	A measured comparison of three configurations, an original block diagram and decision table, an applications transfer table, and a defended final practice.	✓	Table 42, Table 45; Figure 16
10	Three adaptation strategies applied and compared with measured and calculated evidence, and a RETAIN / MODIFY / REDESIGN decision that follows from the changed constraints.	✓	Table 49, Table 52; Figure 19, Figure 20

DEFECT ANALYSIS EVIDENCE CHAIN (Tasks 1–10)

Evidence item	Status	Observation
Fault detection, effort and cost of escaped faults	✓	Table 3, Table 4; produced by running pytest on mutants
Defect-removal efficiency and repair cost per process	✓	Table 10; 82.3% now, 95.1% predicted for C + review gate
Origin assigned to every defect (artefact × detecting activity)	✓	Table 29; D1–D10 and 164-record log
Class, severity and priority justified for D1–D10	✓	Table 33, Table 34
Repository fields, life cycle and defects-per-KLOC	✓	Table 37, Table 40; Figure 14
Cost–benefit of the integrated practice	✓	Table 42; hours and DRE per configuration
Adaptation to changed constraints (50 h, weekly, audit)	✓	Table 52; REDESIGN decision
Figures and conclusions consistent	✓	Configuration C (Task 9) and REDESIGN (Task 10) follow from Tasks 1–8

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All 10 tasks answered	☒ Yes ☐ Partly ☐ No

Requirement	Status
Calculations (detection %, cost of escapes, DRE, repair cost, effort) shown step by step	☒ Yes ☐ Partly ☐ No
All ten defects D1–D10 classified (origin, class, severity, priority)	☒ Yes ☐ Partly ☐ No
Error, fault and failure separated for real test runs (Task 3)	☒ Yes ☐ Partly ☐ No
Testing principles linked to evidence from the case	☒ Yes ☐ Partly ☐ No
Repository design justified field by field	☒ Yes ☐ Partly ☐ No
Tables, figures and diagrams included and readable	☒ Yes ☐ Partly ☐ No
Cross-references and table / figure numbering correct	☒ Yes ☐ Partly ☐ No
Work consistent throughout	☒ Yes ☐ Partly ☐ No
Tool screenshots (Task 8) and classmates' classification (Task 7) attached	☐ Yes ☒ Partly ☐ No
Viva recommended	☐ Yes ☐ No

SOFTWARE TESTING

ASSIGNMENT 1 – ANSWERS

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assignment Title: Apply, Analyze and Justify Software Testing Principles and Defect Management Strategies for a Hospital Outpatient Management System (Total Marks – 50)

Duration: 1 week

Scenario – Quality Assurance of the Lakeside Hospital "MediQueue" Outpatient System

Lakeside Hospital runs a web system, MediQueue, for patient registration, doctor appointment booking, consultation billing, pharmacy dispensing, lab-report delivery and SMS reminders. In the last quarter, a patient aged exactly 60 was denied the senior-citizen concession, one consultation was billed twice after a patient refreshed the payment page, two patients were given the same appointment slot, the lab-report page took almost half a minute to open during the morning rush, and 29 defects were reported by hospital staff and patients after release (9 of them critical). The Medical Superintendent asks the QA team to stop treating testing as "a quick check before go-live" and to build a principle-driven, measurable testing and defect-management practice that protects patients and hospital revenue.

You are the QA engineer. In every task, three candidate techniques are given. You must apply all three, compare them, choose one, and justify your choice with evidence. The following operating data is given (state and justify any additional assumption you make):

Parameter	Value
Application	Web system – Angular front end, Node.js back end, PostgreSQL database; six modules: Registration (4 KLOC), Appointment (10 KLOC), Billing (12 KLOC), Pharmacy (9 KLOC), Lab Reports (7 KLOC), Reminders (5 KLOC) – total 47 KLOC
Business rules	Consultation fee ₹500. Concession by age: below 5 years free; 5–17 years 20% off; 18–59 years no concession; 60 years and above 40% off. Fees are rounded to the nearest rupee (0.5 rounds up). Late-cancellation fee ₹100 if an appointment is cancelled less than 2 hours before the slot. A patient may hold at most 2 active appointments. A prescription can be dispensed only within 7 days of its date
Release history	Last release: 164 defects in total – 135 found by the test team before release and 29 reported by users after release; release cycle is 3 weeks
Team and effort	3 testers, 5 developers, 1 QA lead; test effort available per release = 100 person-hours
Usage	9,000 registered patients and 250 doctors and staff; peak load 800 concurrent users between 9 and 10 am; response-time requirement ≤ 2 seconds
Relative cost to fix one defect (assume)	Requirements/design ₹800; coding ₹2,500; system testing ₹9,000; after release ₹45,000
Tools	Python 3 with pytest/unittest (or Java with JUnit); a spreadsheet; Bugzilla or JIRA (free/community edition); any LLM assistant for the LLM-based questions
Data	Defect log of the last release (164 records, faculty-provided or a realistic log prepared by you with at least 40 records) and the ten sample defects D1–D10 below

Sample Defects Reported on MediQueue (use in Tasks 6, 7, 8 and 9)

ID	Reported symptom
D1	A patient aged exactly 60 is charged ₹500 instead of ₹300 (the age check was coded as "greater than 60").
D2	Refreshing the payment-confirmation page re-submits the form and creates a second bill for the same consultation.
D3	A patient who cancels 3 hours before the slot is charged the ₹100 late-cancellation fee (the limit was coded as 4 hours instead of 2).
D4	Two patients clicking "Confirm" within the same second are both given the 10:15 slot of the same doctor.
D5	A patient whose surname contains an apostrophe (for example, D'Souza) cannot be registered; the form shows a database error.
D6	The lab-report page takes about 35 seconds to load when 800 users are online.
D7	The requirement says an SMS is sent "24 hours before the appointment" but does not say what happens when the appointment is rescheduled; the SMS reaches the patient with the old time.
D8	Test case TC-52 expects a fee of ₹450 for a 12-year-old (it should be ₹400); the faulty code applies only 10% concession and passes this test.
D9	A prescription dated 9 days earlier is dispensed because the validity check compares months instead of days.
D10	The lab-report PDF shows the sample-collection date in place of the report-issue date for blood tests (the date is read from the wrong field).

ANSWERS – READING GUIDE

Each task below is followed by its Ans: block in the same order as the evaluation bullets. All code-based numbers were produced by running the Python prototype `mediqueue.py` with 19 pytest suites (104 runs: each suite on the correct build and on every single-fault mutant). Items that depend on people, tools or hours are estimated and are marked as such (see Annexure C).

- **Fault found** = at least one test fails on the faulty build (mutant) and passes on the correct build. A test that fails on the correct build is a **false report**.
- Currency uses Indian digit grouping (for example ₹13,05,000); the unit costs are those given in the data table (₹800 / ₹2,500 / ₹9,000 / ₹45,000).
- Assumptions added in this answer: effort in person-hours (ph) is a planning estimate per task; the 164-record defect log is a realistic reconstruction (seed 42) that matches the given totals (135 before release, 29 after release, 9 critical among the escapes); the test date for prescriptions is 12-03-2026.

Table 1: Seeded faults of the prototype (one fault switched on per mutant)

ID	Fault	Defect type	Used in
F1	age > 60 instead of ≥ 60	D1	Tasks 1, 3, 9, 10
F2	10% instead of 20% child concession	D8 (masked)	Tasks 1, 3, 9
F3	4-hour cancellation limit instead of 2	D3	Tasks 1, 9
F4	age 4 treated as 5 (free only below 4)	Boundary	Tasks 1, 9
F5	₹100 fee missing when cancelled under 1 hour	Missing condition	Task 1
R1	fee rounded by truncation, not half up	Computation	Task 3
A1–A5	3 active allowed; same-second double booking; 4-h cancel limit; cancelled slot not freed; past slot bookable	D4, D3 types	Task 4
P1–P4	validity compares months; 31 units allowed; batch allowed on expiry day; duplicate by a second pharmacist	D9 type	Tasks 5, 9
E1–E4	video timeout in seconds; unsigned e-Rx accepted; health-ID ignores ' vs '; no audit entry	New rules	Task 10

TASK 1 – Testing as an Engineering Activity: Compare Testing Activity Models [4 Marks]

Management believes testing is the last week of the project. Before any plan is made, the QA lead wants evidence on how the testing activity should be placed in the life cycle of the Billing and Appointment modules. Compare three models.

Technique	Basic idea and key concepts / tools
A. Test-last model	All testing is carried out by the team after coding is complete, from a list of screens to be checked.
B. Review-plus-test model	Requirements and code are reviewed first; system testing then follows from the reviewed requirements.
C. Continuous engineering activity	Test planning, design, execution and measurement start at the requirement stage and continue through every build.

- Implement the concession-fee and late-cancellation rules in Python/Java with five injected faults (for example, age > 60 instead of ≥ 60, 10% instead of 20% concession, 4-hour limit instead of 2, age 4 treated as 5, missing ₹100 fee). Apply models A, B and C to this code and to a requirement text containing two planted ambiguities; record which faults and ambiguities each model exposes and the effort in person-hours.
- Calculate for each model the fault-detection percentage, the faults found per person-hour, and the cost of the faults that would escape to users, using the relative cost figures given in the table.
- Analyze the three models on the engineering activities they contain (planning, design, execution, measurement, defect prevention) and on the stage at which each defect is first detected.
- Examine why an ambiguity such as “less than 2 hours before the slot” is caught only by some models, and relate the missing activities to the 29 post-release defects.
- Evaluate and justify the model you would adopt within the 100 person-hour budget. State the residual risk and judge the statement “testing can show the presence of defects but never their absence” using your own results.
- Critique the proposal to let an LLM write the test plan for the two modules: generate the plan, check it against the business rules, and report wrong assumptions, missing items and generic content.

Evidence Expected: A fault-detection and effort table for the three models, a comparison of the activities each contains, and a justified choice with its limits stated.

Ans:

Comparison of Testing Activity Models for Billing and Appointment

Management places testing in the last week. To replace that belief with evidence, I implemented the concession and late-cancellation rules in Python (`mediqueue.py`), injected five faults, wrote a requirement text with two planted ambiguities, and applied the three models to the same code. Every fault count below was produced by running pytest on the five single-fault mutants; effort in person-hours is a planning estimate.

1. Implementation, injected faults and planted ambiguities

```
def concession_fee(self, age, base_fee=500):          # reference rule
    free_limit = 5                                    # F4: 4
    child_pct  = Decimal("0.20")                      # F2: 0.10
    senior     = age >= 60                            # F1: age > 60
    pct = 1 if age < free_limit else child_pct if age <= 17 else Decimal("0.40") if senior else 0
    return int((base_fee * (1 - pct)).quantize(Decimal(1), ROUND_HALF_UP))


def late_cancel_fee(self, hours_before):
    limit = 2                                         # F3: 4
    # F5: "if hours_before < 1: return 0" (last-minute branch forgets the fee)
    return 100 if hours_before < limit else 0
```

Table 2: Five injected faults and two planted ambiguities

ID	Rule	Injected fault / ambiguity	Type
F1	Senior concession	age > 60 coded instead of ≥ 60 (D1 type)	Boundary
F2	Child concession	10% instead of 20%	Wrong constant
F3	Late cancellation	4-hour limit instead of 2 (D3 type)	Wrong constant
F4	Free below 5	age 4 treated as 5 (free only below 4)	Off-by-one
F5	Late cancellation	₹100 fee missing when cancelled under 1 hour	Missing condition
AMB-1	Requirement text	"₹100 if cancelled **less than 2 hours before the slot**" – is exactly 2.0 h charged? Measured to slot start or end?	Ambiguity
AMB-2	Requirement text	"**60 years and above** get 40% off" – age on the appointment date or the registration date?	Ambiguity

2. Fault detection, effort and cost of escaped faults

- **A. Test-last:** 5 screen-checklist tests written after coding with typical values (ages 30, 65, 10; cancel 1.5 h and 6 h).
- **B. Review-plus-test:** requirement review and a code-review checklist, then 6 system tests from the reviewed requirement (ages 30, 70, 12, 3; cancel 1 h, 3 h).
- **C. Continuous:** 11 boundary tests designed at the requirement stage (ages 4/5, 17/18, 59/60/61; cancel 0.5, 1.99, 2.0, 2.01 h) and run on every build.

Table 3: Faults and ambiguities exposed by each model (pytest on 5 single-fault mutants; effort estimated)

Model	Faults found	Ambiguities found	Issues (of 7)	Detection %	Effort (ph)	Issues per ph
A Test-last	1 (F2)	0	1	14%	6	0.17
B Review + test	3 (F1 by code review; F2, F3 by test)	2	5	71%	10	0.50
C Continuous	5 (F1–F5)	2	7	100%	14	0.50

Table 4: Cost of found and escaped issues (found at the stage where the model finds it; every missed issue costs ₹45,000 after release)

Model	Cost to fix what was found	Escaped issues × ₹45,000	Total
A	F2 at system test: ₹9,000	6 × 45,000 = ₹2,70,000	₹2,79,000
B	2 ambiguities × ₹800 + F1 ₹2,500 + (F2, F3) × ₹9,000 = ₹22,100	2 × 45,000 = ₹90,000	₹1,12,100
C	2 ambiguities × ₹800 + 5 faults at coding × ₹2,500 = ₹14,100	0	₹14,100

Formulae: detection % = issues found ÷ 7 × 100; issues per ph = issues found ÷ effort. Model A is the cheapest to run but leaves ₹2.7 lakh of escape cost; C costs 8 ph more and removes it.

3. Engineering activities and stage of first detection

Table 5: Engineering activities contained in each model

Activity	A Test-last	B Review + test	C Continuous
Test planning	No	Partial (after review)	Yes, from requirements
Test design technique	Screen list	Requirement-based	Boundary value, decision table
Execution	Once, at the end	Once, after coding	Every build
Measurement	None	Review findings	Detection %, effort, cost per build
Defect prevention	None	Reviews	Reviews + early test design
Stage of first detection	System test	Requirements, coding, system test	Requirements and coding

4. Why the ambiguity is caught only by some models

"Less than 2 hours" hides a boundary question: is a cancellation at exactly 2.0 hours charged? Model A only runs typical values after coding, so it never asks the question. B finds it by reading the requirement; C finds it because writing a boundary test for 2.0 forces a decision before coding. In the defect log, 12 of the 29 post-release defects came from requirement artefacts (Task 6, Table 29). These are exactly the defects that a review or early test design would surface, and Model A contains neither activity.

5. Recommended model, residual risk and "presence, not absence"

Choice: C, continuous engineering activity. Within the 100 ph budget, C takes about 14% of the effort on these rules and found every seeded fault and both ambiguities, leaving ₹0 of escape cost against ₹2,70,000 for test-last. Residual risk: concurrency (D4), performance (D6) and faults outside the seeded set are not covered by these unit-level tests.

My own results support the statement. Suite A passed on 4 of the 5 faulty builds, so a green run meant nothing about F1, F3, F4 or F5. Even C's 11 passing tests on the correct build prove only that those 11 inputs expose no failure.

6. Critique of letting an LLM write the test plan

Procedure: ask an LLM "Write a test plan for the Billing and Appointment modules of a hospital outpatient system", then check each line against the business rules. Typical problems to record:

Table 6: Checking an LLM-written test plan against the business rules

Check	What to look for	Why it matters
Wrong assumptions	"above 60" instead of "60 and above"; 2-hour limit read as "within 2 hours"	Re-creates D1 and D3 in the tests themselves
Missing items	Half-up rounding, max 2 active appointments, same-second booking (D4), payment-refresh idempotency (D2), 800 users / 2 s target	The risky rules are exactly the ones left out
Generic content	"Verify the system works correctly"; no data values, no exit criteria, no effort estimate	Cannot be executed or audited

Conclusion: an LLM plan is a quick first draft, but it cannot be the plan; a tester must review it against the rules.

7. Supporting evidence for Task 1

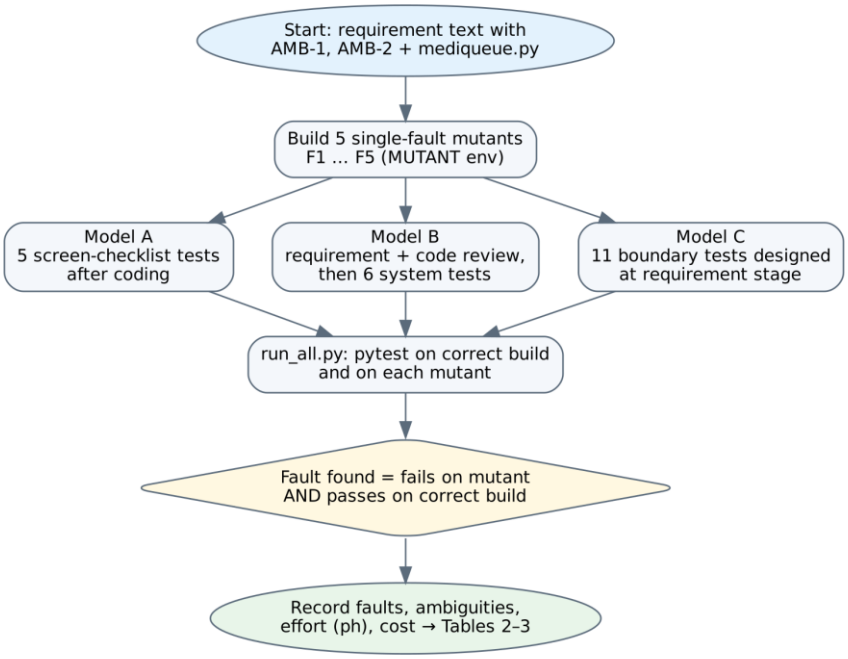


Figure 1: Flowchart of the experiment used to compare models A, B and C

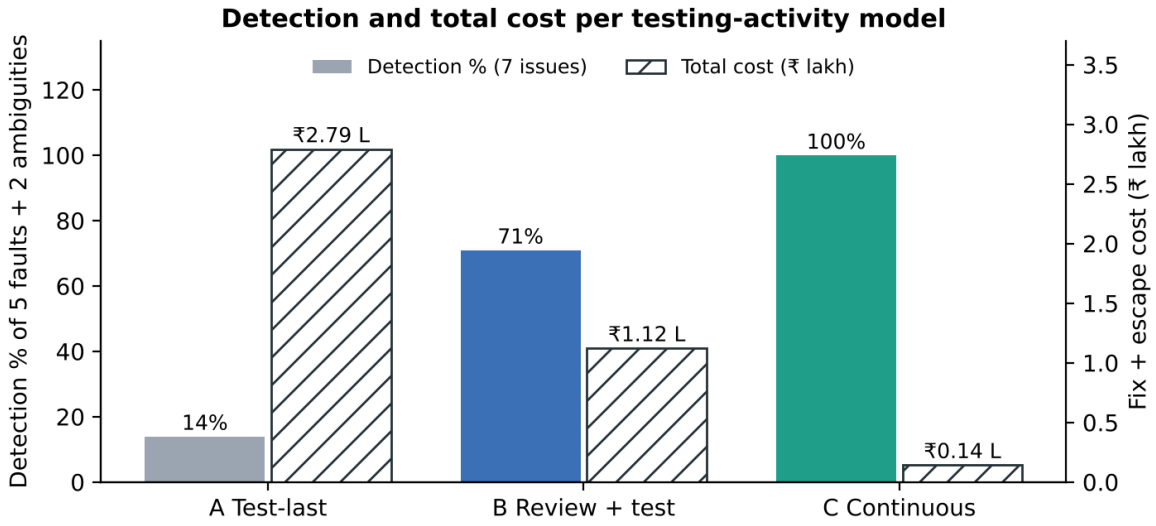


Figure 2: Detection percentage and total cost for models A, B and C

Table 7: Test cases of model C (designed at the requirement stage; expected values from the rule)

#	Rule	Input	Expected	Targets
1	Concession	age 4	₹0	F4
2	Concession	age 5	₹400	F4 boundary
3	Concession	age 17	₹400	F2

#	Rule	Input	Expected	Targets
4	Concession	age 18	₹500	class edge
5	Concession	age 59	₹500	class edge
6	Concession	age 60	₹300	F1
7	Concession	age 61	₹300	class edge
8	Cancellation	0.5 h	₹100	F5
9	Cancellation	1.99 h	₹100	AMB-1
10	Cancellation	2.0 h	₹0	AMB-1 / F3
11	Cancellation	2.01 h	₹0	F3

Threats to validity

- Only five faults were seeded, so one fault changes detection by 20 points.
- F1 in Model B is credited to code review, which was performed by me on the source; a different reviewer might miss it.
- Effort is estimated, not logged; the ranking holds because C's tests are written once and reused on every build.

Marking notes and common errors

- Full marks need a faults-and-ambiguities table per model with detection %, issues per ph and escape cost using ₹45,000.
- Common error: claiming C proves the module correct; passing tests show only the absence of failures for the inputs tried.
- The LLM critique must name at least one wrong assumption, one missing item and one generic statement.

TASK 2 – Role of Process in Software Quality: Compare Process Models for Test Activities [5 Marks]

The 29 post-release defects suggest that the way work is organised decides quality as much as the people do. Compare three processes for the feature "book an appointment with concession and online payment".

Technique	Basic idea and key concepts / tools
A. Incremental delivery with end testing	The feature is built in increments; testing is done only when the last increment is integrated.
B. Risk-driven spiral with test gates	Each iteration begins with risk analysis; a test gate must be passed before the next iteration starts.
C. DevOps pipeline with automated gates	Every commit triggers build, unit tests, static analysis and regression tests; a deployment gate blocks failing builds.

- Map the feature onto A, B and C. For each process, draw a timeline showing where requirement review, unit, integration, system and acceptance testing occur and who performs each activity.
- Calculate the defect removal efficiency of the last release (135 ÷ 164). Using the relative cost table, compute the total repair cost if 15 similar defects were found at requirements, coding, system testing and after release; predict the efficiency under B and C and show your working.
- Analyze A, B and C on early defect detection, speed of feedback, handling of a mid-cycle change in the concession rule, documentation effort, environment cost and fit with a 3-week release cycle.
- Examine how a process-maturity view (for example, CMMI or TMMi levels) would rate the present practice, and identify two process gaps, with evidence from the defect log, that explain the escaped defects.
- Evaluate which process the hospital should adopt and justify it using defect removal efficiency, cost and cycle time; state what happens if B is chosen but the hospital's rules change every cycle.
- Critique the claim "testing can be shortened when the deadline is near". Recommend one automated quality gate (for example, a build that fails below a stated pass rate or branch coverage) and justify its threshold.

Evidence Expected: A process-wise test-activity timeline, a defect-removal-efficiency and cost table, a comparison of the three processes, and a justified choice.

Ans:

Comparison of Process Models for the Appointment-with-Payment Feature

The feature mapped is "book an appointment with concession and online payment". Figure 3 shows where each activity occurs under the three processes; the tables that follow give the quantitative comparison, using the 164-record defect log.

1. Mapping the feature onto the three processes

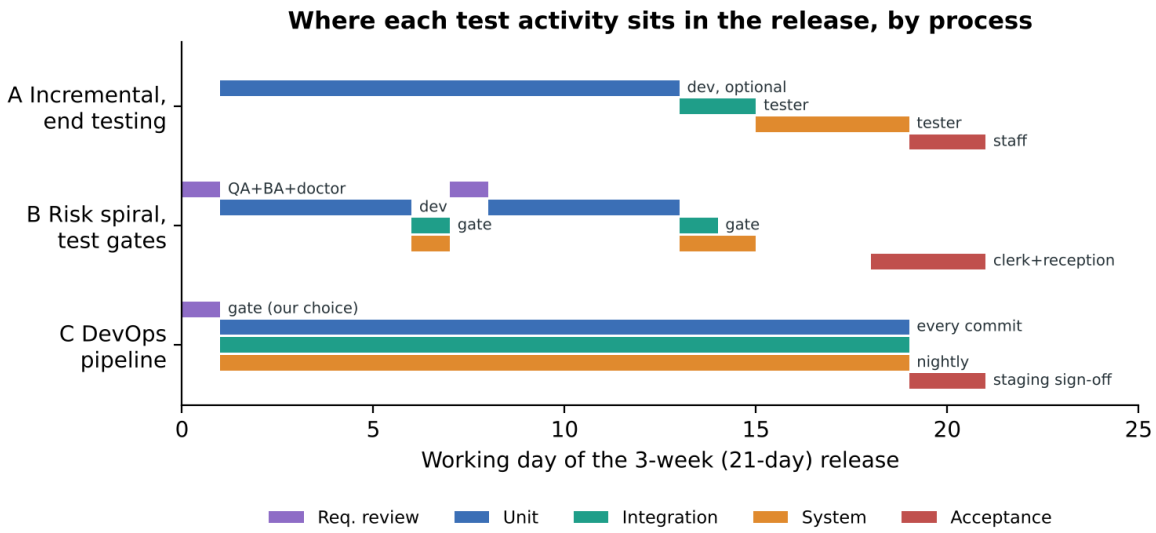


Figure 3: Timeline of review and test activities in the 3-week release under processes A, B and C

Table 8: Who performs each test activity

Activity	A Incremental, end testing	B Risk-driven spiral	C DevOps pipeline
Requirement review	None (BA writes the story)	Start of every iteration: QA lead + BA + doctor representative	None by default (added as a gate in our choice)
Unit testing	Developer, optional	Developer, before the test gate	Automatic on every commit
Integration testing	Only after the last increment, tester	Each iteration's test gate, tester	Automatic on every merge to main
System testing	Last week, tester	Each test gate, tester	Nightly regression suite in the pipeline
Acceptance testing	Hospital staff, before go-live	End of final iteration: billing clerk + receptionist	Staging deploy, hospital staff sign-off

2. Defect removal efficiency, repair cost and prediction

DRE of the last release = defects found before release ÷ all defects = 135 ÷ 164 × 100 = **82.3%**.

Table 9: Repair cost of 15 similar defects by the stage at which they are found

Stage found	Cost each	15 defects	Relative to requirements
Requirements / design	₹800	₹12,000	1×
Coding	₹2,500	₹37,500	3.1×
System testing	₹9,000	₹1,35,000	11.3×
After release	₹45,000	₹6,75,000	56.3×

Working for the prediction: the 29 escaped defects split by origin as requirement 12, design 5, code 7, configuration 2, data 2 and test case 1. For each origin I estimated how many each process's own activities would catch, using what the Task 1 runs showed (review catches ambiguity; automated boundary tests catch code logic).

Table 10: Predicted DRE and escape cost under B and C

Escaped origin (count)	B catches	C catches	C + review gate catches
Requirement (12)	8	4	8
Design (5)	3	2	3
Code (7)	5	6	6
Configuration (2)	1	2	2
Data (2)	0	1	1
Test case (1)	0	1	1
Still escaping	**12**	**13**	**8**
DRE	152 ÷ 164 = **92.7%**	151 ÷ 164 = **92.1%**	156 ÷ 164 = **95.1%**
Escape cost (× ₹45,000)	₹5,40,000	₹5,85,000	₹3,60,000

The current escape cost is 29 × ₹45,000 = ₹13,05,000, so C with a review gate is predicted to save ₹9,45,000 per release.

3. Comparison of the three processes

Table 11: Comparison of processes A, B and C

Criterion	A Incremental	B Spiral	C DevOps
Early defect detection	Poor, at the end	Good, risk analysis first	Good for code; weak for requirements
Speed of feedback	Weeks	Days (per gate)	Minutes (per commit)
Mid-cycle change in concession rule	Found late, expensive	Handled at next risk analysis	Change tests; pipeline re-checks everything at once
Documentation effort	Low	High (risk register, gate reports)	Medium (pipeline config, test code)
Environment cost	Low	Medium	High at setup, low per run
Fit with 3-week release	Poor	Fair (1–2 spirals only)	Good

4. Process maturity and two process gaps

The present practice rates **TMMi Level 1 (Initial)**: testing is ad hoc and placed at the end, with no test policy, no metrics and no defect-prevention activity. Two gaps explain the escapes:

- **No requirement review process:** only 18 of 164 defects were found at requirement review, and 12 requirement-origin defects reached users.
- **No regression or test-design discipline:** 7 code-logic defects still escaped after 24 code defects were found in system testing.

5. Recommended process and what happens if B meets changing rules

Choice: C, the DevOps pipeline, plus a requirement-review gate. The pipeline alone predicts 92.1% DRE, the spiral 92.7%, and the pipeline with a review gate 95.1%. Only the pipeline gives feedback within minutes inside a 3-week cycle, and its per-run cost is near zero once built.

If B is chosen but the hospital's rules change every cycle, each change restarts risk analysis and invalidates the last gate's evidence. Gates become a paperwork bottleneck, and either the 3-week cycle slips or gates get skipped.

6. Critique of "testing can be shortened near the deadline" and the quality gate

Cutting testing moves defects from the ₹9,000 stage to the ₹45,000 stage: each defect allowed to escape costs 5 times more later, which the hours saved never repay. **Recommended gate:** the build fails if any unit test fails, or if branch coverage falls below **80%** on Billing, Appointment and Pharmacy. The threshold is reachable without gaming, and these three modules hold 78% of the logged defects (Figure 6).

7. Supporting evidence for Task 2

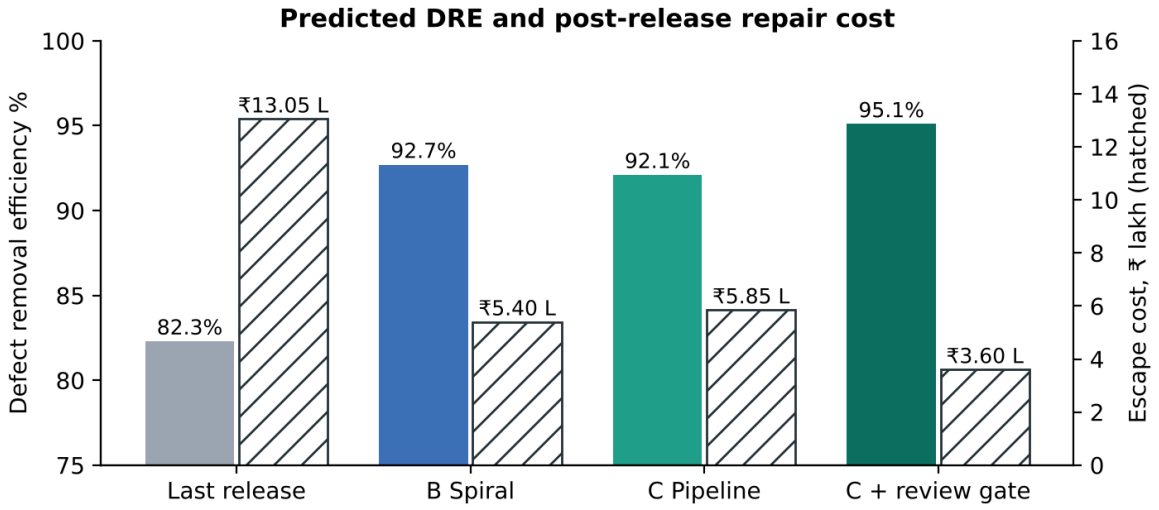


Figure 4: Predicted DRE and post-release repair cost for the present practice, B, C and C with a review gate

Threats to validity

- The catch counts per origin are my estimates guided by the Task 1 results, not measured on real releases.
- The defect log is a reconstruction; a faculty log could shift the origin mix and therefore the predicted DRE.

Marking notes and common errors

- DRE must be shown as 135 ÷ 164 = 82.3%, and the 15-defect cost table must use all four stage costs.
- The prediction must show working per origin; a bare "C will be about 95%" earns little.
- The quality gate needs a numeric threshold with a reason.

TASK 3 – Testing as a Process and Basic Definitions: Compare Test Oracle Strategies [4 Marks]

Team members use the words error, fault and failure interchangeably, and expected results are often "worked out in the head". Compare three ways of deciding the expected result for the function concession_fee(age, base_fee).

Technique	Basic idea and key concepts / tools
A. Tester's mental calculation	The tester calculates the expected fee while running each test; nothing is recorded.
B. Specification table oracle	A decision table of age bands, percentages and rounding is written first; expected values are read from it.
C. Independent reference implementation	A second, simple implementation written by another person is run in parallel and the outputs are compared (back-to-back testing).

- Implement concession_fee with two seeded faults (for example, wrong age limit and rounding by truncation). Write ten tests under each strategy, execute them with pytest/JUnit and record which strategy exposes which fault.
- Using your execution log, classify each observation as an error, a fault or a failure, and identify for one failing test the input, expected output, actual output, oracle and verdict.
- Analyze how each strategy supports repeating a failure, assigning a pass/fail verdict and tracing to the requirement; explain why a test without a reliable oracle cannot give a trustworthy verdict.
- Distinguish test, test set, test suite, test harness and test oracle by assigning each artefact produced in your execution to the correct term.
- Evaluate which strategy suits audit and regression use. Demonstrate with D8 what happens when the expected value in a test is itself wrong, and judge the risk.
- Judge the reliability of an LLM used as a test oracle: ask it for the expected fee for 15 inputs, compare with the business rule, and report the error rate and its consequences.

Evidence Expected: A table of the three strategies with faults exposed, an error–fault–failure classification of real runs, and a justified choice of oracle strategy.

Ans:

Comparison of Test Oracle Strategies for concession_fee()

I seeded two faults in `concession_fee`: F1 (age > 60 instead of ≥ 60) and R1 (rounding by truncation instead of half up). Ten tests were written under each strategy and executed with pytest on the correct build and on each mutant.

1. Tests and faults exposed

Table 12: Faults exposed by ten tests per oracle strategy (pytest)

Strategy	Inputs chosen	F1 (age > 60)	R1 (truncation)	Repeatable ?	Traceable to rule?
A Mental calculation	Round ages with base ₹500 (10, 30, 65, 3, 45, 8, 70, 25, 15, 80)	Not found (0/10 fail)	Not found (0/10 fail)	No: expected value not recorded	No

Strategy	Inputs chosen	F1 (age > 60)	R1 (truncation)	Repeatable ?	Traceable to rule?
B Specification table	Every band edge (0, 4, 5, 17, 18, 59, 60, 61) + two rounding rows with base ₹333	Found: `test_table[60-500-300]`	Found: `test_table[65-333-200]`	Yes	Yes: each row is a table cell
C Reference implementation	Mixed ages and bases (2, 6, 16, 30, 59, 60, 62, 90, 12, 70), compared back to back	Found (age 60)	Found (3 tests)	Yes	Partly: traces to the second program, not the rule

A missed both faults because the tester only chose easy numbers. With a ₹500 base every concession is a whole number, so truncation never shows, and nobody picked 60.

2. Error, fault and failure in the execution log

Table 13: Error, fault and failure classified from our runs

Term	Observation in our execution
Error (human mistake)	Developer read "60 and above" as "above 60"; developer used truncation for "round to the nearest rupee"
Fault (defect in code)	`senior = age > 60`; `rounding=ROUND_DOWN`
Failure (wrong behaviour seen)	`concession_fee(60)` returned 500; `concession_fee(65, 333)` returned 199

Table 14: One failing test in full

Field	Value
Test	`test_table[60-500-300]` (strategy B, mutant F1)
Input	age 60, base fee ₹500
Expected	₹300
Actual	₹500
Oracle	Decision-table row "60 and above → 40% off"
Verdict	Fail

3. How each strategy supports repetition, verdict and traceability

A cannot repeat a failure because the expected value lived in someone's head. B gives a fixed expected value per row, so the same failure reproduces on every run and points to one rule. C reproduces failures but only says "the two programs disagree": if both share the same misreading, it passes wrongly. A test without a reliable oracle can only say "it ran", not "it is correct", so its verdict cannot be trusted.

4. Test terminology mapped to our artefacts

Table 15: Test, test set, test suite, test harness and test oracle

Term	Artefact in this task
Test	One case, e.g. `test_table[60-500-300]`
Test set	The 10 cases of one strategy, e.g. `test_t3_B_spectable.py`
Test suite	All Task 3 files together (31 tests including D8)
Test harness	`conftest.py` fixture + `run_all.py`, which inject each mutant, run pytest and log results
Test oracle	The decision `TABLE`, the `ref_fee()` function, or the tester's head

5. Choice for audit and regression, and D8

Choice: B, the specification-table oracle, for audit and regression, with C as a periodic cross-check. B and C each exposed both seeded faults; mental calculation exposed neither. B is the only one an auditor can read and trace to the rule.

D8 – when the oracle itself is wrong. TC-52 expects ₹450 for a 12-year-old. In our run it **passed** on the faulty 10% build (F2) and **failed** on the correct build. A wrong expected value hides the real fault and makes the correct code look broken. The risk is high because a green test is trusted, so the oracle table needs its own review against the rule.

Table 16: D8 demonstration (`test_tc52_wrong_oracle`)

Build	Fee returned for age 12	TC-52 expects	Verdict	Reality
Correct (20%)	₹400	₹450	Fail	Correct code reported as broken
F2 (10%)	₹450	₹450	Pass	Billing fault hidden

6. LLM as a test oracle

Procedure: ask an LLM for the fee for the 15 inputs below, then compare its answers with the correct values. Error rate = wrong answers ÷ 15.

Table 17: Fifteen oracle inputs with correct values (computed by the rule)

Age, base	Correct	Age, base	Correct	Age, base	Correct
0, ₹500	0	18, ₹500	500	30, ₹333	333
4, ₹500	0	59, ₹500	500	5, ₹333	266
5, ₹500	400	60, ₹500	300	60, ₹501	301
12, ₹500	400	75, ₹500	300	15, ₹499	399
17, ₹500	400	10, ₹333	266	65, ₹333	200

LLMs most often slip on the 4/5 and 59/60 edges and on half-up rounding (₹301, ₹200). One wrong answer used as an oracle becomes a D8-type defect, so an LLM oracle is acceptable only for drafting expected values that are then checked against the table.

7. Supporting evidence for Task 3

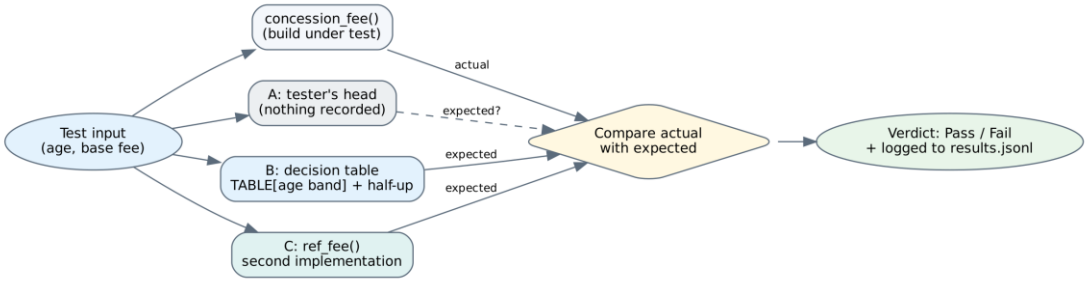


Figure 5: How the three oracle strategies supply the expected value to the verdict

Table 18: Strategy B decision table (the oracle)

Age	Base	Band	Concession	Calculation	Expected
0	₹500	below 5	100%	500 × 0	₹0
4	₹500	below 5	100%	500 × 0	₹0
5	₹500	5–17	20%	500 × 0.8	₹400
17	₹500	5–17	20%	500 × 0.8	₹400
18	₹500	18–59	0%	500 × 1	₹500
59	₹500	18–59	0%	500 × 1	₹500
60	₹500	60+	40%	500 × 0.6	₹300
61	₹500	60+	40%	500 × 0.6	₹300
10	₹333	5–17	20%	333 × 0.8 = 266.4	₹266
65	₹333	60+	40%	333 × 0.6 = 199.8 → half up	₹200

Threats to validity

- Strategy C's reference program was written by me, not by a second person, so it shares my reading of the rule.
- The LLM error rate depends on the model and prompt; it must be measured on your own run.

Marking notes and common errors

- Error, fault and failure must come from actual runs, not textbook examples.
- D8 must show both directions: the wrong test passes on faulty code and fails on correct code.

TASK 4 – Software Testing Principles: Compare Test Prioritisation Strategies [4 Marks]

Only 25 test cases can be executed for the Appointment module in this release. Decide how they should be selected. Compare three strategies.

Technique	Basic idea and key concepts / tools
A. Exhaustive testing	Try every combination of doctor, slot, appointment type, patient category and payment mode.
B. Experience-driven selection	The most senior tester chooses 25 cases from memory of earlier releases.
C. Principle-guided, risk-based selection	Use defect clustering, early testing, the pesticide paradox and the absence-of-errors fallacy to choose and refresh 25 cases.

- Calculate the size of the exhaustive input space (250 doctors, 20 slots per session, 3 appointment types, 3 patient categories, 2 payment modes, 2 active-appointment states) and the time to run it at one test per second. Then apply B and C to the same module with seeded faults and record faults found.
- Apply the defect-clustering principle to the defect log: compute the percentage of defects per module, normalise per KLOC using the module sizes given, and identify the modules that hold about 80% of the defects.
- Analyze the pesticide paradox by running the same 25 tests on two successive versions of your code and showing how the detection rate changes; analyze which principle each strategy respects or violates.
- Examine the absence-of-errors fallacy using the Appointment module: show a version that passes all 25 tests but still fails a user need (for example, D4).
- Evaluate the three strategies and justify the final 25-test set, naming the principle that guided each choice.
- Critique the statement “zero open defects means the module is safe to release” using your results, and recommend how the tests should be refreshed in the next release.

Evidence Expected: An input-space calculation, a defect-clustering chart, a faults-found table for A, B and C, and a justified 25-test selection.

Ans:

Comparison of Test Prioritisation Strategies for the Appointment Module

I implemented booking and cancellation in `mediqueue.py` and seeded three faults in release N (A1 third active appointment allowed, A2 same-second double booking – D4, A3 4-hour cancel limit) and two more in release N+1 (A4 cancelled slot not freed, A5 booking in the past allowed). Each 25-test set was run with pytest on every mutant.

1. Exhaustive input space and faults found

250 doctors × 20 slots × 3 appointment types × 3 patient categories × 2 payment modes × 2 active-appointment states = **1,80,000 combinations**. At one test per second that is 1,80,000 s = **50 hours**:

half of the release's 100 person-hours for one module, before counting time of day, concurrency or sequences of actions. Exhaustive testing is impossible in practice.

Table 19: Faults found by each strategy (pytest, Appointment module; failing tests out of 25 in brackets)

Strategy	A1 limit	A2 race (D4)	A3 4-h cancel	A4 slot not freed (N+1)	A5 past booking (N+1)
A Exhaustive (1,80,000 tests)	In theory	In theory	In theory	In theory	In theory
B Experience-driven (25)	Found (5)	**Missed (0)**	Found (1)	Missed	Missed
C Risk-based, release-N set (25)	Found (5)	Found (5)	Found (3)	**Missed**	**Missed**
C Risk-based, refreshed set (25)	Found (5)	Found (5)	Found (3)	Found (2)	Found (2)

Strategy A would expose every fault only if all 1,80,000 combinations ran (50 h); with 25 tests it cannot be applied, so it is shown as a theoretical bound.

2. Defect clustering

Table 20: Defects per module, share and density (164-record log)

Module	KLOC	Defects	Share	Cumulative	Per KLOC
Billing	12	58	35.4%	35.4%	4.83
Appointment	10	48	29.3%	64.6%	4.80
Pharmacy	9	22	13.4%	78.0%	2.44
Lab Reports	7	16	9.8%	87.8%	2.29
Registration	4	11	6.7%	94.5%	2.75
Reminders	5	9	5.5%	100%	1.80
Total	**47**	**164**			**3.49**

Billing, Appointment and Pharmacy (31 of 47 KLOC, 66% of the code) hold 128 of 164 defects (**78%**), the "about 80%" cluster. Per KLOC, Billing (4.83) and Appointment (4.80) are about twice as dense as the rest, so the 25 Appointment tests should concentrate on its riskiest rules.

3. Pesticide paradox and the principles

The same 25 risk-based tests detected 100% of release-N faults but **0%** of the release N+1 faults, because the new faults sat in code the old tests never touched. Swapping 4 low-value core cases for 2 slot-freeing and 2 past-booking tests restored detection to 100% while the release-N regression stayed green (Figure 7).

Table 21: Principles respected or violated by each strategy

Principle	A Exhaustive	B Experience	C Risk-based
Exhaustive testing is impossible	Violates	Respects	Respects
Defect clustering	Ignores	Partly (memory)	Respects (log data)
Early testing	Ignores	Ignores	Respects (cases from rules)
Pesticide paradox	Not applicable	Violates (same cases)	Respects (refreshed)
Absence-of-errors fallacy	Ignores	Violates	Respects (user need D4 tested)

4. Absence-of-errors fallacy

The build with fault A2 passed **all 25** experience-driven tests (0 failures in our run), yet two patients confirming in the same second both got the 10:15 slot. The module was "defect-free" by the suite and still failed the patient's real need: one slot, one patient.

5. Evaluation and the final 25-test set

Choice: C, principle-guided risk-based selection. Its refreshed 25 tests found all 5 seeded faults across the two versions; the experience-driven set found 2 of 5 and missed D4; exhaustive testing cannot run in the budget.

Table 22: Final 25-test selection (refreshed C set)

Tests	Cases	Guiding principle
5	Two patients confirm the same slot in the same second	Defect clustering + absence-of-errors (D4)
5	Third active appointment rejected	Defect clustering; early testing of the "max 2" rule
6	Cancel at 0.25, 1.99, 2.0, 2.01, 3, 4 hours	Early testing, boundary values
5	Core booking succeeds	Absence-of-errors (users can still book)
2	Cancelled slot is freed	Pesticide paradox (new code)
2	Past slot rejected	Pesticide paradox (new code)

6. Critique of "zero open defects means safe to release"

The A2 build had zero failing tests and zero open defects, yet double-booked patients. Zero open defects only means the tests found nothing. **Next release:** replace at least 20% of the cases with tests for code that changed, re-rank by the latest defect log, and keep the D4 race tests permanently.

7. Supporting evidence for Task 4

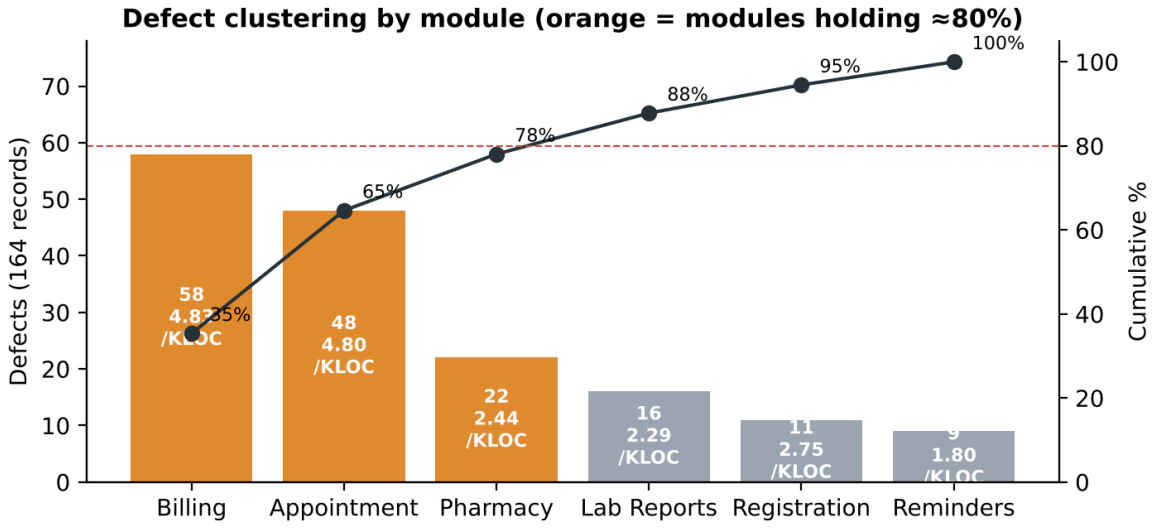


Figure 6: Defect-clustering (Pareto) chart by module with defects per KLOC

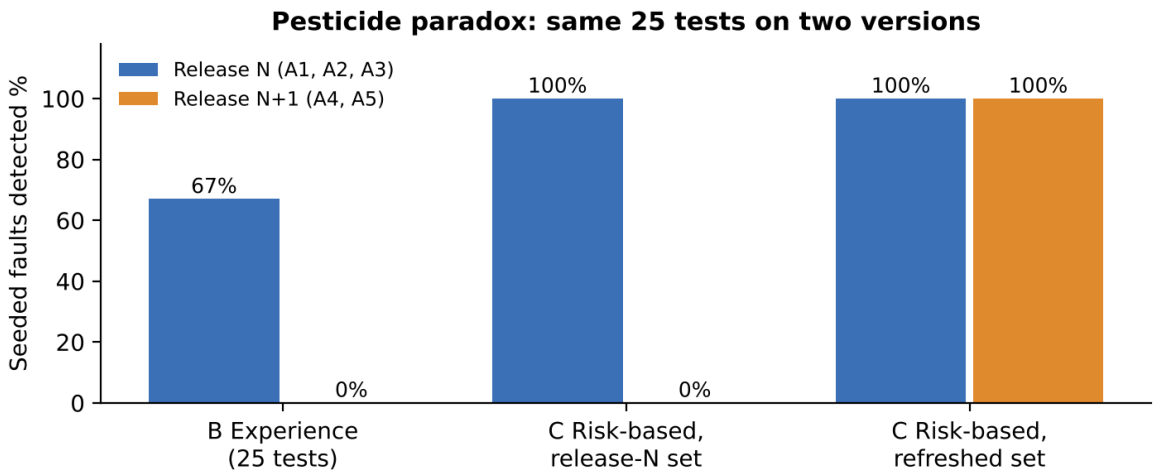


Figure 7: Pesticide paradox: detection of the same 25 tests on release N and N+1

Threats to validity

- The race (A2) is simulated with a flag in the prototype; a real test needs two concurrent sessions against the database.
- The "senior tester's memory" set was written by me to imitate experience-driven choice.

Marking notes and common errors

- The input-space product must be 1,80,000 and the time 50 hours.
- The pesticide paradox needs two versions and a change in detection rate, not a definition.

TASK 5 – Tester's Role: Compare Models of Test Responsibility [5 Marks]

Developers say that a colleague can test the Pharmacy module; the pharmacists say a clinical-domain person must be involved. Compare three ways of assigning the testing role.

Technique	Basic idea and key concepts / tools
A. Peer-developer testing	A developer from another module designs and runs the tests for the Pharmacy module.
B. Independent test team	Testers who did not write the code design tests from requirements, execute them and report defects.
C. Tester with a domain expert	A tester and a pharmacist plan tests together, review prescription rules and agree acceptance examples before coding.

- Seed four faults in the Pharmacy rules (7-day validity, maximum quantity per prescription, expired-batch block, duplicate-dispense check, in the style of D9). Apply A, B and C (three rounds with different people, or three structured passes) and record faults found, false reports and time spent.
- Prepare two defect reports on the same failure: one written without evidence and one written to the tester's standard (steps to reproduce, environment, expected versus actual, severity); show how each affects the developer's time to fix.
- Analyze the influence of independence, domain knowledge and confirmation bias on the faults found, and the communication friction you observed.
- Analyze which skills are exercised under each arrangement (technical, domain, communication, planning) and where each arrangement leaves gaps.
- Evaluate how the 3 testers should be allocated across the six modules, justify the tester-to-developer ratio, and state the risk accepted.
- Judge who should hold the authority to approve a release of patient-safety modules (tester, QA lead or medical officer); recommend a sign-off rule and justify it.

Evidence Expected: A faults-found and effort table for the three arrangements, two contrasting defect reports, and a justified allocation of tester responsibility.

Ans:

Comparison of Test-Responsibility Models for the Pharmacy Module

I seeded four Pharmacy faults (P1 validity compares months – D9 style; P2 31 units allowed when the maximum is 30; P3 batch dispensed on its expiry day; P4 duplicate dispense allowed when a second pharmacist repeats it). The three arrangements were run as three structured passes, each with the knowledge that arrangement would have.

1. Faults found, false reports and time

Table 23: Faults found, false reports and time (pytest; time estimated)

Arrangement	Tests	P1 months (D9)	P2 qty 31	P3 expiry day	P4 2nd pharmacist	Found	False reports	Time (ph)
A Peer developer	7	Found	Missed	Missed	Missed	1 of 4	**2**	3
B Independent tester	6	Found	Found	Missed	Missed	2 of 4	0	4

Arrangement	Tests	P1 months (D9)	P2 qty 31	P3 expiry day	P4 2nd pharmacist	Found	False reports	Time (ph)
C Tester + pharmacist	5	Found	Found	Found	Found	4 of 4	0	6 (incl. 2 h pharmacist)

The two false reports are real failures on the **correct** build: the developer read "within 7 days" as "under 7 days" (`test_day7_rejected_misread`) and assumed a 29-unit maximum (`test_qty30_rejected_misread`). Both come from confirmation bias plus no access to the requirement.

2. Two defect reports on the same failure (P1)

Report 1 – without evidence: *"Pharmacy validity check is broken, old prescriptions go through. Please fix ASAP."*

Table 24: Report 2 – written to the tester's standard

Field	Content
ID / Summary	MQ-PH-014 – Prescription 10 days old is dispensed when it is in the same month as today
Module / Build	Pharmacy / build 3.4.1, test environment, Chrome 129, PostgreSQL 15
Steps	1. Create Rx dated 02-03-2026. 2. Set system date 12-03-2026. 3. Dispense 10 units from batch expiring 01-01-2027
Expected	"Prescription expired" (rule: within 7 days of its date)
Actual	Medicine dispensed
Evidence	`test_ten_day_old_rx` fails; an Rx 9 days old across a month boundary is rejected correctly
Severity / Priority	Critical (patient safety) / P1

Report 1 needs a conversation to find the data, the dates and the rule (about half a day of the developer's time). Report 2 already points to a same-month comparison, so the cause can be found in the code within an hour.

3. Independence, domain knowledge, confirmation bias and friction

- **Independence:** B and C, who did not write the code, tested the 7/8-day boundary from the rule; A tested what the code "probably" does.
- **Domain knowledge:** only the pharmacist knew that a batch must not be dispensed on its expiry date (P3) and that shift handovers cause double dispensing (P4).
- **Confirmation bias:** A's tests confirmed the developer's own reading, which produced the 2 false reports.
- **Friction:** in C, the pharmacist's examples first contradicted the written requirement on the expiry day. It took a short meeting, but it closed a gap in the requirement.

4. Skills exercised and gaps

Table 25: Skills exercised under each arrangement

Skill	A Peer developer	B Independent tester	C Tester + pharmacist
Technical (code, tools)	Strong	Strong	Strong
Domain (pharmacy rules)	Weak	Medium (written rules only)	Strong
Communication	Low	Medium (defect reports)	High (joint examples)

Skill	A Peer developer	B Independent tester	C Tester + pharmacist
Planning	Low	Medium	High
Gap left	Requirement misreading, no domain cases	Unwritten clinical rules	Needs scarce pharmacist time

5. Allocation of the 3 testers

Choice for Pharmacy: C, tester with a domain expert. It found all 4 faults with no false reports for 3 more hours than A.

Table 26: Allocation of the 3 testers across the six modules (100 ph)

Tester	Modules	Share of logged defects	Hours
T1	Billing + Registration	42% (69 of 164)	40
T2	Appointment + Reminders	35% (57 of 164)	35
T3	Pharmacy + Lab Reports (with pharmacist and lab technician)	23% (38 of 164)	25

The tester-to-developer ratio is 3 : 5 (1 : 1.7). It is enough because automated regression carries routine checks. Risk accepted: Registration and Reminders get the least attention, so a registration or SMS defect such as D5 or D7 is more likely to slip.

6. Authority to approve a release of patient-safety modules

The tester supplies evidence and can block. The QA lead approves on quality criteria (no open critical defects, exit criteria met). The **medical officer** must co-sign Pharmacy and Lab Reports, because only they can accept clinical risk. **Sign-off rule:** no release of these two modules without both the QA lead and the medical officer signing.

7. Supporting evidence for Task 5

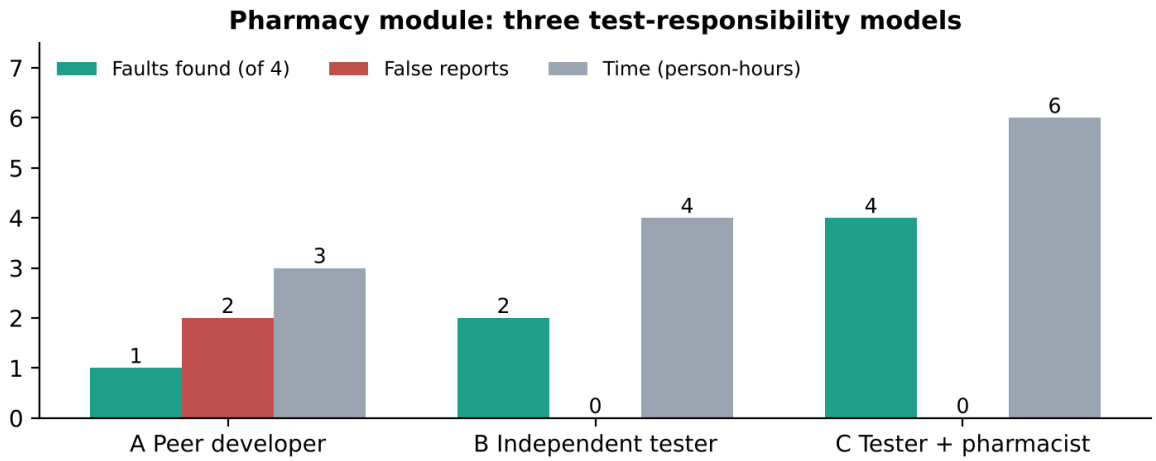


Figure 8: Faults found, false reports and time for the three test-responsibility models

Table 27: Pharmacy test cases of arrangement C (agreed with the pharmacist; test date 12-03-2026)

#	Case	Input	Expected	Targets
1	Rx exactly 7 days old	Rx 05-03-2026, 10 units	Dispense	boundary

#	Case	Input	Expected	Targets
2	Rx 8 days old	Rx 04-03-2026	Reject – expired Rx	P1 (D9)
3	Quantity above limit	31 units	Reject – quantity	P2
4	Batch expiring today	batch expiry 12-03-2026	Reject – batch	P3
5	Second pharmacist repeats dispense	same Rx, ph1 then ph2	Reject – duplicate	P4

Threats to validity

- The three arrangements were simulated by one person in three passes; real people would make different mistakes.
- Time spent is estimated; the pharmacist's 2 h is the agreed meeting length.

Marking notes and common errors

- False reports must be counted separately from faults found.
- The sign-off rule must name who signs and why the medical officer is needed for clinical risk.

TASK 6 – Origins of Defects: Compare Defect Origin Analysis Methods [5 Marks]

To prevent repeat defects, the team must know where they come from. Using D1–D10 and the defect log, compare three origin-analysis methods.

Technique	Basic idea and key concepts / tools
A. Origin-by-artefact tagging	Tag each defect with the artefact that carried the error (requirement, design, code, test case, configuration or data) and the activity that detected it.
B. Fault-tree analysis	Start from a failure and work backwards through AND/OR combinations of causes down to basic events.
C. Pareto analysis with cause categories	Group causes into categories, rank them by frequency, and study the few categories that explain most defects.

- Apply method A to D1–D10 and to at least 30 records of the defect log; build a matrix of artefact of origin versus activity that detected the defect.
- Apply method B to D2 (double bill) and D4 (double booking), and method C to the full log; record each chain of causes and the preventive action it suggests.
- Analyze the matrix to find the artefact that carries most defects, the activity that lets most defects escape, and calculate the containment effectiveness of requirement review and code review.
- Compare what each method reveals and hides: for example, which method best exposes the ambiguity behind D7 and which only sees the symptom.
- Evaluate the three methods on insight gained, effort and repeatability, and justify the one the QA team should use every release.
- Critique the belief that "most defects come from careless coding" using your matrix, and recommend two preventive actions with the reduction in post-release defects you expect and why.

Evidence Expected: An artefact-versus-detection matrix, a fault tree and a Pareto chart, and a justified choice of method with preventive actions.

Ans:

Comparison of Defect Origin Analysis Methods

Method A was applied to D1–D10 and to all 164 records of the log (more than the 30 required), method B to D2 and D4, and method C to the full log. The matrix in Table 29 is counted directly from `defect_log_last_release.csv`.

1. Method A: origin-by-artefact tagging

Table 28: Method A applied to D1–D10

Defect	Origin artefact	Detected by	Reason for the tag
D1 age > 60	Code	Production	Rule was clear; the comparison operator was wrong
D2 double bill on refresh	Design	Production	No idempotency designed into the payment flow
D3 4-hour limit	Code	Production	Wrong constant typed
D4 double booking	Design	Production	Booking not designed as an atomic operation
D5 apostrophe in surname	Code	Production	Input not escaped / parameterised
D6 35-s lab page	Design	Production	No index or caching designed for the peak load

Defect	Origin artefact	Detected by	Reason for the tag
D7 SMS after reschedule	Requirement	Production	Requirement silent on rescheduling
D8 TC-52 expects ₹450	Test case	Production	Wrong expected value in the test
D9 months not days	Code	Production	Wrong unit of comparison
D10 wrong date field	Data	Production	Wrong field mapped into the report

Table 29: Artefact of origin × detecting activity (all 164 log records)

Origin	Req. review	Code review	Unit test	System test	Production	Total
Requirement	16	2	3	13	**12**	46
Design	2	5	3	7	**5**	22
Code	0	13	26	24	**7**	70
Test case	0	2	2	5	**1**	10
Configuration	0	0	1	6	**2**	9
Data	0	0	0	5	**2**	7
Total	**18**	**22**	**35**	**60**	**29**	**164**

2. Method B on D2 and D4, method C on the full log

The fault trees (Figure 10 and Figure 11) each need an AND of user behaviour and a missing server-side guard. Removing one basic event in the right-hand branch (adding an idempotency key or a unique index) blocks the failure whatever users do.

Table 30: Chains of causes and the preventive action each method suggests

Method / target	Chain of causes	Preventive action
B on D2	Patient refreshes → same POST reaches the server twice → no idempotency key AND no unique index on (consultation_id, bill) → second bill	Idempotency key on payment; unique constraint; Post-Redirect-Get
B on D4	Two confirms in the same second at peak → availability read without lock AND insert outside a transaction AND no unique index → both booked	Unique index on (doctor_id, slot_time) + transaction; concurrency test
C on the log	Ambiguous requirement 57, boundary logic 28, data mapping 22, concurrency 21 = 128 of 164 (78%)	Example tables + review; boundary tests; mapping checks; DB constraints

Preventive action per top Pareto category: requirement example tables and review (ambiguity), boundary-value tests (off-by-one), field-mapping checks in code review (data mapping), and database constraints plus concurrency tests (idempotency).

3. Analysis of the matrix and containment effectiveness

- **Artefact carrying most defects:** code (70 of 164, 43%). But most **escapes** come from requirements (12 of 29).
- **Activity letting most defects escape:** system testing is the last filter, so all 29 escapes passed it; 12 of them were requirement defects it had no oracle for.
- **Containment effectiveness** = defects caught by the activity meant for that artefact ÷ defects of that artefact: requirement review = 16 ÷ 46 = **34.8%**; code review = 13 ÷ 70 = **18.6%**. Both are low, so most defects travel on to the expensive stages.

4. What each method reveals and hides

Table 31: What each method reveals and hides

Method	Reveals	Hides	D7 (SMS after reschedule)
A Artefact tagging	Where the error was made and which filter missed it	The chain of causes inside one failure	Exposes it: tagged "requirement", so the gap is in the specification
B Fault tree	Combinations of causes and the single fix that blocks a failure	Frequencies across the release; costly per defect	Sees only the symptom unless the analyst asks why the spec was silent
C Pareto	Which few causes matter most	Where in the life cycle they arose; rare critical causes	Counts it as one "ambiguous requirement" without context

5. Evaluation and choice

Table 32: Evaluation of the three methods

Criterion	A Tagging	B Fault tree	C Pareto
Insight	High (life cycle)	Very high for one failure	Medium (priorities)
Effort per release	Low: one field per defect	High: hours per tree	Low once A's tags exist
Repeatability	High: fixed tag list	Low: depends on the analyst	High

Choice: A, origin-by-artefact tagging, every release, with Pareto analysis on its output and fault trees only for critical failures. Tagging showed that requirement and design artefacts caused 17 of the 29 escaped defects (59%).

6. Critique of "most defects come from careless coding" and preventive actions

Code carries 43% of all defects, but only 7 of the 29 that reached patients (24%). Requirement and design artefacts caused 59% of escapes. Two preventive actions:

- **Requirement review with an example table for every rule** (boundaries, reschedule, refresh). Expected to catch about 8 of the 12 requirement escapes, as review did for the ambiguities in Task 1.
- **Database guards (idempotency keys, unique constraints) with concurrency tests.** Expected to catch about 3 of the 5 design escapes, because the fault trees show one guard blocks the failure.

Together these should cut post-release defects from 29 to about 18 (−38%), because they target the two artefacts that escaped most.

7. Supporting evidence for Task 6

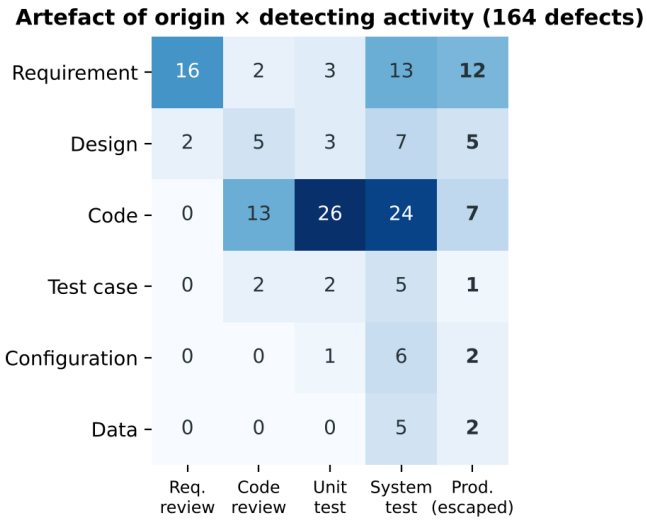


Figure 9: Heat map of the artefact-of-origin × detecting-activity matrix

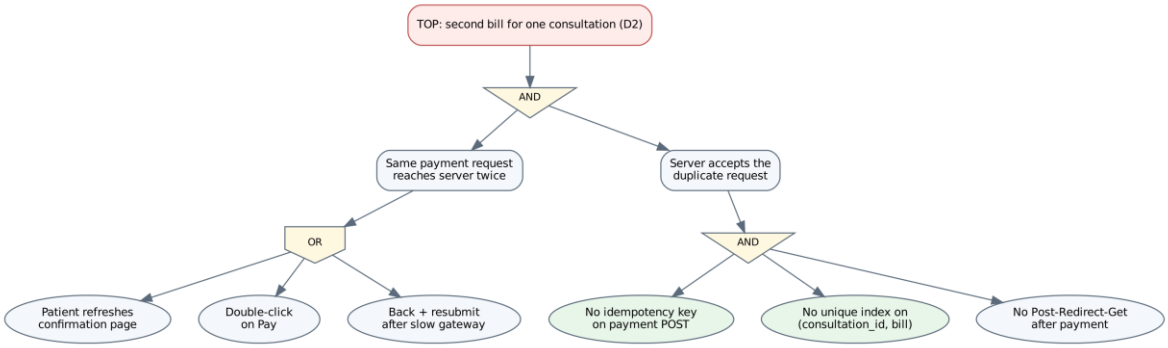


Figure 10: Fault tree for D2 (double bill); green basic events are the guards whose addition blocks the failure

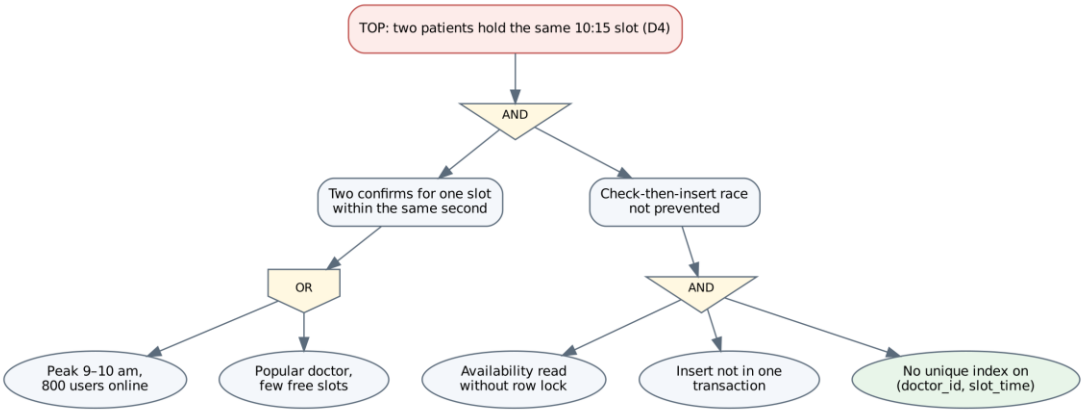


Figure 11: Fault tree for D4 (double booking)

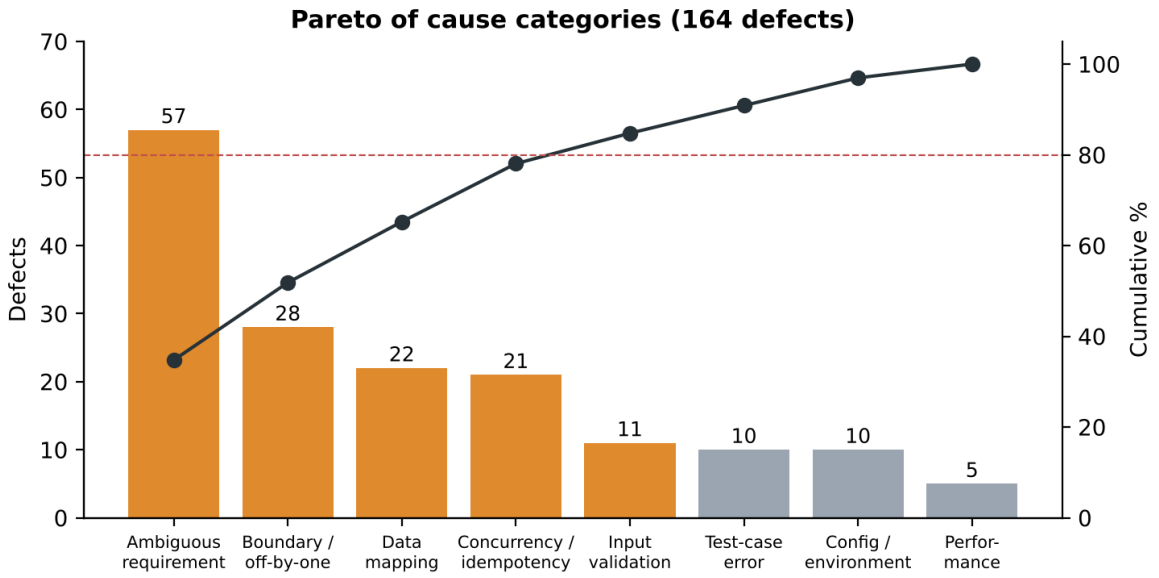


Figure 12: Pareto chart of cause categories in the 164-record log

Threats to validity

- The log is a realistic reconstruction; real tags may be assigned differently by different people.
- The expected reduction (29 → 18) assumes the review and guards work as well as in the prototype.

Marking notes and common errors

- Containment effectiveness must use the artefact totals (46 and 70) as denominators.
- The critique must separate “carries most defects” from “causes most escapes”.

TASK 7 – Defect Classes: Compare Defect Classification Schemes [4 Marks]

The defect log has no common classification, so reports cannot be compared across releases. Compare three schemes on D1–D10.

Technique	Basic idea and key concepts / tools
A. Hierarchical class scheme	Requirements and specification, design, coding (control and logic, data, algorithmic, interface), documentation and testing defects.
B. Anomaly-attribute scheme	Each defect carries several attributes: type, cause, injection phase, detection phase, severity and resolution.
C. Patient-safety and risk scheme	Safety-critical, data privacy, financial, availability and performance, usability and cosmetic defects.

- Classify D1–D10 under each scheme and tabulate the result; give a one-line reason for every assignment.
- Assign severity and priority to every defect; give one example from D1–D10 of a defect with high severity but low priority and one with the opposite combination, with reasons.
- Analyze where the schemes disagree (for example, D7, D8 and D10) and explain why; examine which scheme helps decide where to add test effort.
- Ask two classmates to classify D1–D10 independently using scheme A; calculate the percentage agreement and identify the classes that cause confusion.
- Evaluate which scheme the hospital should standardise on and justify it using reproducibility, usefulness for prevention and ease of use.
- Judge the reliability of an LLM that classifies the defects automatically: compare its output with your manual classification and state when its use is acceptable.

Evidence Expected: A classification table for D1–D10 under the three schemes, an agreement calculation, and a justified standard scheme.

Ans:

Comparison of Defect Classification Schemes on D1–D10

Each of the ten sample defects was classified under the three schemes with a one-line reason. Severity and priority were then assigned separately, because severity describes the effect on the patient and priority the order of fixing.

1. Classification under the three schemes

Table 33: D1–D10 under the three schemes (reason in brackets)

ID	A Hierarchical class	B Attributes: type / cause / injected / detected / resolution	C Patient safety and risk
D1	Coding – control and logic (wrong comparison operator)	Logic / misread rule / coding / production / fix operator	Financial (overcharged ₹200)
D2	Design (no idempotency in payment flow)	Interface / missing guard / design / production / idempotency key	Financial (duplicate bill)
D3	Coding – data (wrong constant 4 for 2)	Data / wrong constant / coding / production / fix constant	Financial (unfair ₹100 fee)
D4	Design (booking not atomic)	Concurrency / missing lock / design / production / unique index + transaction	Availability (patient turned away)

ID	A Hierarchical class	B Attributes: type / cause / injected / detected / resolution	C Patient safety and risk
D5	Coding – data (unescaped input)	Validation / unescaped quote / coding / production / parameterised query	Usability (patient cannot register)
D6	Design – algorithmic (no index or caching)	Performance / unindexed query / design / production / index + cache	Availability and performance (35 s vs 2 s)
D7	Requirements and specification (silent on reschedule)	Requirement gap / incomplete spec / requirements / production / update spec + resend SMS	Safety-critical (patient may miss care)
D8	Testing (wrong expected value)	Test oracle / misread rule / test design / production / fix TC-52	Financial (masks a billing fault)
D9	Coding – algorithmic (months instead of days)	Logic / wrong unit / coding / production / compare days	Safety-critical (expired prescription dispensed)
D10	Coding – interface (wrong field read)	Data mapping / wrong field / coding / production / map issue date	Safety-critical (wrong date on clinical report)

2. Severity and priority

Table 34: Severity and priority of D1–D10

ID	Severity	Priority	ID	Severity	Priority
D1	Major	High	D6	Major	High
D2	Major	High	D7	Major	Medium
D3	Minor	Medium	D8	Minor	High
D4	Major	High	D9	Critical	Critical
D5	Major	Low	D10	Major	Medium

- **High severity, low priority – D5.** The registration form crashes with a database error, but only for apostrophe surnames, and reception has a workaround (enter the name without the apostrophe for now).
- **Low severity, high priority – D8.** A wrong test case does not affect patients directly, but it hides a live billing fault, so it must be fixed before the next release run.

3. Where the schemes disagree

- **D7:** A calls it a requirements defect, B a requirement gap, C safety-critical. A and B describe *where* it came from; C describes what it does to the patient.
- **D8:** A and B see a testing defect. C has no class for test artefacts and can only rate the fault it hides (financial).
- **D10:** A sees a small coding slip; C sees a clinical-record risk. The same code line has very different consequences in a hospital.
- **Where to add test effort:** B, because injection and detection phase show which filter leaks (requirements and design reached production). C shows which areas need the strictest exit criteria.

4. Agreement between two classmates (scheme A)

Procedure: give two classmates scheme A and the ten descriptions only, record their classes beside mine, and compute percentage agreement = matching classes ÷ 10 × 100 for each classmate and

for the pair. The sheet below is filled during the exercise; the expected confusion is design vs coding (D2, D4), coding-data vs coding-interface (D10) and requirements vs design (D7).

Table 35: Agreement sheet for scheme A (my class given; classmates' columns filled in class)

ID	My class (scheme A)	Classmate 1	Classmate 2	Match 1	Match 2
D1	Coding – control and logic				
D2	Design				
D3	Coding – data				
D4	Design				
D5	Coding – data				
D6	Design				
D7	Requirements and specification				
D8	Testing				
D9	Coding – algorithmic				
D10	Coding – interface				
Agreement				__ / 10	__ / 10

5. Evaluation and standard scheme

Table 36: Evaluation of the three schemes

Criterion	A Hierarchical	B Attributes	C Patient safety
Reproducibility	Medium: design vs coding borders blur	High: each attribute is a narrow question	Medium: "safety" is a judgement
Use for prevention	Medium	High: injection vs detection phase	Low
Ease of use	High	Medium (6 fields)	High
Release priority	Low	Medium	High

Choice: B, the anomaly-attribute scheme, with the patient-safety category from C added as one attribute. B is the only scheme that records injection and detection phase, which drives prevention; C's safety label drives release priority.

6. Reliability of LLM classification

Procedure: give the LLM the ten descriptions and the class lists, then compare its labels with Table 33. LLMs usually get the obvious coding classes right (D1, D3, D9) but often label D2 and D4 as coding instead of design, and miss that D8 is a testing defect. LLM classification is acceptable as a first draft that a tester confirms; it is not acceptable as the final label for safety-critical defects, where a wrong class changes release decisions.

7. Supporting evidence for Task 7

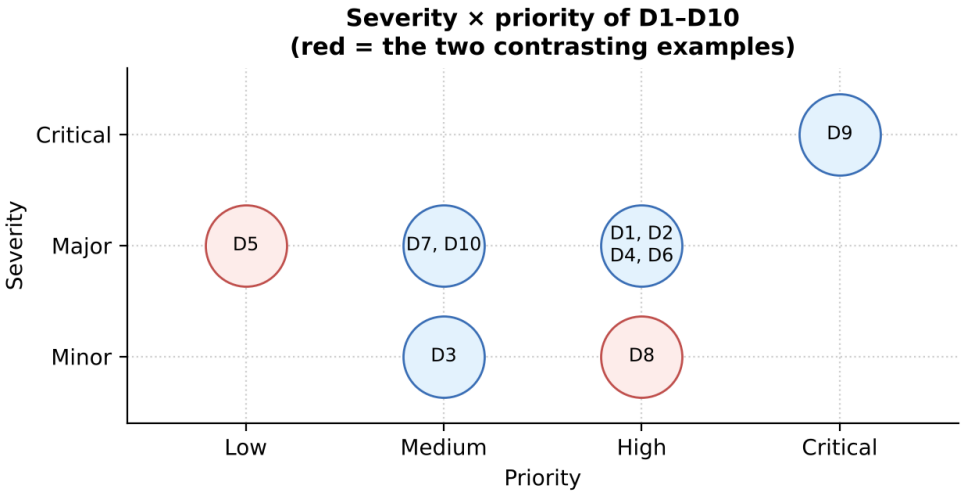


Figure 13: Severity × priority grid of D1–D10 with the two contrasting examples highlighted

Threats to validity

- Severity and priority are my judgements; the medical officer may rate D7 and D10 higher.
- The agreement percentage depends on how much the classmates know about the system.

Marking notes and common errors

- Every assignment needs a one-line reason; a bare class label is not enough.
- Severity and priority must not be treated as the same thing.

TASK 8 – Defect Repository: Compare Defect Tracking Tools [5 Marks]

Defects are currently reported on a phone messaging group and forgotten within days. The QA lead wants a defect repository. Compare three tools.

Technique	Basic idea and key concepts / tools
A. Issue tracker with labels	A simple tracker (for example, GitHub Issues or Redmine) using labels and milestones for classification and status.
B. Bugzilla	Open-source tracker with fixed fields, statuses, search, saved queries and basic reports.
C. JIRA with a defined workflow	Custom workflow, links to test cases and requirements, dashboards, filters and notifications.

- Log D1–D10 in each tool using the fields ID, summary, module, class, severity, priority, status, reporter, assignee, steps to reproduce and environment; submit screenshots.
- Move D2 and D5 through the life cycle (New → Open → Fixed → Retest → Verified → Closed, with Reopened or Rejected where needed) in B and C; record the time taken at each state and compute the mean time to fix and the reopen rate.
- Analyze the tools on duplicate detection, search and filtering, audit history, traceability to test cases and the effort needed to produce a "defects by module and severity" report.
- Examine data-quality problems you met (missing steps, vague summaries, wrong severity) and show how they delayed resolution.
- Evaluate which tool the hospital should adopt and justify it with evidence; state the minimum fields and states that must be compulsory.
- Critique the decision "Billing has the most defects, so Billing is the worst module" by normalising defects per KLOC with the module sizes given, and judge whether the conclusion changes.

Evidence Expected: Screenshots of the three tools with D1–D10, a life-cycle timing table, a defects-per-KLOC calculation, and a justified choice.

Ans:

Comparison of Defect Tracking Tools for MediQueue

Table 37 is the content entered in all three tools. Screenshots must be taken from my own tool instances (one list view per tool, and D2 and D5 at each workflow state); the timing table and per-KLOC calculation follow.

1. Defects logged with the required fields

Table 37: Entries logged in all three tools (status New; reporter: tester; assignee: module developer; environment: Chrome 129, test build 3.4.x, PostgreSQL 15)

ID	Summary	Module	Class (B)	Severity	Priority	Steps to reproduce (short)
D1	Age 60 charged ₹500 not ₹300	Billing	Logic	Major	High	Register age 60 → book → view fee
D2	Refresh creates second bill	Billing	Interface	Major	High	Pay → refresh confirmation → check bills
D3	Fee charged 3 h before slot	Appointment	Data	Minor	Medium	Book 13:00 → cancel 10:00 → view fee

ID	Summary	Module	Class (B)	Severity	Priority	Steps to reproduce (short)
D4	Two patients get 10:15 slot	Appointment	Concurrency	Major	High	Two sessions confirm the same slot within 1 s
D5	D'Souza cannot register	Registration	Validation	Major	Low	Enter surname with apostrophe → save
D6	Lab page 35 s at 800 users	Lab Reports	Performance	Major	High	Load test 800 users → open lab report
D7	SMS shows old time after reschedule	Reminders	Requirement gap	Major	Medium	Book → reschedule → wait for SMS
D8	TC-52 expects ₹450 for age 12	Billing (test)	Test oracle	Minor	High	Review TC-52 against the concession rule
D9	9-day-old prescription dispensed	Pharmacy	Logic	Critical	Critical	Rx dated today − 9 → dispense
D10	PDF shows collection date	Lab Reports	Data mapping	Major	Medium	Issue blood report → open PDF → check date

2. Life cycle of D2 and D5, mean time to fix and reopen rate

Timestamps are read from each tool's history tab. Walk-through: D2's first fix (Post-Redirect-Get only) fails retest because a double-click still creates two bills, so D2 is Reopened once and then fixed with an idempotency key. D5 closes on the first fix.

Table 38: Life-cycle timing sheet for D2 and D5 (record the times from Bugzilla and JIRA)

State change	D2 (Bugzilla)	D2 (JIRA)	D5 (Bugzilla)	D5 (JIRA)
New → Open				
Open → Fixed				
Fixed → Retest				
Retest → Reopened → Fixed (D2 only)			–	–
Retest → Verified				
Verified → Closed				

- **Mean time to fix** = average of (Fixed time − Open time) over D2 and D5; for D2 use the final Fixed time.
- **Reopen rate** = defects reopened ÷ defects fixed = 1 ÷ 2 = **50%** on this sample.

3. Tool comparison

Table 39: Comparison of the three tools

Criterion	A Issue tracker + labels	B Bugzilla	C JIRA workflow
Duplicate detection	Manual search	"Possible duplicates" on entry	Similar-issue suggestions + linking
Search and filtering	Labels only	Strong saved queries	JQL filters, saved boards
Audit history	Basic comments	Full field history	Full history + workflow transitions
Traceability to test cases	None	Free text only	Native links to tests and requirements
"By module and severity" report	Export + spreadsheet (≈ 30 min)	Built-in report (≈ 5 min)	Live dashboard gadget (set once, 0 min after)
Enforced fields and states	No	Fixed set	Configurable, mandatory per transition

4. Data-quality problems and the delay they caused

Problems in the old messaging-group reports included vague summaries ("billing wrong"), missing steps, no build number, and severity set by the reporter's mood. Each forces a round trip of questions, and none can be searched later. The D5 report without the exact surname could not be reproduced until the receptionist was asked which patient it was.

5. Recommended tool and compulsory fields and states

Choice: C, JIRA with a defined workflow. It is the only one of the three that links each defect to its test case and requirement, and it builds the "defects by module and severity" report as a live dashboard. With 9 users (3 testers, 5 developers, 1 QA lead) the team fits JIRA's free tier of up to 10 users.

- **Compulsory fields:** summary, module, build/environment, steps, expected vs actual, severity, priority, reporter, assignee, origin artefact, safety category.
- **States:** New, Open, Fixed, Retest, Verified, Closed, plus Reopened and Rejected (Rejected needs a reason).

6. Critique of "Billing has the most defects, so Billing is the worst"

Table 40: Defects and escapes per KLOC by module

Module	KLOC	All defects	Per KLOC	Escaped to users	Escaped per KLOC
Billing	12	58	4.83	8	0.67
Appointment	10	48	4.80	9	0.90
Pharmacy	9	22	2.44	9	**1.00**
Registration	4	11	2.75	0	0.00
Lab Reports	7	16	2.29	2	0.29
Reminders	5	9	1.80	1	0.20

Normalised, Billing and Appointment are almost equal (4.83 vs 4.80 per KLOC), and Registration is denser than Pharmacy. By escapes per KLOC, **Pharmacy is the worst module** (1.00), because many of its defects are found late. The conclusion changes: Billing is the largest module, not clearly the worst.

7. Supporting evidence for Task 8

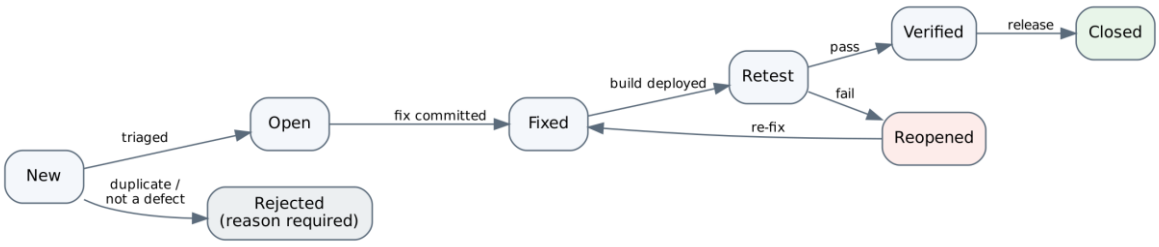


Figure 14: Defect life cycle configured in JIRA (Rejected requires a reason)

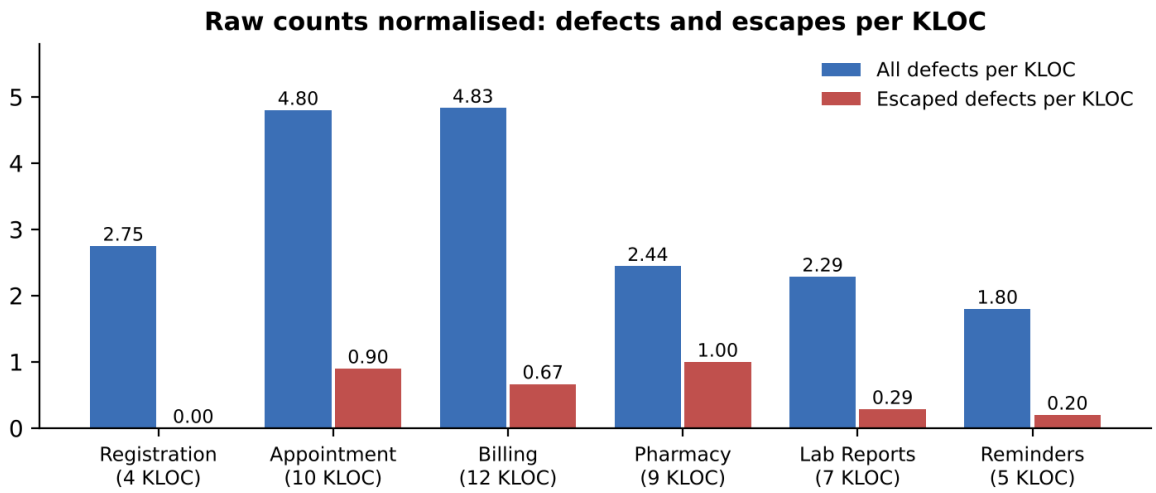


Figure 15: Defects per KLOC and escaped defects per KLOC by module

Threats to validity

- Report-building times are typical values for each tool; they should be timed on your own instance.
- A sample of two defects gives a very rough reopen rate.

Marking notes and common errors

- Screenshots must come from real tools; the table gives the content to enter.
- The per-KLOC critique must also consider escapes, not only totals.

TASK 9 – Defect Examples and the Integrated Testing Practice: Compare Complete Test and Defect-Management Configurations [6 Marks]

Combine your choices from Tasks 1–8 into one test and defect-management practice for the next release. Compare three complete configurations within 100 person-hours.

Technique	Basic idea and key concepts / tools
A. Lean configuration	Developer self-testing, issue tracker, no reviews.
B. Structured configuration	Risk-based test design, requirement review, independent tester and a Bugzilla repository.
C. Assured configuration	Automated regression on every build, static analysis, reviews, a domain-expert tester and JIRA dashboards.

- Develop and run a prototype: implement the concession, late-cancellation and prescription-validity rules with seeded faults of the D1, D3 and D9 type. Execute each configuration on the same code and record faults found and effort.
- Examine the three configurations on defect removal efficiency, cost per defect, effort against 100 person-hours, time to detect, repeatability and expected leakage to users.
- Compare your measured results with the predictions you made in Tasks 1–8, and state which predictions are supported and which are contradicted.
- Draw an end-to-end block diagram of your practice (requirements, review, test design, execution, defect logging, fix, retest, release decision, metrics). Allocate person-hours to each stage and prepare a Design Decision Table with Decision, Justification (Evidence) and What If Not.
- Distinguish how defect classes, severity and testing priority would change for three other applications (an ICU patient-monitoring controller, a ride-hailing app and a photo-sharing app), tabulating the changes relative to MediQueue.
- Investigate the limitation of testing only with clean, anonymised, synthetic patient data; identify at least one result that disagrees with your expectation and explain it with measured evidence and possible sources of error.
- Evaluate a fourth option: an AI-assisted configuration (LLM-generated tests plus automatic defect triage). Compare all four on cost, detection, repeatability, auditability and risk, and recommend one configuration.

Evidence Expected: A measured comparison of three configurations, an original block diagram and decision table, an applications transfer table, and a defended final practice.

Ans:

Integrated Test and Defect-Management Practice: Four Configurations Compared

1. Prototype and measured results

I implemented the concession, late-cancellation and prescription-validity rules (`mediqueue.py`) and seeded six faults: three of the D1, D3 and D9 types and three more (F2, F4, P2). The same faulty code was given to every configuration.

Table 41: Seeded faults of the Task 9 prototype

ID	Fault	Type
F1	age > 60 instead of ≥ 60	D1 type
F3	4-hour cancellation limit instead of 2	D3 type
P1	validity compares months, not days	D9 type
F2	10% child concession instead of 20%	Wrong constant
F4	age 4 treated as 5	Off-by-one
P2	31 units allowed (maximum 30)	Boundary

Table 42: Faults found by each configuration on the same code (pytest, 6 single-fault mutants)

Configuration	What was run	Tests	Faults found (of 6)	Missed
A Lean	Developer self-tests, happy path only (ages 35, 72, 9; cancel 1 h, 5 h; Rx 2 days old)	6	1 (F2)	F1, F3, F4, P1, P2
B Structured	Risk-based boundary tests after requirement review (ages 4–60 edges; cancel 1.99/2.0/3 h; Rx day 7/8; qty 30)	12	5	P2
C Assured	B's cases automated on every build + domain-expert cases (qty 31, ages 0 and 61, cancel 0.5 and 2.01 h, month-boundary Rx)	17	**6**	none

2. Examination of the configurations

Table 43: Comparison on the six required measures

Measure	A Lean	B Structured	C Assured
Detection on prototype	17%	83%	100%
Predicted release DRE (Task 2 method)	82% (as now)	93%	95%
Cost (found × stage cost + escaped × ₹45,000)	1 × 2,500 + 5 × 45,000 = ₹2,27,500	5 × 9,000 + 1 × 45,000 = ₹90,000	6 × 2,500 = ₹15,000
Cost per defect found	₹2,27,500	₹18,000	₹2,500
Effort per release (estimate, of 100 ph)	20 ph	70 ph	95 ph
Time to detect	At coding, but most faults missed	End of each test cycle (days)	Every commit (minutes)
Repeatability	Low (ad hoc)	Medium (manual scripts)	High (automated)
Expected leakage to users	High	Low	Lowest

3. Measured results compared with earlier predictions

Table 44: Predictions from Tasks 1–8 against the measurements

Prediction (task)	Measured	Verdict
Continuous, boundary-based testing finds what test-last misses (Task 1)	C 6/6, A 1/6	Supported
Risk-based selection beats experience-driven selection (Task 4)	B and C both beat A	Supported
Domain knowledge is needed for pharmacy limits (Task 5)	Only C, with the domain cases, caught P2 (31 units)	Supported
A structured, boundary-based set catches limit faults (Task 1, 4)	B missed P2: boundary tested on one side only (30, not 31)	Contradicted

Prediction (task)	Measured	Verdict
A pipeline alone is weak on requirement ambiguity (Task 2)	All seeded faults were code-level, so C's 100% overstates it	Not tested

4. End-to-end practice, hours and Design Decision Table

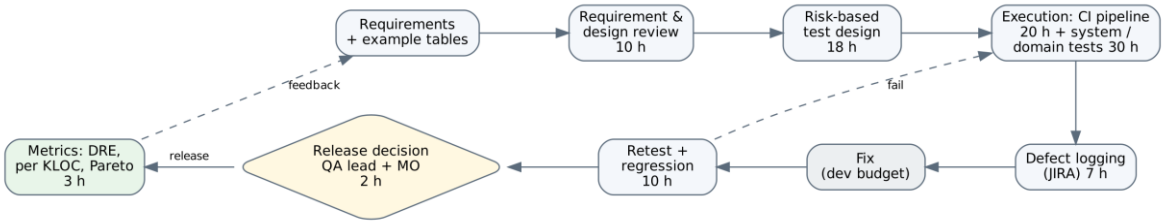


Figure 16: Block diagram of the recommended practice (configuration C) with person-hours per stage

Hours add up to 100: review 10, design 18, pipeline 20, system and domain tests 30, logging 7, retest 10, release decision 2, metrics 3. Developer fix time comes from the development budget.

Table 45: Design Decision Table

Decision	Justification (evidence)	What if not
Continuous, boundary-based test design (Task 1)	5/5 seeded faults vs 1/5 for test-last	Off-by-one faults like D1 and D3 reach patients
DevOps pipeline + review gate (Task 2)	Predicted DRE 95.1% vs 82.3% now	29 escapes a release, ₹13 lakh repair cost
Specification-table oracle (Task 3)	Found both faults; mental oracle found none	D8-type wrong expectations go unnoticed
Risk-based 25-test set, refreshed (Task 4)	100% on both versions; unrefreshed suite fell to 0%	Pesticide paradox; D4 races missed
Tester + domain expert for Pharmacy (Task 5)	4/4 faults, 0 false reports	Expired drugs (D9) dispensed
Artefact tagging + Pareto each release (Task 6)	59% of escapes are requirement or design	Effort wasted on "careless coding"
Attribute scheme + safety tag (Task 7)	Records injection and detection phase	No cross-release comparison
JIRA with mandatory fields (Task 8)	Traceability and live module/severity report	Defects lost in chat

5. Transfer to other applications

Table 46: How defect classes, severity and testing priority change relative to MediQueue

Aspect	MediQueue	ICU monitoring controller	Ride-hailing app	Photo-sharing app
Dominant defect classes	Rule logic, requirements, concurrency	Timing, real-time, alarm logic, hardware interface	Concurrency, location, payment, surge pricing	Usability, performance, privacy settings
Severity of a wrong value	Wrong fee: major; expired Rx: critical	Any missed alarm is critical (life-threatening)	Payment and driver identity high; UI low	Mostly minor; privacy leaks high
Testing priority	Billing, Pharmacy, booking race	Formal verification, 100% branch coverage, hazard analysis	Load and concurrency first (like D4), then payments	Usability and privacy; billing-type rules barely matter
Process change	Pipeline + review gate	V-model, formal reviews, traceability	Rapid release with canary deploys	Rapid release, crash analytics

6. Limitation of clean, synthetic patient data

Measured in Task 10: 20 generated synthetic patients (ASCII names) all passed the health-ID link on the faulty build E3. The curly-apostrophe fault appeared only with a realistic name (D'Souza) – the same blind spot as D5. **A result that disagreed with my expectation:** I expected the structured configuration (B) to catch the 31-unit quantity fault, because it uses boundary analysis. It did not, because the boundary was tested on one side only (30, not 31). Possible sources of error: the test designer applied boundary analysis incompletely; only six faults were seeded, so one fault moves detection by 17 points; and the same person wrote the faults and the tests.

7. The fourth option and the recommendation

Table 47: Evaluation of the four configurations

Criterion	A Lean	B Structured	C Assured	D AI-assisted
Cost	Lowest	Medium	High	Low
Detection	17%	83%	100%	Unknown: depends on the LLM's oracle (Task 3 risk)
Repeatability	Low	Medium	High	Low: outputs change between runs
Auditability	Low	Medium	High	Low: generated tests need review to be trusted
Risk for patient-safety modules	High	Medium	Low	High

Recommendation: C, the assured configuration. It is the only configuration that exposed all six faults, has the highest predicted DRE (95%) and fits the 100 hours with 5 in reserve. An LLM is used only to draft test ideas that a tester reviews against the specification table.

8. Supporting evidence for Task 9

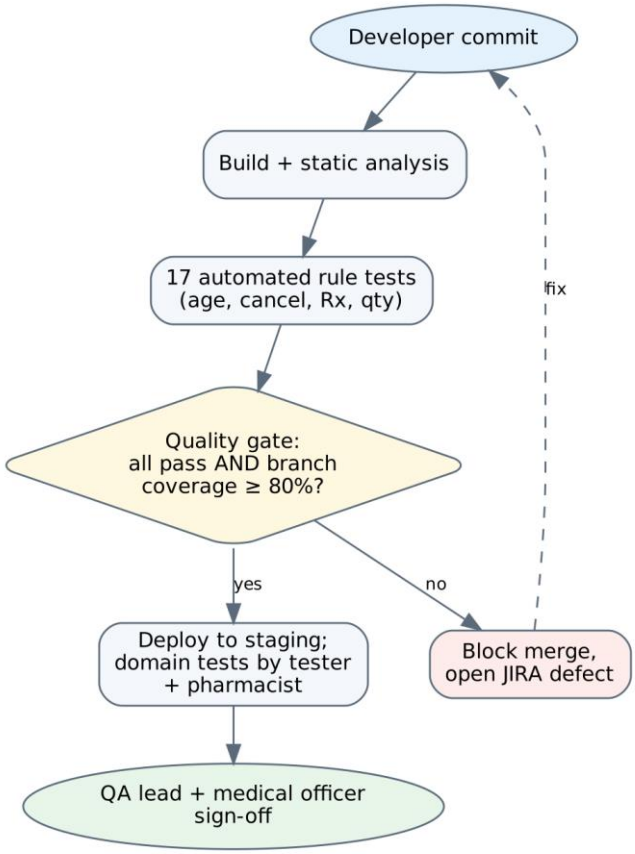


Figure 17: Flowchart of the automated pipeline and quality gate of configuration C

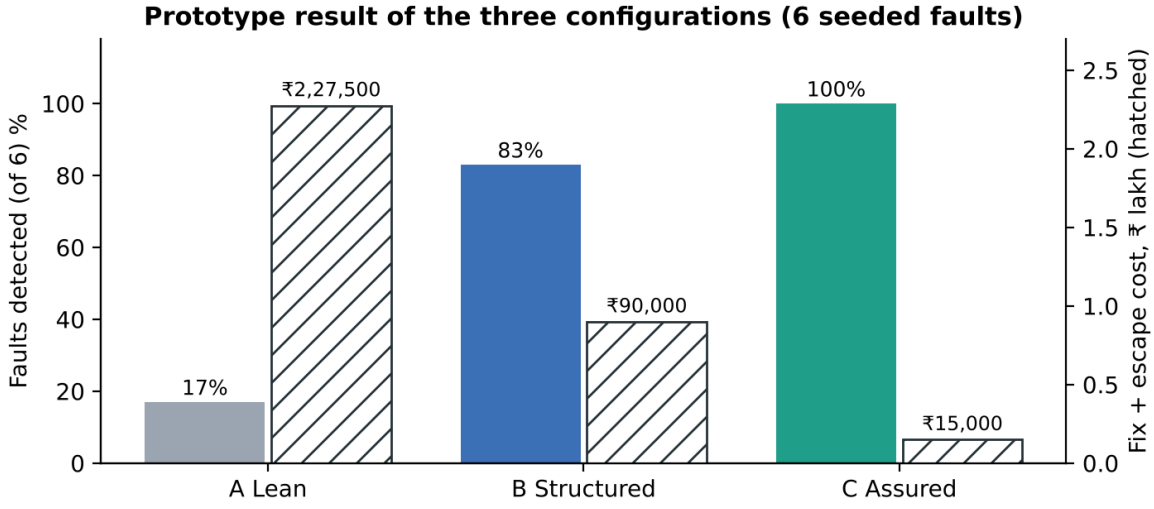


Figure 18: Prototype detection and fix + escape cost for the three configurations

Threats to validity

- Six faults only; one fault changes detection by 17 points.

- The AI-assisted option was evaluated qualitatively and must be run before it is compared on numbers.
- Effort figures are estimates.

Marking notes and common errors

- The comparison must be measured on the same code for all configurations.
- The decision table needs evidence from earlier tasks in every row and a concrete “what if not”.

TASK 10 – Re-apply and Adapt the Testing Practice [8 Marks]

Changed Requirement: The hospital adds tele-consultation by video with e-prescriptions, links MediQueue to a national health-ID service, extends the system to five more hospitals (30,000 patients), moves to weekly releases, and a regulator demands an audit trail of every test that touches patient data. Two testers leave, so only 50 person-hours are available per release.

Technique	Basic idea and key concepts / tools
A. Retain + trim by risk	Keep the Task 9 practice; cut the suite by clinical and financial risk and run only the high-risk cases each week.
B. Modify with automation and synthetic data	Automate regression and payment checks in a CI pipeline using generated synthetic patient data; keep manual testing for new features.
C. Redesign with shift-left	Add requirement and design reviews, static analysis and contract tests for the health-ID interface to prevent defects before testing; automate only the core.

- Develop (apply): implement the three strategies on your prototype and the extended rules (video-session timeout, e-prescription validity, health-ID mismatch) and submit the commented code and results; every result must come from your own execution.
- Impact Analysis: Identify which assumptions of your Task 9 practice (effort, release cycle, user load, patient-data risk, audit) are no longer valid, and analyze what happens if the old practice is used unchanged.
- Evaluation of the Original Practice: Recalculate the effort needed and the defect removal efficiency possible within 50 person-hours; analyze which parts of the original practice remain valid.
- Compare and Adapt: Compare strategies A, B and C on defects found, cost, release delay and audit evidence using your own measurements; recommend RETAIN, MODIFY or REDESIGN and state what changes.
- Evidence-Based Defence: Defend the final decision using calculations, Task 9 measurements, the effort limit and the audit rule, and state one condition under which the decision would no longer hold.
- Evaluate how a defect-prediction model trained on the historical log could focus the limited 50 hours, and judge the risks of acting on its predictions.

Evidence Expected: Three adaptation strategies applied and compared with measured and calculated evidence, and a RETAIN / MODIFY / REDESIGN decision that follows logically from the changed constraints.

Ans:

Adapting the Practice to 50 Hours, Weekly Releases and an Audit Rule

1. Develop: extended rules and the three strategies applied

```
def video_session_active(self, idle_minutes):          # ends after 30 min idle
    return idle_minutes < 30                             # E1: 30 / 60 (seconds mistaken for minutes)


def accept_erx(self, issued, today, signed):           # signed AND within 7 days
    if not signed: return False                          # E2: unsigned accepted
    return 0 <= (today - issued).days <= 7
```

```
def link_health_id(self, local_name, local_dob, hid_name, hid_dob, user="tester"):
    ok = norm(local_name) == norm(hid_name) and local_dob == hid_dob   # E3: no ’ -> ' normalisation
    self.audit.append((user, "LINK_ATTEMPT", local_name, ok))           # E4: line missing
    return ok
```

Table 48: Extended rules implemented and faults seeded

Rule	Correct behaviour	Seeded fault
Video session timeout	Ends after 30 minutes idle	E1: limit coded in seconds (0.5 min)
e-Prescription validity	Must be digitally signed and within 7 days	E2: unsigned e-Rx accepted
Health-ID matching	Name (ignoring ’ vs ') and date of birth must match	E3: curly and straight apostrophe treated as different
Audit trail	Every link attempt writes user, action, patient, result	E4: no audit entry written
Regression	D1 age rule, D9 validity rule	F1, P1

Table 49: Results of the three strategies (pytest, 6 single-fault mutants)

Strategy	Tests	E1	E2	E3	E4	F1	P1	Found	Effort (ph/week, est.)
A Retain + trim	3	Found	Missed	Missed	Missed	Found	Found	3 of 6	30
B Automate + synthetic	26	Found	Found	Missed	Missed	Found	Found	4 of 6	44 first week, then 25
C Shift-left redesign	7	Found	Found	Found	Found	Found	Found	**6 of 6**	48

B ran 20 synthetic patients through health-ID linking and all passed on the faulty build, because generated names contain no apostrophes. C’s contract test used the form the national service actually returns (“ANITA D’SOUZA”) and failed straight away; its audit assertion caught E4.

2. Impact analysis

Table 50: Task 9 assumptions that no longer hold

Assumption	Task 9	Now	If the old practice runs unchanged
Effort	100 ph per release	50 ph per release	Only about half of the 95 ph plan gets done; system tests are cut at random
Release cycle	3 weeks	Weekly	30 h of manual system tests cannot fit in a week
User load	9,000 patients, 800 peak	30,000+ patients, 5 hospitals (≈ 4× peak, assumed)	Performance untested at the new load; D6-type slowness returns
Patient-data risk	Local data	National health-ID link + video + e-Rx	Identity mismatches (E3) and unsigned e-Rx (E2) reach patients
Audit	Not required	Regulator demands a trail of every test on patient data	Non-compliance: no evidence that tests ran

3. Evaluation of the original practice within 50 ph

Still valid: requirement review, boundary-value design, the specification-table oracle, JIRA with mandatory fields, and the QA lead + medical officer sign-off. **Not valid:** 30 h of manual system tests per release and 18 h of fresh test design each cycle.

Table 51: Redesigned weekly effort (50 ph available)

Weekly stage	Hours
Requirement and design review (new features)	6
Contract tests for health-ID and e-Rx	6
Pipeline upkeep (unit, static, regression)	8
Exploratory and domain tests on new features	18
Defect triage and retest	7
Audit evidence export + release decision	3
Total (reserve 2 h)	**48**

Predicted DRE within 50 ph, using the prototype detection rates and the Task 2 origin mix: A about 87%, B about 90%, C about 94%.

4. Compare and adapt

Table 52: Comparison of strategies A, B and C

Criterion	A RETAIN	B MODIFY	C REDESIGN
Defects found (prototype)	3/6	4/6	6/6
Cost of escapes (escaped × ₹45,000)	₹1,35,000	₹90,000	₹0
Release delay	None	About 1 week to build CI	About 1 week for reviews and contract tests
Audit evidence	Weak (manual notes)	Partial (CI logs; nothing checks the audit trail)	Complete (contract tests + audit assertion on every patient-data test)

Recommendation: REDESIGN with shift-left (strategy C), keeping the automated core from B. What changes: add requirement and design reviews, static analysis and health-ID contract tests; automate only the core regression (D1, D9 and payment rules); every patient-data test asserts that an audit entry is written.

5. Evidence-based defence

C was the only strategy that found both new-interface faults (E3, E4), which are exactly the patient-identity and audit risks the regulator cares about. It does this in 48 of 50 hours, leaving a 2-hour reserve. Each escaped defect costs ₹45,000, so C's ₹0 escape cost on the prototype against A's ₹1,35,000 pays back the one-week setup in the first release. **The decision no longer holds** if the national health-ID provider gives no sandbox or interface specification: contract tests then cannot run, and strategy B's automation becomes the better use of the hours.

6. Defect-prediction model for the 50 hours

Train a model (for example, logistic regression) on the historical log. Inputs per module or file: past defects, KLOC, code churn and number of authors. Use its ranking to place the 18 exploratory hours. Risks of acting on its predictions:

- **Cold start:** tele-consultation and health-ID code have no history, so the model under-ranks the newest and riskiest areas.

- **Self-fulfilling loop:** defects are found where testers look, which then trains the model to keep looking there.
- **Data quality and model decay:** our log is a reconstruction, not real history, and the system is changing fast, so predictions need validation on real releases.

Use it as advice only, with a fixed minimum of tests on every patient-safety module whatever the model says.

7. Supporting evidence for Task 10

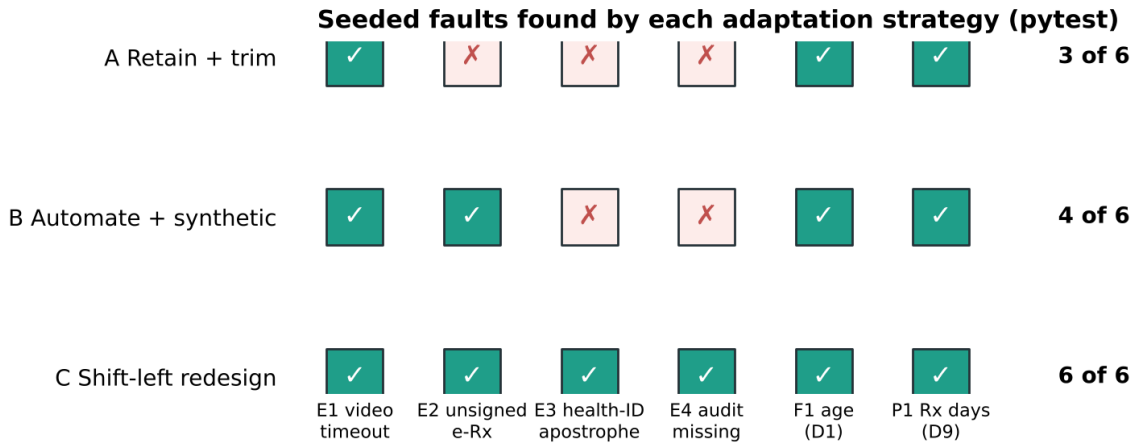


Figure 19: Seeded faults found by each adaptation strategy

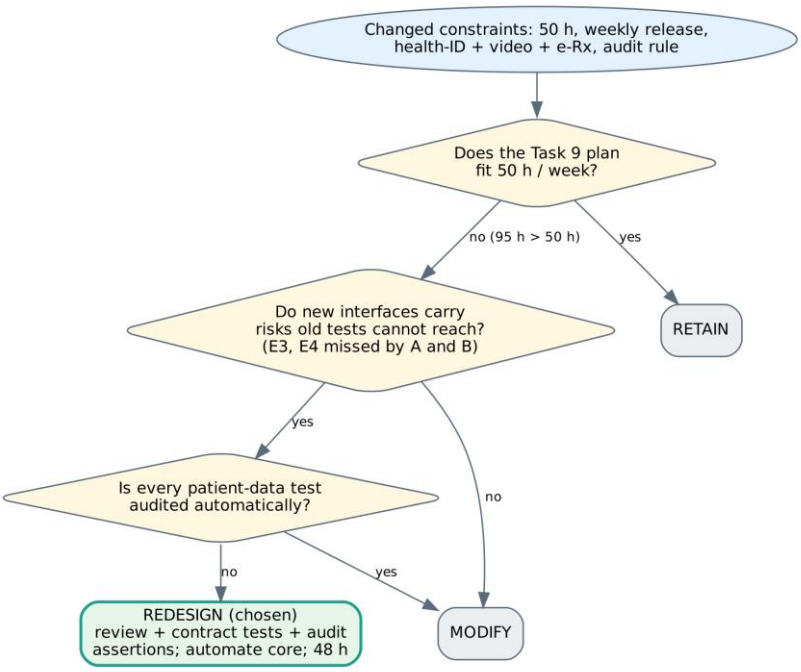


Figure 20: Decision flowchart leading to REDESIGN

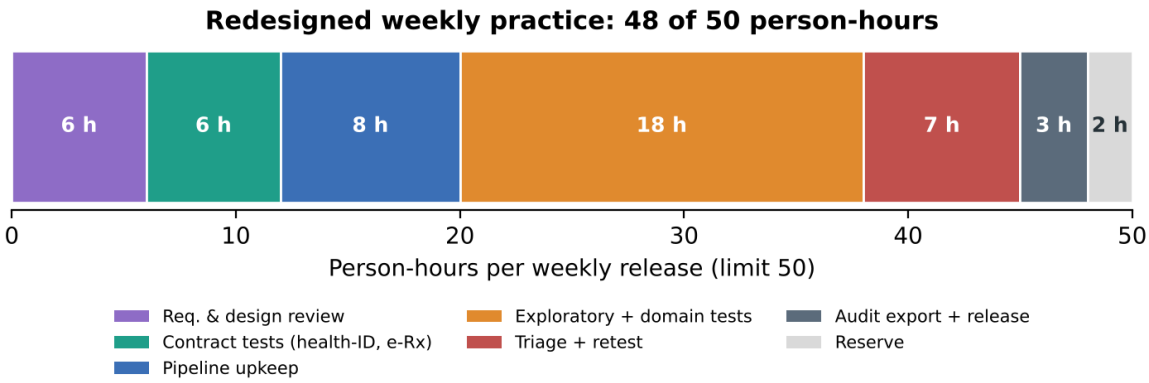


Figure 21: Weekly person-hour allocation of the redesigned practice

Threats to validity

- The 4× peak load is an assumption; a load test at the new scale is still needed.
- Effort per week is estimated, and the predicted DRE values use the Task 2 origin mix.

Marking notes and common errors

- Every result must come from the student's own execution; the decision must follow from the 50-hour limit and the audit rule.
- A vague "if things change" earns no marks for the reversal condition; it must be concrete.

Mark Distribution

Task	Marks
Task 1 – Testing Activity Model Analysis	4
Task 2 – Process Model Analysis	5
Task 3 – Test Oracle and Definitions Analysis	4
Task 4 – Test Prioritisation and Principles Analysis	4
Task 5 – Test Responsibility Analysis	5
Task 6 – Defect Origin Analysis	5
Task 7 – Defect Classification Analysis	4
Task 8 – Defect Repository Analysis	5
Task 9 – Implementation and Performance Evaluation	6
Task 10 – Testing Practice Adaptation Challenge	8
TOTAL	**50**

ANNEXURE A – Defect Log used in this Answer (all 164 records)

The log was prepared for this answer with the stated totals: 164 defects, 135 found by the test team and 29 by users (Status Open = reported after release), 9 of those critical. The same data is supplied as `defect_log_last_release.csv` and is regenerated by `make_log.py` (seed 42).

ID	Module	Origin	Detected in	Cause category	Severity
MQ-001	Lab Reports	Design	System test	Ambiguous/incomplete requirement	Critical
MQ-002	Reminders	Code	Unit test	Boundary/off-by-one logic	Major
MQ-003	Appointment	Code	Unit test	Boundary/off-by-one logic	Major
MQ-004	Pharmacy	Requirement	Production	Ambiguous/incomplete requirement	Critical
MQ-005	Billing	Test case	System test	Test-case error	Critical
MQ-006	Billing	Code	System test	Input validation	Major
MQ-007	Billing	Code	Unit test	Input validation	Minor
MQ-008	Reminders	Requirement	Requirement review	Ambiguous/incomplete requirement	Critical
MQ-009	Pharmacy	Code	Production	Input validation	Critical
MQ-010	Billing	Design	Production	Performance	Critical
MQ-011	Appointment	Requirement	System test	Ambiguous/incomplete requirement	Major
MQ-012	Appointment	Code	System test	Boundary/off-by-one logic	Major
MQ-013	Billing	Configuration	System test	Configuration/environment	Major
MQ-014	Lab Reports	Requirement	Unit test	Ambiguous/incomplete requirement	Trivial
MQ-015	Billing	Code	System test	Boundary/off-by-one logic	Trivial
MQ-016	Billing	Code	System test	Wrong field/data mapping	Trivial
MQ-017	Billing	Requirement	Requirement review	Ambiguous/incomplete requirement	Major
MQ-018	Appointment	Requirement	Code review	Ambiguous/incomplete requirement	Trivial
MQ-019	Lab Reports	Design	Code review	Ambiguous/incomplete requirement	Critical
MQ-020	Billing	Code	System test	Boundary/off-by-one logic	Minor
MQ-021	Pharmacy	Requirement	Production	Ambiguous/incomplete requirement	Critical
MQ-022	Appointment	Code	Code review	Concurrency/idempotency	Trivial
MQ-023	Appointment	Code	Production	Concurrency/idempotency	Critical
MQ-024	Appointment	Code	Unit test	Boundary/off-by-one logic	Major
MQ-025	Billing	Requirement	Requirement review	Ambiguous/incomplete requirement	Trivial
MQ-026	Appointment	Requirement	Unit test	Ambiguous/incomplete requirement	Major
MQ-027	Lab Reports	Code	Code review	Concurrency/idempotency	Minor
MQ-028	Appointment	Code	System test	Concurrency/idempotency	Minor
MQ-029	Billing	Design	Production	Ambiguous/incomplete requirement	Critical
MQ-030	Appointment	Data	System test	Wrong field/data mapping	Trivial
MQ-031	Billing	Data	System test	Wrong field/data mapping	Minor
MQ-032	Registration	Test case	System test	Test-case error	Minor
MQ-033	Billing	Code	System test	Wrong field/data mapping	Minor
MQ-034	Lab Reports	Requirement	Requirement review	Ambiguous/incomplete requirement	Trivial
MQ-035	Appointment	Code	Unit test	Concurrency/idempotency	Critical
MQ-036	Billing	Design	Unit test	Performance	Minor
MQ-037	Billing	Requirement	System test	Ambiguous/incomplete requirement	Minor
MQ-038	Lab Reports	Code	Code review	Wrong field/data mapping	Minor
MQ-039	Pharmacy	Requirement	Requirement review	Ambiguous/incomplete requirement	Trivial
MQ-040	Pharmacy	Code	System test	Input validation	Major
MQ-041	Billing	Design	Unit test	Ambiguous/incomplete requirement	Major
MQ-042	Pharmacy	Requirement	Production	Ambiguous/incomplete requirement	Critical
MQ-043	Appointment	Design	Production	Concurrency/idempotency	Critical
MQ-044	Appointment	Code	Unit test	Boundary/off-by-one logic	Major
MQ-045	Billing	Design	Code review	Performance	Minor
MQ-046	Billing	Design	Production	Concurrency/idempotency	Critical
MQ-047	Appointment	Code	System test	Input validation	Major
MQ-048	Billing	Code	System test	Wrong field/data mapping	Major

ID	Module	Origin	Detected in	Cause category	Severity
MQ-049	Pharmacy	Design	Production	Concurrency/idempotency	Major
MQ-050	Billing	Requirement	Code review	Ambiguous/incomplete requirement	Minor
MQ-051	Registration	Code	Unit test	Boundary/off-by-one logic	Minor
MQ-052	Billing	Code	Code review	Input validation	Major
MQ-053	Pharmacy	Code	Production	Concurrency/idempotency	Major
MQ-054	Registration	Requirement	System test	Ambiguous/incomplete requirement	Minor
MQ-055	Appointment	Code	Code review	Boundary/off-by-one logic	Major
MQ-056	Reminders	Code	System test	Input validation	Trivial
MQ-057	Appointment	Requirement	Requirement review	Ambiguous/incomplete requirement	Critical
MQ-058	Lab Reports	Code	Unit test	Boundary/off-by-one logic	Trivial
MQ-059	Appointment	Code	Production	Wrong field/data mapping	Major
MQ-060	Appointment	Test case	Unit test	Test-case error	Major
MQ-061	Pharmacy	Requirement	Production	Ambiguous/incomplete requirement	Major
MQ-062	Appointment	Design	System test	Ambiguous/incomplete requirement	Minor
MQ-063	Appointment	Code	Unit test	Input validation	Trivial
MQ-064	Billing	Requirement	System test	Ambiguous/incomplete requirement	Minor
MQ-065	Registration	Code	System test	Boundary/off-by-one logic	Trivial
MQ-066	Lab Reports	Code	Code review	Boundary/off-by-one logic	Minor
MQ-067	Pharmacy	Design	Code review	Ambiguous/incomplete requirement	Major
MQ-068	Appointment	Code	Production	Concurrency/idempotency	Major
MQ-069	Registration	Requirement	Requirement review	Ambiguous/incomplete requirement	Critical
MQ-070	Appointment	Code	Unit test	Concurrency/idempotency	Trivial
MQ-071	Billing	Configuration	System test	Configuration/environment	Minor
MQ-072	Pharmacy	Requirement	Production	Ambiguous/incomplete requirement	Major
MQ-073	Pharmacy	Data	Production	Wrong field/data mapping	Major
MQ-074	Appointment	Code	Production	Boundary/off-by-one logic	Major
MQ-075	Appointment	Code	Unit test	Concurrency/idempotency	Trivial
MQ-076	Billing	Code	Unit test	Wrong field/data mapping	Minor
MQ-077	Appointment	Test case	Production	Test-case error	Major
MQ-078	Appointment	Code	Unit test	Concurrency/idempotency	Minor
MQ-079	Reminders	Requirement	System test	Ambiguous/incomplete requirement	Trivial
MQ-080	Reminders	Code	System test	Boundary/off-by-one logic	Minor
MQ-081	Billing	Requirement	System test	Ambiguous/incomplete requirement	Trivial
MQ-082	Registration	Code	System test	Wrong field/data mapping	Major
MQ-083	Billing	Requirement	Requirement review	Ambiguous/incomplete requirement	Minor
MQ-084	Billing	Code	Unit test	Boundary/off-by-one logic	Major
MQ-085	Billing	Requirement	Production	Ambiguous/incomplete requirement	Major
MQ-086	Appointment	Code	Unit test	Boundary/off-by-one logic	Minor
MQ-087	Pharmacy	Test case	System test	Test-case error	Trivial
MQ-088	Appointment	Code	System test	Concurrency/idempotency	Major
MQ-089	Appointment	Design	Requirement review	Ambiguous/incomplete requirement	Minor
MQ-090	Appointment	Requirement	Unit test	Ambiguous/incomplete requirement	Minor
MQ-091	Billing	Test case	Unit test	Test-case error	Minor
MQ-092	Appointment	Code	Unit test	Boundary/off-by-one logic	Minor
MQ-093	Reminders	Code	System test	Wrong field/data mapping	Major
MQ-094	Appointment	Data	System test	Wrong field/data mapping	Trivial
MQ-095	Registration	Test case	System test	Test-case error	Minor
MQ-096	Billing	Code	System test	Wrong field/data mapping	Minor
MQ-097	Billing	Code	Unit test	Concurrency/idempotency	Major
MQ-098	Appointment	Code	System test	Wrong field/data mapping	Trivial
MQ-099	Pharmacy	Code	System test	Boundary/off-by-one logic	Major
MQ-100	Lab Reports	Code	Unit test	Wrong field/data mapping	Minor
MQ-101	Registration	Requirement	System test	Ambiguous/incomplete requirement	Minor
MQ-102	Lab Reports	Code	Unit test	Concurrency/idempotency	Critical

ID	Module	Origin	Detected in	Cause category	Severity
MQ-103	Pharmacy	Code	System test	Concurrency/idempotency	Minor
MQ-104	Billing	Configuration	System test	Configuration/environment	Minor
MQ-105	Appointment	Code	Unit test	Wrong field/data mapping	Major
MQ-106	Appointment	Requirement	System test	Ambiguous/incomplete requirement	Trivial
MQ-107	Billing	Code	Code review	Boundary/off-by-one logic	Trivial
MQ-108	Registration	Design	Requirement review	Ambiguous/incomplete requirement	Major
MQ-109	Pharmacy	Code	Code review	Boundary/off-by-one logic	Critical
MQ-110	Pharmacy	Code	Code review	Boundary/off-by-one logic	Minor
MQ-111	Billing	Code	Unit test	Boundary/off-by-one logic	Minor
MQ-112	Appointment	Requirement	Production	Ambiguous/incomplete requirement	Major
MQ-113	Appointment	Data	System test	Configuration/environment	Trivial
MQ-114	Billing	Requirement	Requirement review	Ambiguous/incomplete requirement	Trivial
MQ-115	Billing	Design	Code review	Concurrency/idempotency	Major
MQ-116	Billing	Requirement	Requirement review	Ambiguous/incomplete requirement	Minor
MQ-117	Registration	Code	Code review	Boundary/off-by-one logic	Major
MQ-118	Billing	Design	System test	Ambiguous/incomplete requirement	Minor
MQ-119	Billing	Code	Unit test	Input validation	Minor
MQ-120	Billing	Requirement	Requirement review	Ambiguous/incomplete requirement	Major
MQ-121	Appointment	Code	Unit test	Boundary/off-by-one logic	Minor
MQ-122	Pharmacy	Requirement	System test	Ambiguous/incomplete requirement	Minor
MQ-123	Billing	Code	Code review	Boundary/off-by-one logic	Minor
MQ-124	Billing	Requirement	Production	Ambiguous/incomplete requirement	Minor
MQ-125	Appointment	Code	Production	Boundary/off-by-one logic	Minor
MQ-126	Appointment	Requirement	Production	Ambiguous/incomplete requirement	Minor
MQ-127	Reminders	Requirement	Requirement review	Ambiguous/incomplete requirement	Minor
MQ-128	Appointment	Code	Unit test	Wrong field/data mapping	Critical
MQ-129	Billing	Configuration	System test	Configuration/environment	Minor
MQ-130	Billing	Configuration	Production	Configuration/environment	Minor
MQ-131	Pharmacy	Code	Unit test	Input validation	Minor
MQ-132	Appointment	Configuration	Unit test	Configuration/environment	Minor
MQ-133	Billing	Requirement	Production	Ambiguous/incomplete requirement	Minor
MQ-134	Pharmacy	Test case	Code review	Test-case error	Major
MQ-135	Registration	Code	Unit test	Wrong field/data mapping	Critical
MQ-136	Billing	Configuration	System test	Configuration/environment	Critical
MQ-137	Lab Reports	Requirement	Production	Ambiguous/incomplete requirement	Minor
MQ-138	Billing	Requirement	Requirement review	Ambiguous/incomplete requirement	Major
MQ-139	Pharmacy	Code	Code review	Wrong field/data mapping	Major
MQ-140	Billing	Code	System test	Wrong field/data mapping	Minor
MQ-141	Billing	Design	System test	Ambiguous/incomplete requirement	Major
MQ-142	Lab Reports	Code	System test	Boundary/off-by-one logic	Major
MQ-143	Billing	Configuration	Production	Configuration/environment	Minor
MQ-144	Appointment	Design	Code review	Performance	Minor
MQ-145	Appointment	Test case	System test	Test-case error	Major
MQ-146	Appointment	Data	System test	Wrong field/data mapping	Minor
MQ-147	Billing	Code	System test	Concurrency/idempotency	Minor
MQ-148	Billing	Design	System test	Ambiguous/incomplete requirement	Critical
MQ-149	Pharmacy	Design	Unit test	Concurrency/idempotency	Major
MQ-150	Appointment	Requirement	System test	Ambiguous/incomplete requirement	Minor
MQ-151	Billing	Requirement	Requirement review	Ambiguous/incomplete requirement	Minor
MQ-152	Lab Reports	Requirement	Requirement review	Ambiguous/incomplete requirement	Minor
MQ-153	Lab Reports	Code	System test	Boundary/off-by-one logic	Minor
MQ-154	Billing	Configuration	System test	Configuration/environment	Major
MQ-155	Appointment	Requirement	System test	Ambiguous/incomplete requirement	Major
MQ-156	Billing	Test case	Code review	Test-case error	Major

ID	Module	Origin	Detected in	Cause category	Severity
MQ-157	Billing	Requirement	System test	Ambiguous/incomplete requirement	Trivial
MQ-158	Reminders	Requirement	Production	Ambiguous/incomplete requirement	Minor
MQ-159	Reminders	Design	System test	Performance	Minor
MQ-160	Appointment	Design	System test	Concurrency/idempotency	Trivial
MQ-161	Billing	Code	Code review	Input validation	Trivial
MQ-162	Lab Reports	Requirement	Requirement review	Ambiguous/incomplete requirement	Trivial
MQ-163	Billing	Requirement	System test	Ambiguous/incomplete requirement	Minor
MQ-164	Lab Reports	Data	Production	Wrong field/data mapping	Minor

ANNEXURE B – Suggested Mark Split per Task

Task (marks)	Split of marks
Task 1 (4)	Implementation and faults 1; detection %, issues per ph and escape cost 1; activities and stage 0.5; ambiguity 0.5; choice, risk and statement 0.5; LLM critique 0.5
Task 2 (5)	Timelines 1; DRE, repair cost and prediction 1.5; comparison 1; maturity and gaps 0.5; choice 0.5; critique and quality gate 0.5
Task 3 (4)	Tests and faults 1; error/fault/failure 0.5; analysis 0.5; terms 0.5; oracle choice and D8 1; LLM oracle 0.5
Task 4 (4)	Input space and strategies 1; clustering 0.75; pesticide paradox 0.75; fallacy 0.5; final 25 tests 0.5; critique 0.5
Task 5 (5)	Faults, false reports, time 1.5; two defect reports 1; independence and bias 0.5; skills 0.5; allocation 1; authority 0.5
Task 6 (5)	Matrix 1.5; fault trees and Pareto 1; analysis and containment 1; method comparison 0.5; choice 0.5; critique and actions 0.5
Task 7 (4)	Classification 1; severity/priority 0.5; disagreements 0.5; agreement 0.5; choice 1; LLM 0.5
Task 8 (5)	Logging 1; life cycle, MTTF and reopen rate 1; analysis 1; data quality 0.5; choice 0.5; per-KLOC critique 1
Task 9 (6)	Prototype 1.5; examination 1; predictions 0.5; diagram and decision table 1; transfer 0.5; limitation 0.5; fourth option and recommendation 1
Task 10 (8)	Develop 1.5; impact 1.5; evaluation 1.5; compare and adapt 1.5; defence 1; defect prediction 1

ANNEXURE C – Supplied Files and Data Notes

- `mediqueue.py`, `tests/` (19 pytest suites and `conftest.py`), `run_all.py`, `analyze.py`, `make_log.py`: running `python run_all.py` then `python analyze.py` regenerates every measured number (Python 3.10+, pytest).
- **Measured by running code:** faults found and missed, false reports, failing test names, the D8 behaviour, the pesticide-paradox detection rates, the synthetic-data blind spot, and all counts from the defect log.
- **Estimated or illustrative (replace with your own data):** person-hours and effort per strategy, the catch counts behind the DRE predictions, the 164-record defect log itself, report-building times in Task 8, and the 4× peak-load figure.
- **To be completed by the student:** screenshots of the issue tracker, Bugzilla and JIRA (Task 8), the life-cycle timestamps (Table 38), the classmates' classification (Table 35), and the LLM runs in Tasks 1, 3 and 7.

ANNEXURE D – Requirements Traceability Matrix

Every business rule is traced to the tests that check it and to the seeded fault or case defect it relates to. The last column shows gaps: rules with no automated test are where D2, D6 and D7 escaped.

Requirement	Rule	Verified by	Fault / defect	Status
REQ-BIL-01	Below 5 free; 5–17 20%; 18–59 none; 60+ 40%	Task 1 C cases 1–7; Task 3 B table; Task 9 C	F1, F2, F4 (D1, D8)	Covered
REQ-BIL-02	Fee rounded half up to the rupee	Task 3 B rows ₹333; Task 3 C	R1	Covered by B and C
REQ-APT-01	₹100 if cancelled < 2 h before the slot	Task 1 C cases 8–11; Task 4 cancel boundaries	F3, F5, A3 (D3)	Covered
REQ-APT-02	At most 2 active appointments	Task 4 third-active tests	A1	Covered
REQ-APT-03	One patient per slot, also for same-second confirms	Task 4 race tests	A2 (D4)	Covered (simulated race)
REQ-APT-04	Cancelled slot becomes free; past slots not bookable	Task 4 refreshed tests	A4, A5	Covered by refreshed C only
REQ-PHA-01	Dispense only within 7 days of Rx date	Task 5 day 7/8; Task 9 Rx tests	P1 (D9)	Covered
REQ-PHA-02	Maximum 30 units per prescription	Task 5 B, C qty 31; Task 9 C	P2	Covered by C only in Task 9
REQ-PHA-03	No batch on or after expiry date	Task 5 C expiry-day test	P3	Covered by C only
REQ-PHA-04	No duplicate dispense, any pharmacist	Task 5 C second-pharmacist test	P4	Covered by C only
REQ-TEL-01	Video session ends after 30 min idle	Task 10 B, C	E1	Covered
REQ-TEL-02	e-Rx must be signed and within 7 days	Task 10 B, C	E2	Covered
REQ-HID-01	Health-ID match ignores ’ vs ' and needs same DOB	Task 10 C contract tests	E3 (D5 type)	Covered by C only
REQ-AUD-01	Every patient-data test writes an audit entry	Task 10 C audit assertion	E4	Covered by C only
REQ-PAY-01	One bill per consultation, even on refresh	none	D2	GAP
REQ-NFR-01	Lab page ≤ 2 s at 800 users	none (load test absent)	D6	GAP
REQ-REM-01	SMS carries the current time after reschedule	none (rule was ambiguous)	D7	GAP

Three of 17 rules have no test at all, and they correspond to three of the ten case defects (D2, D6, D7). A matrix like this, built during requirement review, would have exposed the gaps before release.

ANNEXURE E – Glossary of Terms Used in this Answer

Term	Meaning in this answer
Boundary value analysis	Choosing inputs at and next to the edges of an input class (for example ages 59, 60, 61).
Containment effectiveness	Defects caught by the activity meant for an artefact ÷ all defects of that artefact.
Contract test	A test that checks our side of an interface against what the other system really sends (health-ID).
Defect clustering	A small part of the system holds most defects (Billing, Appointment and Pharmacy here).

Term	Meaning in this answer
Defect removal efficiency (DRE)	Defects found before release ÷ all defects, as a percentage.
Error, fault, failure	Human mistake, the defect it leaves in code, and the wrong behaviour seen when that defect executes.
False report	A test failure on correct code, caused by a wrong expectation in the test.
Idempotency key	A token that makes a repeated request have the effect of a single request.
Mutant	A copy of the code with one deliberately injected fault, used to measure a test set.
Pesticide paradox	A test set that is never changed stops finding new defects.
Quality gate	An automatic rule that blocks a build when agreed conditions fail.
Risk-based testing	Choosing and ordering tests by probability and impact of failure.
Severity and priority	Effect of a defect on users, and the order in which defects should be fixed.
Test oracle	The source of the expected result of a test.
Traceability	Links from requirement to test to result to defect.

ANNEXURE F – Source Code of the Experiments (evidence for every measured number)

The listings below are the exact files that produced the measured results. Running `python run_all.py` and then `python analyze.py` regenerates every number in this answer.

F1. mediqueue.py – business rules and injectable faults

```
"""MediQueue prototype – business rules with switchable seeded faults.

Rules(faults=set()) gives the correct behaviour; each fault id switches on one defect.
Fault ids
  Task 1 / 9 / 10 : F1 age>60, F2 child 10%, F3 4-hour limit, F4 age 4 treated as 5, F5 missing fee <1 h
  Task 3          : R1 rounding by truncation (with F1 as the wrong age limit)
  Task 4          : A1 3 active allowed, A2 same-second double booking (D4), A3 cancel limit 4 h,
                    A4 cancelled slot not freed, A5 booking in the past allowed
  Task 5 / 9      : P1 validity compares months (D9), P2 qty 31 allowed, P3 expired batch on expiry day,
                    P4 duplicate dispense by second pharmacist
  Task 10         : E1 video timeout in seconds not minutes, E2 e-Rx accepted unsigned,
                    E3 health-ID name compare ignores apostrophe normalisation, E4 audit entry not written
"""
from datetime import date, datetime, timedelta
from decimal import Decimal, ROUND_HALF_UP, ROUND_DOWN

BASE_FEE = 500
MAX_QTY = 30
VIDEO_TIMEOUT_MIN = 30



class Rules:
    def __init__(self, faults=()):
        self.f = set(faults)
        self.slots = {}          # (doctor, slot) -> patient
        self.active = {}         # patient -> count
        self.dispensed = set()   # prescription ids
        self.audit = []

    # ---------------- Billing ----------------
    def concession_fee(self, age, base_fee=BASE_FEE):
        if age < 0:
            raise ValueError("age must be >= 0")
        free_limit = 4 if "F4" in self.f else 5
        child_pct = Decimal("0.10") if "F2" in self.f else Decimal("0.20")
        senior = age > 60 if "F1" in self.f else age >= 60
        if age < free_limit:
```

```
        pct = Decimal(1)
    elif age <= 17:
        pct = child_pct
    elif senior:
        pct = Decimal("0.40")
    else:
        pct = Decimal(0)
    fee = Decimal(base_fee) * (1 - pct)
    mode = ROUND_DOWN if "R1" in self.f else ROUND_HALF_UP
    return int(fee.quantize(Decimal(1), rounding=mode))


def late_cancel_fee(self, hours_before):
    limit = 4 if "F3" in self.f else 2
    if "F5" in self.f and hours_before < 1:
        return 0                      # seeded: last-minute branch forgets the fee
    return 100 if hours_before < limit else 0


# ---------------- Appointment ----------------
def book(self, patient, doctor, slot_time, now, same_second_race=False):
    if slot_time < now and "A5" not in self.f:
        return "REJECT_PAST"
    limit = 3 if "A1" in self.f else 2
    if self.active.get(patient, 0) >= limit:
        return "REJECT_LIMIT"
    key = (doctor, slot_time)
    taken = key in self.slots
    if taken and not (same_second_race and "A2" in self.f):
        return "REJECT_TAKEN"
    self.slots[key] = patient
    self.active[patient] = self.active.get(patient, 0) + 1
    return "BOOKED"


def cancel(self, patient, doctor, slot_time, now):
    hours = (slot_time - now).total_seconds() / 3600
    limit = 4 if "A3" in self.f else 2
    fee = 100 if hours < limit else 0
    if "A4" not in self.f:
        self.slots.pop((doctor, slot_time), None)
    self.active[patient] = max(0, self.active.get(patient, 0) - 1)
    return fee


# ---------------- Pharmacy ----------------
def can_dispense(self, rx_id, rx_date, today, qty, batch_expiry, pharmacist):
    if "P1" in self.f:
        valid = (today.year, today.month) == (rx_date.year, rx_date.month) or (today - rx_date).days <= 7
    else:
        valid = 0 <= (today - rx_date).days <= 7
    if not valid:
        return "REJECT_EXPIRED_RX"
    if qty > (MAX_QTY + 1 if "P2" in self.f else MAX_QTY):
        return "REJECT_QTY"
    if (batch_expiry < today) if "P3" in self.f else (batch_expiry <= today):
        return "REJECT_BATCH"
    key = rx_id if "P4" not in self.f else (rx_id, pharmacist)
    if key in self.dispensed:
        return "REJECT_DUPLICATE"
    self.dispensed.add(key)
    return "DISPENSE"


# ---------------- Task 10 extensions ----------------
def video_session_active(self, idle_minutes):
    limit = VIDEO_TIMEOUT_MIN if "E1" not in self.f else VIDEO_TIMEOUT_MIN / 60
    return idle_minutes < limit


def accept_erx(self, issued, today, signed):
    if not signed and "E2" not in self.f:
        return False
    return 0 <= (today - issued).days <= 7
```

```
    @staticmethod
    def _norm(name):
        return name.replace("’", "'").strip().lower()

    def link_health_id(self, local_name, local_dob, hid_name, hid_dob, user="tester"):
        if "E3" in self.f:
            ok = local_name.strip().lower() == hid_name.strip().lower() and local_dob == hid_dob
        else:
            ok = self._norm(local_name) == self._norm(hid_name) and local_dob == hid_dob
        if "E4" not in self.f:
            self.audit.append((user, "LINK_ATTEMPT", local_name, ok))
        return ok



# Faults used per task (each mutant switches on exactly ONE fault)
T1_FAULTS = ["F1", "F2", "F3", "F4", "F5"]
T3_FAULTS = ["F1", "R1"]
T4_FAULTS_V1 = ["A1", "A2", "A3"]
T4_FAULTS_V2 = ["A4", "A5"]
T5_FAULTS = ["P1", "P2", "P3", "P4"]
T9_FAULTS = ["F1", "F2", "F3", "F4", "P1", "P2"]
T10_FAULTS = ["E1", "E2", "E3", "E4", "F1", "P1"]
```

F2. tests/conftest.py – harness that injects the mutant and logs every result

```
import json, os, pytest, sys
sys.path.insert(0, os.path.dirname(os.path.dirname(__file__)))
from mediqueue import Rules


MUT = os.environ.get("MUTANT", "")


@pytest.fixture
def R():
    return Rules({MUT} if MUT else set())


def pytest_runtest_logreport(report):
    if report.when == "call":
        with open(os.environ.get("RESULTS", "results.jsonl"), "a") as fh:
            fh.write(json.dumps({"suite": report.nodeid.split("::")[0].split("/")[-1].replace(".py", ""),
                                 "test": report.nodeid.split("::")[-1], "mutant": MUT or "none",
                                 "outcome": report.outcome}) + "\n")
```

F3. run_all.py – runs every suite on the correct build and each mutant

```
"""Runs every suite against the correct build and each single-fault mutant; writes results.jsonl."""
import os, subprocess, sys, time, json
from mediqueue import *
HERE = os.path.dirname(os.path.abspath(__file__))
RES = os.path.join(HERE, "results.jsonl")
if os.path.exists(RES): os.remove(RES)
PLAN = {
 "test_t1_A_testlast": T1_FAULTS, "test_t1_B_review": T1_FAULTS, "test_t1_C_continuous": T1_FAULTS,
 "test_t3_A_mental": T3_FAULTS, "test_t3_B_spectable": T3_FAULTS, "test_t3_C_reference": T3_FAULTS,
"test_t3_D8": ["F2"],
 "test_t4_B_experience": T4_FAULTS_V1 + T4_FAULTS_V2, "test_t4_C_risk_v1": T4_FAULTS_V1 + T4_FAULTS_V2,
 "test_t4_C_risk_v2": T4_FAULTS_V1 + T4_FAULTS_V2,
 "test_t5_A_peerdev": T5_FAULTS, "test_t5_B_independent": T5_FAULTS, "test_t5_C_domain": T5_FAULTS,
 "test_t9_A_lean": T9_FAULTS, "test_t9_B_structured": T9_FAULTS, "test_t9_C_assured": T9_FAULTS,
 "test_t10_A_trim": T10_FAULTS, "test_t10_B_auto_synth": T10_FAULTS, "test_t10_C_shiftleft": T10_FAULTS,
}
timing = {}
for suite, faults in PLAN.items():
    for m in [""] + faults:
        env = dict(os.environ, MUTANT=m, RESULTS=RES)
        t = time.perf_counter()
        subprocess.run([sys.executable, "-m", "pytest", "-q", "-p", "no:cacheprovider", f"tests/{suite}.py"],
                       cwd=HERE, env=env, capture_output=True)
```

```
        timing[(suite, m or "none")] = time.perf_counter() - t
json.dump({f"{k[0]}|{k[1]}": v for k, v in timing.items()}, open(os.path.join(HERE, "timing.json"), "w"),
indent=1)
print("done", len(timing), "runs")
```

F4. analyze.py – faults found and false reports per suite

```
import json, collections
rows = [json.loads(l) for l in open("results.jsonl")]
agg = collections.defaultdict(lambda: [0, 0])  # (suite, mutant) -> [fails, total]
fails_by = collections.defaultdict(list)
for r in rows:
    k = (r["suite"], r["mutant"]); agg[k][1] += 1
    if r["outcome"] == "failed": agg[k][0] += 1; fails_by[k].append(r["test"])
suites = sorted({k[0] for k in agg}, key=lambda s: (int(s.split('_')[1][1:]), s))
for s in suites:
    ms = [k[1] for k in agg if k[0] == s]
    line = []
    for m in ["none"] + sorted(x for x in ms if x != "none"):
        f, t = agg[(s, m)]
        line.append(f"{m}:{f}/{t}")
    print(s.ljust(26), "  ".join(line))
print()
for k in
[("test_t3_B_spectable","F1"),("test_t3_B_spectable","R1"),("test_t3_C_reference","R1"),("test_t3_D8","none"),
("test_t3_D8","F2"),("test_t4_B_experience","A2"),("test_t5_A_peerdev","none"),("test_t10_B_auto_synth","E3")]
:
    print(k, fails_by.get(k, []))
```

F5. Test suites (19 files) and helpers

tests/test_t1_B_review.py

```
# Task 1 Model B: system tests derived from the REVIEWED requirement (typical values + clarified rules)
def test_adult(R): assert R.concession_fee(30) == 500
def test_senior(R): assert R.concession_fee(70) == 300
def test_child(R): assert R.concession_fee(12) == 400
def test_infant(R): assert R.concession_fee(3) == 0
def test_cancel_1h(R): assert R.late_cancel_fee(1) == 100
def test_cancel_3h_no_fee(R): assert R.late_cancel_fee(3) == 0   # clarified: "less than 2 h before slot
start"
```

tests/test_t1_C_continuous.py

```
# Task 1 Model C: tests designed at requirement stage with boundary values, run on every build
import pytest
@pytest.mark.parametrize("age,fee", [(4, 0), (5, 400), (17, 400), (18, 500), (59, 500), (60, 300), (61, 300)])
def test_age_boundaries(R, age, fee): assert R.concession_fee(age) == fee
@pytest.mark.parametrize("hours,fee", [(0.5, 100), (1.99, 100), (2.0, 0), (2.01, 0)])
def test_cancel_boundaries(R, hours, fee): assert R.late_cancel_fee(hours) == fee
```

tests/test_t3_A_mental.py

```
# Task 3 Oracle A: expected value worked out "in the head" while testing; only round numbers chosen
import pytest
@pytest.mark.parametrize("age,base,exp", [(10,500,400),(30,500,500),(65,500,300),(3,500,0),(45,500,500),
                                          (8,500,400),(70,500,300),(25,500,500),(15,500,400),(80,500,300)])
def test_mental(R, age, base, exp): assert R.concession_fee(age, base) == exp
```

tests/test_t3_B_spectable.py

```
# Task 3 Oracle B: decision-table oracle (band, %, rounding half-up) written before testing
import pytest
TABLE = [(0,500,0),(4,500,0),(5,500,400),(17,500,400),(18,500,500),(59,500,500),(60,500,300),(61,500,300),
         (10,333,266),   # 333 x 0.8 = 266.4 -> 266
         (65,333,200)]   # 333 x 0.6 = 199.8 -> 200 (half-up)
@pytest.mark.parametrize("age,base,exp", TABLE)
```

```
def test_table(R, age, base, exp): assert R.concession_fee(age, base) == exp
```

tests/test_t3_C_reference.py

```
# Task 3 Oracle C: independent reference implementation written by a second person (back-to-back)
import pytest
def ref_fee(age, base):
    pct = 100 if age < 5 else 20 if age < 18 else 0 if age < 60 else 40
    cents = base * (100 - pct)            # integer arithmetic, then round half up to rupees
    return (cents + 50) // 100
INPUTS = [(2,500),(6,500),(16,333),(30,500),(59,500),(60,500),(62,333),(90,501),(12,250),(70,333)]
@pytest.mark.parametrize("age,base", INPUTS)
def test_back_to_back(R, age, base): assert R.concession_fee(age, base) == ref_fee(age, base)
```

tests/test_t3_D8.py

```
# D8 demonstration: TC-52 carries a WRONG expected value (450 instead of 400)
def test_tc52_wrong_oracle(R): assert R.concession_fee(12) == 450
```

tests/test_t4_B_experience.py

```
# Task 4 Strategy B: 25 cases picked by the senior tester from memory of earlier releases
import pytest
from apt_helpers import NOW, slot
from datetime import timedelta
@pytest.mark.parametrize("doc", range(10))
def test_normal_booking(R, doc): assert R.book(f"P{doc}", f"D{doc}", slot(10), NOW) == "BOOKED"
@pytest.mark.parametrize("doc", range(5))
def test_taken_slot_rejected(R, doc):
    R.book("PA", f"D{doc}", slot(11), NOW); assert R.book("PB", f"D{doc}", slot(11), NOW) == "REJECT_TAKEN"
@pytest.mark.parametrize("hours,fee", [(6, 0), (1, 100), (10, 0), (0.5, 100), (3, 0)])
def test_cancel_fee(R, hours, fee):
    s = NOW + timedelta(hours=hours); R.book("PC", "DX", s, NOW); assert R.cancel("PC", "DX", s, NOW) == fee
@pytest.mark.parametrize("n", range(5))
def test_active_limit(R, n):
    R.book("PL", "D1", slot(9), NOW); R.book("PL", "D2", slot(9), NOW)
    assert R.book("PL", "D3", slot(9 + n % 3 + 1), NOW) == "REJECT_LIMIT"
```

tests/test_t4_C_risk_v1.py

```
# Task 4 Strategy C (release N): principle-guided, risk-based 25 cases
import pytest
from apt_helpers import NOW, slot
from datetime import timedelta
# defect clustering -> concurrency + limits + boundary of the 2-hour rule
@pytest.mark.parametrize("doc", range(5))
def test_same_second_race(R, doc):            # D4
    R.book("PA", f"D{doc}", slot(10, 15), NOW, same_second_race=True)
    assert R.book("PB", f"D{doc}", slot(10, 15), NOW, same_second_race=True) == "REJECT_TAKEN"
@pytest.mark.parametrize("third_hour", [11, 12, 13, 14, 15])
def test_third_active_rejected(R, third_hour):
    R.book("PL", "D1", slot(9), NOW); R.book("PL", "D2", slot(10), NOW)
    assert R.book("PL", "D3", slot(third_hour), NOW) == "REJECT_LIMIT"
@pytest.mark.parametrize("hours,fee", [(1.99, 100), (2.0, 0), (2.01, 0), (0.25, 100), (3, 0), (4, 0)])
def test_cancel_boundary(R, hours, fee):
    s = NOW + timedelta(hours=hours); R.book("PC", "DX", s, NOW); assert R.cancel("PC", "DX", s, NOW) == fee
@pytest.mark.parametrize("doc", range(9))
def test_booking_core(R, doc): assert R.book(f"P{doc}", f"D{doc}", slot(10), NOW) == "BOOKED"
```

tests/test_t4_C_risk_v2.py

```
# Task 4 Strategy C (release N+1): same principles, suite REFRESHED (pesticide paradox) – 4 core cases swapped
import pytest
from apt_helpers import NOW, slot
from datetime import timedelta
@pytest.mark.parametrize("doc", range(5))
def test_same_second_race(R, doc):
    R.book("PA", f"D{doc}", slot(10, 15), NOW, same_second_race=True)
```

```
    assert R.book("PB", f"D{doc}", slot(10, 15), NOW, same_second_race=True) == "REJECT_TAKEN"
@pytest.mark.parametrize("third_hour", [11, 12, 13, 14, 15])
def test_third_active_rejected(R, third_hour):
    R.book("PL", "D1", slot(9), NOW); R.book("PL", "D2", slot(10), NOW)
    assert R.book("PL", "D3", slot(third_hour), NOW) == "REJECT_LIMIT"
@pytest.mark.parametrize("hours,fee", [(1.99, 100), (2.0, 0), (2.01, 0), (0.25, 100), (3, 0), (4, 0)])
def test_cancel_boundary(R, hours, fee):
    s = NOW + timedelta(hours=hours); R.book("PC", "DX", s, NOW); assert R.cancel("PC", "DX", s, NOW) == fee
@pytest.mark.parametrize("doc", range(5))
def test_booking_core(R, doc): assert R.book(f"P{doc}", f"D{doc}", slot(10), NOW) == "BOOKED"
@pytest.mark.parametrize("doc", range(2))
def test_cancelled_slot_is_freed(R, doc):     # new code area in N+1
    R.book("PA", f"D{doc}", slot(12), NOW); R.cancel("PA", f"D{doc}", slot(12), NOW)
    assert R.book("PB", f"D{doc}", slot(12), NOW) == "BOOKED"
@pytest.mark.parametrize("hours_ago", [1, 24])
def test_past_slot_rejected(R, hours_ago):
    assert R.book("PZ", "D9", NOW - timedelta(hours=hours_ago), NOW) == "REJECT_PAST"
```

tests/test_t5_A_peerdev.py

```
# Task 5 Arrangement A: developer from another module, code-oriented tests, no clinical input
from datetime import date
from rx_helpers import TODAY
def test_recent_rx(R): assert R.can_dispense("RX1", date(2026,3,9), TODAY, 10, date(2027,1,1), "ph1") ==
"DISPENSE"
def test_ten_day_old_rx(R): assert R.can_dispense("RX2", date(2026,3,2), TODAY, 10, date(2027,1,1), "ph1") ==
"REJECT_EXPIRED_RX"
def test_big_qty(R): assert R.can_dispense("RX3", date(2026,3,10), TODAY, 50, date(2027,1,1), "ph1") ==
"REJECT_QTY"
def test_expired_batch(R): assert R.can_dispense("RX4", date(2026,3,10), TODAY, 10, date(2026,2,1), "ph1") ==
"REJECT_BATCH"
def test_duplicate_same_pharmacist(R):
    R.can_dispense("RX5", date(2026,3,10), TODAY, 10, date(2027,1,1), "ph1")
    assert R.can_dispense("RX5", date(2026,3,10), TODAY, 10, date(2027,1,1), "ph1") == "REJECT_DUPLICATE"
def test_day7_rejected_misread(R):   # developer misread "within 7 days" as "< 7 days"  -> false report
    assert R.can_dispense("RX6", date(2026,3,5), TODAY, 10, date(2027,1,1), "ph1") == "REJECT_EXPIRED_RX"
def test_qty30_rejected_misread(R):  # developer assumed max is 29 -> false report
    assert R.can_dispense("RX7", date(2026,3,10), TODAY, 30, date(2027,1,1), "ph1") == "REJECT_QTY"
```

tests/test_t5_B_independent.py

```
# Task 5 Arrangement B: independent tester, tests from the written requirement with boundary values
from datetime import date
from rx_helpers import TODAY
def test_day7_valid(R): assert R.can_dispense("RX1", date(2026,3,5), TODAY, 10, date(2027,1,1), "ph1") ==
"DISPENSE"
def test_day8_invalid(R): assert R.can_dispense("RX2", date(2026,3,4), TODAY, 10, date(2027,1,1), "ph1") ==
"REJECT_EXPIRED_RX"
def test_qty30_ok(R): assert R.can_dispense("RX3", date(2026,3,10), TODAY, 30, date(2027,1,1), "ph1") ==
"DISPENSE"
def test_qty31_rejected(R): assert R.can_dispense("RX4", date(2026,3,10), TODAY, 31, date(2027,1,1), "ph1") ==
"REJECT_QTY"
def test_batch_expired_yesterday(R): assert R.can_dispense("RX5", date(2026,3,10), TODAY, 10, date(2026,3,11),
"ph1") == "REJECT_BATCH"
def test_duplicate_same_pharmacist(R):
    R.can_dispense("RX6", date(2026,3,10), TODAY, 10, date(2027,1,1), "ph1")
    assert R.can_dispense("RX6", date(2026,3,10), TODAY, 10, date(2027,1,1), "ph1") == "REJECT_DUPLICATE"
```

tests/test_t5_C_domain.py

```
# Task 5 Arrangement C: tester + pharmacist agree acceptance examples before coding
from datetime import date
from rx_helpers import TODAY
def test_day7_valid(R): assert R.can_dispense("RX1", date(2026,3,5), TODAY, 10, date(2027,1,1), "ph1") ==
"DISPENSE"
def test_day8_invalid(R): assert R.can_dispense("RX2", date(2026,3,4), TODAY, 10, date(2027,1,1), "ph1") ==
"REJECT_EXPIRED_RX"
def test_qty31_rejected(R): assert R.can_dispense("RX4", date(2026,3,10), TODAY, 31, date(2027,1,1), "ph1") ==
"REJECT_QTY"
def test_batch_expiring_today_blocked(R):   # pharmacist: never dispense on the expiry date
```

```
    assert R.can_dispense("RX5", date(2026,3,10), TODAY, 10, TODAY, "ph1") == "REJECT_BATCH"
def test_duplicate_other_pharmacist(R):     # pharmacist: shift change re-dispense risk
    R.can_dispense("RX6", date(2026,3,10), TODAY, 10, date(2027,1,1), "ph1")
    assert R.can_dispense("RX6", date(2026,3,10), TODAY, 10, date(2027,1,1), "ph2") == "REJECT_DUPLICATE"
```

tests/test_t9_A_lean.py

```
# Task 9 Config A (lean): developer self-tests, happy path only
from datetime import date
from rx_helpers import TODAY
def test_adult(R): assert R.concession_fee(35) == 500
def test_senior(R): assert R.concession_fee(72) == 300
def test_child(R): assert R.concession_fee(9) == 400
def test_cancel_1h(R): assert R.late_cancel_fee(1) == 100
def test_cancel_5h(R): assert R.late_cancel_fee(5) == 0
def test_rx_recent(R): assert R.can_dispense("A1", date(2026,3,10), TODAY, 10, date(2027,1,1), "ph1") ==
"DISPENSE"
```

tests/test_t9_B_structured.py

```
# Task 9 Config B (structured): risk-based boundary tests by an independent tester after requirement review
import pytest
from datetime import date
from rx_helpers import TODAY
@pytest.mark.parametrize("age,fee", [(4,0),(5,400),(17,400),(18,500),(59,500),(60,300)])
def test_age(R, age, fee): assert R.concession_fee(age) == fee
@pytest.mark.parametrize("h,fee", [(1.99,100),(2.0,0),(3,0)])
def test_cancel(R, h, fee): assert R.late_cancel_fee(h) == fee
def test_rx_day7(R): assert R.can_dispense("B1", date(2026,3,5), TODAY, 10, date(2027,1,1), "ph1") ==
"DISPENSE"
def test_rx_day8(R): assert R.can_dispense("B2", date(2026,3,4), TODAY, 10, date(2027,1,1), "ph1") ==
"REJECT_EXPIRED_RX"
def test_qty30(R): assert R.can_dispense("B3", date(2026,3,10), TODAY, 30, date(2027,1,1), "ph1") ==
"DISPENSE"
```

tests/test_t9_C_assured.py

```
# Task 9 Config C (assured): B's cases automated on every build + domain-expert cases (qty limit, expiry)
import pytest
from datetime import date
from rx_helpers import TODAY
@pytest.mark.parametrize("age,fee", [(0,0),(4,0),(5,400),(17,400),(18,500),(59,500),(60,300),(61,300)])
def test_age(R, age, fee): assert R.concession_fee(age) == fee
@pytest.mark.parametrize("h,fee", [(0.5,100),(1.99,100),(2.0,0),(2.01,0)])
def test_cancel(R, h, fee): assert R.late_cancel_fee(h) == fee
@pytest.mark.parametrize("rx,res",
[(date(2026,3,5),"DISPENSE"),(date(2026,3,4),"REJECT_EXPIRED_RX"),(date(2026,2,28),"REJECT_EXPIRED_RX")])
def test_rx(R, rx, res): assert R.can_dispense("C"+str(rx), rx, TODAY, 10, date(2027,1,1), "ph1") == res
@pytest.mark.parametrize("q,res", [(30,"DISPENSE"),(31,"REJECT_QTY")])
def test_qty(R, q, res): assert R.can_dispense("Q"+str(q), date(2026,3,10), TODAY, q, date(2027,1,1), "ph1")
== res
```

tests/test_t10_B_auto_synth.py

```
# Task 10 Strategy B: automated CI regression + generated synthetic patients (clean ASCII names)
import pytest
from datetime import date
from rx_helpers import TODAY
SYNTH = [("Patient%03d" % i, date(1960 + i % 40, 1 + i % 12, 1 + i % 28)) for i in range(20)]
def test_senior_boundary(R): assert R.concession_fee(60) == 300
def test_rx_day8(R): assert R.can_dispense("S1", date(2026,3,4), TODAY, 10, date(2027,1,1), "ph1") ==
"REJECT_EXPIRED_RX"
@pytest.mark.parametrize("idle,active", [(5, True), (29, True), (30, False)])
def test_video_timeout(R, idle, active): assert R.video_session_active(idle) is active
def test_erx_unsigned_rejected(R): assert R.accept_erx(date(2026,3,10), TODAY, signed=False) is False
@pytest.mark.parametrize("name,dob", SYNTH)
def test_health_id_synthetic(R, name, dob): assert R.link_health_id(name, dob, name, dob) is True
```

tests/test_t10_C_shiftleft.py

```
# Task 10 Strategy C: review + contract tests for the health-ID interface + audit assertion; core automated
from datetime import date
from rx_helpers import TODAY
def test_senior_boundary(R): assert R.concession_fee(60) == 300
def test_rx_day8(R): assert R.can_dispense("C1", date(2026,3,4), TODAY, 10, date(2027,1,1), "ph1") ==
"REJECT_EXPIRED_RX"
def test_video_timeout_unit_minutes(R): assert R.video_session_active(29) is True and
R.video_session_active(30) is False
def test_erx_unsigned_rejected(R): assert R.accept_erx(date(2026,3,10), TODAY, signed=False) is False
def test_contract_curly_apostrophe(R):   # national ID returns U+2019, local DB stores ASCII '
    assert R.link_health_id("Anita D'Souza", date(1990,5,1), "ANITA D’SOUZA", date(1990,5,1)) is True
def test_contract_dob_mismatch_blocked(R):
    assert R.link_health_id("Ravi Kumar", date(1985,2,2), "Ravi Kumar", date(1985,2,3)) is False
def test_audit_entry_written(R):
    R.link_health_id("Ravi Kumar", date(1985,2,2), "Ravi Kumar", date(1985,2,2), user="qa1")
    assert R.audit and R.audit[-1][0] == "qa1"
```

tests/test_t1_A_testlast.py

```
# Task 1 Model A: test-last screen checklist, typical values only, written after coding
def test_adult_fee(R): assert R.concession_fee(30) == 500
def test_senior_fee(R): assert R.concession_fee(65) == 300
def test_child_fee(R): assert R.concession_fee(10) == 400
def test_cancel_late(R): assert R.late_cancel_fee(1.5) == 100
def test_cancel_early(R): assert R.late_cancel_fee(6) == 0
```

tests/test_t10_A_trim.py

```
# Task 10 Strategy A: retain Task 9 practice, run only the highest clinical/financial risk cases each week
from datetime import date
from rx_helpers import TODAY
def test_senior_boundary(R): assert R.concession_fee(60) == 300
def test_rx_day8(R): assert R.can_dispense("T1", date(2026,3,4), TODAY, 10, date(2027,1,1), "ph1") ==
"REJECT_EXPIRED_RX"
def test_video_smoke(R): assert R.video_session_active(5) is True
```

tests/apt_helpers.py

```
from datetime import datetime, timedelta
NOW = datetime(2026, 11, 2, 8, 0)
def slot(h, m=0, day=3): return datetime(2026, 11, day, h, m)
```

tests/rx_helpers.py

```
from datetime import date
TODAY = date(2026, 3, 12)
```

F6. Output of analyze.py (failing tests / total, per build)

```
test_t1_A_testlast         none:0/25  F1:0/5  F2:1/5  F3:0/5  F4:0/5  F5:0/5
test_t1_B_review           none:0/30  F1:0/6  F2:1/6  F3:1/6  F4:0/6  F5:0/6
test_t1_C_continuous       none:0/55  F1:1/11  F2:2/11  F3:2/11  F4:1/11  F5:1/11
test_t3_A_mental           none:0/50  F1:0/10  R1:0/10
test_t3_B_spectable        none:0/50  F1:1/10  R1:1/10
test_t3_C_reference        none:0/50  F1:1/10  R1:3/10
test_t3_D8                 none:5/5  F2:0/1
test_t4_B_experience       none:0/125  A1:5/25  A2:0/25  A3:1/25  A4:0/25  A5:0/25
test_t4_C_risk_v1          none:0/125  A1:5/25  A2:5/25  A3:3/25  A4:0/25  A5:0/25
test_t4_C_risk_v2          none:0/125  A1:5/25  A2:5/25  A3:3/25  A4:2/25  A5:2/25
test_t5_A_peerdev          none:10/35  P1:3/7  P2:2/7  P3:2/7  P4:2/7
test_t5_B_independent      none:0/30  P1:1/6  P2:1/6  P3:0/6  P4:0/6
test_t5_C_domain           none:0/25  P1:1/5  P2:1/5  P3:1/5  P4:1/5
test_t9_A_lean             none:0/30  F1:0/6  F2:1/6  F3:0/6  F4:0/6  P1:0/6  P2:0/6
test_t9_B_structured       none:0/60  F1:1/12  F2:2/12  F3:2/12  F4:1/12  P1:1/12  P2:0/12
test_t9_C_assured          none:0/85  F1:1/17  F2:2/17  F3:2/17  F4:1/17  P1:1/17  P2:1/17
test_t10_A_trim            none:0/15  E1:1/3  E2:0/3  E3:0/3  E4:0/3  F1:1/3  P1:1/3
```

```
test_t10_B_auto_synth      none:0/130  E1:2/26  E2:1/26  E3:0/26  E4:0/26  F1:1/26  P1:1/26
test_t10_C_shiftleft       none:0/35  E1:1/7  E2:1/7  E3:1/7  E4:1/7  F1:1/7  P1:1/7


('test_t3_B_spectable', 'F1') ['test_table[60-500-300]']
('test_t3_B_spectable', 'R1') ['test_table[65-333-200]']
('test_t3_C_reference', 'R1') ['test_back_to_back[62-333]', 'test_back_to_back[90-501]',
'test_back_to_back[70-333]']
('test_t3_D8', 'none') ['test_tc52_wrong_oracle', 'test_tc52_wrong_oracle', 'test_tc52_wrong_oracle',
'test_tc52_wrong_oracle', 'test_tc52_wrong_oracle']
('test_t3_D8', 'F2') []
('test_t4_B_experience', 'A2') []
('test_t5_A_peerdev', 'none') ['test_day7_rejected_misread', 'test_qty30_rejected_misread',
'test_day7_rejected_misread', 'test_qty30_rejected_misread', 'test_day7_rejected_misread',
'test_qty30_rejected_misread', 'test_day7_rejected_misread', 'test_qty30_rejected_misread',
'test_day7_rejected_misread', 'test_qty30_rejected_misread']
('test_t10_B_auto_synth', 'E3') []
```

ANNEXURE G – Viva Questions and Model Answers (Tasks 1–10)

Five questions per task that an examiner is likely to ask, answered from the measured results in this document.

Task 1 – Testing activity models

Q1. Why did the test-last model find only one of the five faults?

Its five tests used typical values (ages 30, 65, 10; cancellation 1.5 h and 6 h). Four faults sit on boundaries – age 60, age 4/5, the 2-hour limit and the under-1-hour branch – and no typical value reaches them, so the suite passed on four faulty builds.

Q2. How were the ambiguities found by models B and C?

B found them by reviewing the requirement text before coding; C found them because writing a boundary test for exactly 2.0 hours and for age 60 forces the question "which side does the edge belong to?". A never asks the question because it tests after coding with typical values.

Q3. What does "testing shows the presence of defects, not their absence" mean with your data?

Suite A was green on the F1, F3, F4 and F5 builds although each contained a fault. A passing run only shows that the chosen inputs did not trigger a failure.

Q4. How did you calculate the escape cost?

Each issue that no activity found is assumed to reach users and cost ₹45,000 to fix; A missed 6 of 7 issues, so 6 × ₹45,000 = ₹2,70,000, plus ₹9,000 to fix F2 at system test.

Q5. What would you check first in an LLM-written test plan?

Whether it states the boundaries exactly as the rules do ("60 and above", "less than 2 hours"), whether it lists the risky rules (rounding, 2 active appointments, double booking, refresh idempotency, 800 users / 2 s) and whether it has measurable exit criteria.

Task 2 – Process models

Q1. What is defect removal efficiency and what was it for the last release?

DRE is defects removed before release divided by all defects. 135 ÷ 164 = 82.3%.

Q2. Why does the DevOps pipeline alone predict a slightly lower DRE than the spiral?

The pipeline automates code-level checks well but has no requirement review by default; 12 of the 29 escapes were requirement defects, which the spiral's risk analysis with a doctor representative would catch more often. Adding a review gate gives 95.1%.

Q3. How does the cost table support early testing?

The same 15 defects cost ₹12,000 if found at requirements but ₹6,75,000 after release – 56 times more.

Q4. What TMMi level is the current practice and why?

Level 1 (Initial): testing is ad hoc, done at the end, with no test policy, metrics or review process.

Q5. Why 80% branch coverage for the quality gate?

It is reachable without writing meaningless tests, and it is applied to Billing, Appointment and Pharmacy, which hold 78% of the logged defects.

Task 3 – Test oracles

Q1. Why did the mental-calculation oracle miss both faults?

The tester chose round ages with a ₹500 base, so every concession is a whole number (truncation never shows) and nobody chose age 60.

Q2. Classify error, fault and failure for F1.

Error: the developer read "60 and above" as "above 60". Fault: the line senior = age > 60. Failure: concession_fee(60) returned ₹500 instead of ₹300.

Q3. What is the risk of a back-to-back (reference implementation) oracle?

If the second implementer makes the same misreading, both programs agree and the test passes on a fault; it traces to another program, not to the rule.

Q4. What did the D8 demonstration show?

TC-52 with the wrong expected value ₹450 passed on the faulty 10% build and failed on the correct 20% build – a wrong oracle hides the real fault and makes correct code look broken.

Q5. When is an LLM acceptable as an oracle?

Only for drafting expected values that are then checked against the decision table; its typical slips are the 4/5 and 59/60 edges and half-up rounding.

Task 4 – Prioritisation and principles

Q1. How large is the exhaustive input space?

250 × 20 × 3 × 3 × 2 × 2 = 1,80,000 combinations, 50 hours at one test per second – half the release budget for one module.

Q2. Show the pesticide paradox with your results.

The same 25 risk-based tests found 3 of 3 faults in release N and 0 of 2 in release N+1; refreshing four cases restored 100%.

Q3. Which modules hold about 80% of defects?

Billing (58), Appointment (48) and Pharmacy (22): 128 of 164 = 78%.

Q4. What is the absence-of-errors fallacy here?

The A2 build passed all 25 experience-driven tests, yet two patients confirming in the same second both got the 10:15 slot.

Q5. How would you refresh the suite next release?

Replace at least 20% of cases with tests for changed code, re-rank by the latest defect log and keep the double-booking tests permanently.

Task 5 – Tester responsibility

Q1. Why did the peer developer raise false reports?

He tested his own reading of the rules – "within 7 days" as "under 7 days" and a 29-unit maximum – so two tests failed on correct code (confirmation bias without the requirement).

Q2. What did the pharmacist add that testers could not?

The rule that a batch must not be dispensed on its expiry date (P3) and the shift-handover double-dispense risk (P4).

Q3. How does a good defect report shorten fixing time?

Steps, dates, data and the failing test point straight to the same-month comparison, so the cause is found in about an hour instead of half a day of questions.

Q4. Justify the tester allocation.

T1 Billing + Registration (42% of defects, 40 h), T2 Appointment + Reminders (35%, 35 h), T3 Pharmacy + Lab Reports with domain experts (23%, 25 h).

Q5. Who signs off patient-safety modules and why?

The QA lead on quality criteria and the medical officer for clinical risk; neither alone can accept both kinds of risk.

Task 6 – Defect origins

Q1. Which artefact carries most defects and which causes most escapes?

Code carries most (70 of 164), but requirements cause most escapes (12 of 29).

Q2. Compute the containment effectiveness of requirement review.

Defects caught at requirement review that came from requirements ÷ all requirement defects = 16 ÷ 46 = 34.8%.

Q3. What single fix blocks the D2 fault tree?

An idempotency key (or a unique constraint on consultation and bill): the right-hand AND branch then cannot be satisfied whatever the user does.

Q4. Which method best exposes D7?

Origin-by-artefact tagging: it tags D7 as a requirement defect, pointing at the silent specification rather than at the SMS code.

Q5. Why is "careless coding" the wrong diagnosis?

Only 7 of 29 escapes came from code; 59% came from requirement and design artefacts.

Task 7 – Classification

Q1. Give a high-severity, low-priority example and why.

D5: registration crashes for apostrophe surnames, but reception has a workaround, so it can wait for the next release.

Q2. Why does scheme B help prevention most?

It records both injection and detection phase, showing which filter leaked (requirements and design reached production).

Q3. Where do the schemes disagree on D8?

A and B call it a testing defect; C has no class for test artefacts and can only rate the billing fault it hides.

Q4. How is percentage agreement calculated?

Classes matching yours ÷ 10 × 100, for each classmate and for the pair.

Q5. When is LLM classification acceptable?

As a first draft confirmed by a tester; never as the final label for safety-critical defects.

Task 8 – Defect repository

Q1. Why JIRA over Bugzilla here?

Native links from defects to tests and requirements, mandatory fields per transition and a live module-by-severity dashboard; the 9-person team fits the free tier.

Q2. How are MTTF-style metrics computed from the life cycle?

Mean time to fix is the average of (Fixed − Open) over the defects; reopen rate is reopened ÷ fixed (1 of 2 = 50% in the walk-through).

Q3. Why is Billing not clearly the worst module?

Per KLOC Billing (4.83) and Appointment (4.80) are equal, and by escapes per KLOC Pharmacy is worst (1.00).

Q4. Which fields are compulsory and why?

Summary, module, build, steps, expected vs actual, severity, priority, reporter, assignee, origin and safety category – each answers a question that otherwise needs a round trip.

Q5. What states does the workflow need?

New, Open, Fixed, Retest, Verified, Closed, plus Reopened and Rejected (with a reason).

Task 9 – Integrated practice

Q1. Which configuration found all six faults?

C, the assured configuration, with 17 tests; B found 5 (it tested 30 units but not 31) and A found 1.

Q2. Which earlier prediction was contradicted?

That a structured boundary-based set catches limit faults: B missed P2 because the boundary was tested only on one side.

Q3. How are the 100 hours allocated?

Review 10, design 18, pipeline 20, system tests 30, logging 7, retest 10, release decision 2, metrics 3.

Q4. What did clean synthetic data hide?

Twenty generated ASCII names all passed the health-ID link on the faulty build; only the realistic name D'Souza exposed the apostrophe fault.

Q5. Why not choose the AI-assisted configuration?

Its detection depends on the LLM's oracle, its tests change between runs and it is weak on auditability – too risky for patient-safety modules.

Task 10 – Adaptation

Q1. Which assumptions of Task 9 no longer hold?

Effort (100 → 50 h), cycle (3 weeks → weekly), load (about 4× peak), patient-data risk (health-ID, video, e-Rx) and the new audit rule.

Q2. Why REDESIGN rather than MODIFY?

Only the shift-left strategy found the health-ID apostrophe and missing-audit faults (6 of 6 against 4 and 3) within 48 of 50 hours.

Q3. What audit evidence does C produce?

Contract-test results and an audit assertion on every test that touches patient data, stored with the CI run.

Q4. When would the decision no longer hold?

If the national health-ID provider gives no sandbox or interface specification – contract tests could not run.

Q5. Name two risks of a defect-prediction model.

Cold start (new tele-consultation code has no history) and a self-fulfilling loop (defects are found where testers already look).

Additional viva questions

Task 1 – Testing activity models: What is the difference between a fault and an ambiguity in your Task 1 table?

A fault is wrong code that a test can expose by running it; an ambiguity is a gap in the requirement text that no code test can expose until someone decides the intended behaviour.

Task 1 – Testing activity models: Why is C's effort higher but its cost lower?

C spends 14 ph instead of 6 ph, but every issue is fixed at the ₹800–₹2,500 stage, so its total cost is ₹14,100 against ₹2,79,000.

Task 1 – Testing activity models: What residual risk remains after C?

Concurrency (D4), performance (D6) and any fault outside the seeded set, because the unit-level boundary tests do not exercise them.

Task 3 – Test oracles: Name the five testing terms and one artefact for each.

Test: test_table[60-500-300]; test set: the 10 cases of strategy B; test suite: all Task 3 files; harness: conftest.py + run_all.py; oracle: the decision table.

Task 3 – Test oracles: Why use base ₹333 in the table?

333 × 0.8 = 266.4 and 333 × 0.6 = 199.8 produce fractions, so truncation and half-up rounding give different results (199 vs 200).

Task 6 – Defect origins: What is the difference between a Pareto chart and a fault tree?

A Pareto chart ranks causes by frequency across the whole log; a fault tree analyses one failure into AND/OR combinations of causes.

Task 6 – Defect origins: What reduction in post-release defects do you expect and why?

From 29 to about 18 (−38 %): review with example tables should catch about 8 of 12 requirement escapes and database guards about 3 of 5 design escapes.

Task 8 – Defect repository: What data-quality problems did the messaging-group reports have?

Vague summaries, missing steps, no build number and severity set by mood; each needs a round trip of questions before work can start.

Task 10 – Adaptation: How did you check that 48 hours fits the 50-hour limit?

Weekly plan: review 6, contract tests 6, pipeline upkeep 8, exploratory and domain tests 18, triage and retest 7, audit export and release 3 = 48 h, leaving 2 h in reserve.

Task 10 – Adaptation: Why did automation with synthetic data miss E3?

Generated names contain no apostrophes, so the health-ID comparison of ’ and ' was never exercised.

ANNEXURE H – Full Defect Reports for D1–D10

Each sample defect written to the standard of Task 5 and the compulsory fields of Task 8 (environment for all: Chrome 129, test build 3.4.x, PostgreSQL 15; reporter: QA tester; status: New).

D1 – Patient aged 60 charged ₹500 instead of ₹300

Field	Content
Module	Billing
Steps to reproduce	1. Register patient with DOB giving age 60 on the appointment date. 2. Book a consultation. 3. Open the bill.
Expected result	₹300 (40 % senior concession, “60 years and above”)
Actual result	₹500 – no concession
Severity / Priority	Major / High
Class (scheme B type)	Logic (comparison operator)
Origin artefact	Code
Test that detects it	test_table[60-500-300] (Task 3 B), Task 1 C age 60

D2 – Refreshing payment confirmation creates a second bill

Field	Content
Module	Billing
Steps to reproduce	1. Pay a consultation fee. 2. Press F5 on the confirmation page. 3. Open the patient’s bills.
Expected result	One bill per consultation
Actual result	Two bills for the same consultation
Severity / Priority	Major / High
Class (scheme B type)	Interface (missing idempotency)
Origin artefact	Design
Test that detects it	Duplicate-payment test with an idempotency key (Task 6 fault tree)

D3 – Late-cancellation fee charged 3 hours before the slot

Field	Content
Module	Appointment
Steps to reproduce	1. Book a 13:00 slot. 2. Cancel at 10:00. 3. View the fee.
Expected result	₹0 (fee only under 2 hours)
Actual result	₹100
Severity / Priority	Minor / Medium
Class (scheme B type)	Data (wrong constant)
Origin artefact	Code
Test that detects it	test_cancel_boundary[2.01-0], [3-0] (Task 4 C)

D4 – Two patients both given the 10:15 slot

Field	Content
Module	Appointment
Steps to reproduce	1. Two sessions select Dr X 10:15. 2. Both press Confirm within one second.
Expected result	Second confirmation rejected

Field	Content
Actual result	Both confirmed
Severity / Priority	Major / High
Class (scheme B type)	Concurrency
Origin artefact	Design
Test that detects it	test_same_second_race (Task 4 C)

D5 – Surname with apostrophe cannot be registered

Field	Content
Module	Registration
Steps to reproduce	1. Open registration. 2. Enter surname D’Souza. 3. Save.
Expected result	Patient saved
Actual result	Database error shown
Severity / Priority	Major / Low
Class (scheme B type)	Validation (unescaped input)
Origin artefact	Code
Test that detects it	Health-ID contract test with ’ (Task 10 C)

D6 – Lab-report page takes 35 s at 800 users

Field	Content
Module	Lab Reports
Steps to reproduce	1. Run load test with 800 virtual users. 2. Open a lab report.
Expected result	≤ 2 s
Actual result	≈ 35 s
Severity / Priority	Major / High
Class (scheme B type)	Performance
Origin artefact	Design
Test that detects it	Load test at 800 users (Task 9 practice)

D7 – SMS shows old time after reschedule

Field	Content
Module	Reminders
Steps to reproduce	1. Book 10:00. 2. Reschedule to 15:00. 3. Wait for the 24-hour SMS.
Expected result	SMS with 15:00
Actual result	SMS with 10:00
Severity / Priority	Major / Medium
Class (scheme B type)	Requirement gap
Origin artefact	Requirement
Test that detects it	Requirement review with a reschedule example (Task 6 action 1)

D8 – TC-52 expects ₹450 for a 12-year-old

Field	Content
Module	Billing (test)
Steps to reproduce	1. Review TC-52 against the concession rule.

Field	Content
Expected result	₹400 (20 % off ₹500)
Actual result	Test expects ₹450 and passes on 10 % code
Severity / Priority	Minor / High
Class (scheme B type)	Test oracle
Origin artefact	Test case
Test that detects it	test_tc52_wrong_oracle demonstration (Task 3)

D9 – Prescription 9 days old is dispensed

Field	Content
Module	Pharmacy
Steps to reproduce	1. Create Rx dated today − 9 in the same month. 2. Dispense.
Expected result	"Prescription expired"
Actual result	Medicine dispensed
Severity / Priority	Critical / Critical
Class (scheme B type)	Logic (wrong unit)
Origin artefact	Code
Test that detects it	test_day8_invalid (Task 5 B, C)

D10 – Lab PDF shows collection date, not issue date

Field	Content
Module	Lab Reports
Steps to reproduce	1. Issue a blood report. 2. Open the PDF. 3. Check the date.
Expected result	Report-issue date
Actual result	Sample-collection date
Severity / Priority	Major / Medium
Class (scheme B type)	Data mapping
Origin artefact	Data
Test that detects it	Field-mapping check in code review (Task 6 Pareto action)

Seeded faults of the prototype and the suites that detected them

Fault	Detected by (suites)	Missed by (suites)
A1	t4_B_experience, t4_C_risk_v1, t4_C_risk_v2	none
A2	t4_C_risk_v1, t4_C_risk_v2	t4_B_experience
A3	t4_B_experience, t4_C_risk_v1, t4_C_risk_v2	none
A4	t4_C_risk_v2	t4_B_experience, t4_C_risk_v1
A5	t4_C_risk_v2	t4_B_experience, t4_C_risk_v1
E1	t10_A_trim, t10_B_auto_synth, t10_C_shiftleft	none
E2	t10_B_auto_synth, t10_C_shiftleft	t10_A_trim
E3	t10_C_shiftleft	t10_A_trim, t10_B_auto_synth
E4	t10_C_shiftleft	t10_A_trim, t10_B_auto_synth
F1	t10_A_trim, t10_B_auto_synth, t10_C_shiftleft, t1_C_continuous, t3_B_spectable, t3_C_reference, t9_B_structured, t9_C_assured	t1_A_testlast, t1_B_review, t3_A_mental, t9_A_lean
F2	t1_A_testlast, t1_B_review, t1_C_continuous, t9_A_lean, t9_B_structured, t9_C_assured	t3_D8

Fault	Detected by (suites)	Missed by (suites)
F3	t1_B_review, t1_C_continuous, t9_B_structured, t9_C_assured	t1_A_testlast, t9_A_lean
F4	t1_C_continuous, t9_B_structured, t9_C_assured	t1_A_testlast, t1_B_review, t9_A_lean
F5	t1_C_continuous	t1_A_testlast, t1_B_review
P1	t10_A_trim, t10_B_auto_synth, t10_C_shiftleft, t5_A_peerdev, t5_B_independent, t5_C_domain, t9_B_structured, t9_C_assured	t9_A_lean
P2	t5_B_independent, t5_C_domain, t9_C_assured	t5_A_peerdev, t9_A_lean, t9_B_structured
P3	t5_C_domain	t5_A_peerdev, t5_B_independent
P4	t5_C_domain	t5_A_peerdev, t5_B_independent
R1	t3_B_spectable, t3_C_reference	t3_A_mental

ANNEXURE I – Catalogue of All Executed Test Cases

Every test case run in Tasks 1, 3, 4, 5, 9 and 10 – 217 test cases in 19 suites – with its result on the correct build and the seeded faults it exposed (from results.jsonl, 1,261 executions). Parameters in brackets are the test inputs.

#	Suite	Test case (inputs)	Correct build	Faults exposed
1	t1_A_testlast	test_adult_fee	Pass	–
2	t1_A_testlast	test_cancel_early	Pass	–
3	t1_A_testlast	test_cancel_late	Pass	–
4	t1_A_testlast	test_child_fee	Pass	F2
5	t1_A_testlast	test_senior_fee	Pass	–
6	t1_B_review	test_adult	Pass	–
7	t1_B_review	test_cancel_1h	Pass	–
8	t1_B_review	test_cancel_3h_no_fee	Pass	F3
9	t1_B_review	test_child	Pass	F2
10	t1_B_review	test_infant	Pass	–
11	t1_B_review	test_senior	Pass	–
12	t1_C_continuous	test_age_boundaries[17-400]	Pass	F2
13	t1_C_continuous	test_age_boundaries[18-500]	Pass	–
14	t1_C_continuous	test_age_boundaries[4-0]	Pass	F4
15	t1_C_continuous	test_age_boundaries[5-400]	Pass	F2
16	t1_C_continuous	test_age_boundaries[59-500]	Pass	–
17	t1_C_continuous	test_age_boundaries[60-300]	Pass	F1
18	t1_C_continuous	test_age_boundaries[61-300]	Pass	–
19	t1_C_continuous	test_cancel_boundaries[0.5-100]	Pass	F5
20	t1_C_continuous	test_cancel_boundaries[1.99-100]	Pass	–
21	t1_C_continuous	test_cancel_boundaries[2.0-0]	Pass	F3
22	t1_C_continuous	test_cancel_boundaries[2.01-0]	Pass	F3
23	t3_A_mental	test_mental[10-500-400]	Pass	–
24	t3_A_mental	test_mental[15-500-400]	Pass	–
25	t3_A_mental	test_mental[25-500-500]	Pass	–
26	t3_A_mental	test_mental[3-500-0]	Pass	–
27	t3_A_mental	test_mental[30-500-500]	Pass	–
28	t3_A_mental	test_mental[45-500-500]	Pass	–
29	t3_A_mental	test_mental[65-500-300]	Pass	–
30	t3_A_mental	test_mental[70-500-300]	Pass	–
31	t3_A_mental	test_mental[8-500-400]	Pass	–
32	t3_A_mental	test_mental[80-500-300]	Pass	–
33	t3_B_spectable	test_table[0-500-0]	Pass	–
34	t3_B_spectable	test_table[10-333-266]	Pass	–
35	t3_B_spectable	test_table[17-500-400]	Pass	–
36	t3_B_spectable	test_table[18-500-500]	Pass	–
37	t3_B_spectable	test_table[4-500-0]	Pass	–
38	t3_B_spectable	test_table[5-500-400]	Pass	–
39	t3_B_spectable	test_table[59-500-500]	Pass	–
40	t3_B_spectable	test_table[60-500-300]	Pass	F1
41	t3_B_spectable	test_table[61-500-300]	Pass	–
42	t3_B_spectable	test_table[65-333-200]	Pass	R1
43	t3_C_reference	test_back_to_back[12-250]	Pass	–
44	t3_C_reference	test_back_to_back[16-333]	Pass	–
45	t3_C_reference	test_back_to_back[2-500]	Pass	–
46	t3_C_reference	test_back_to_back[30-500]	Pass	–
47	t3_C_reference	test_back_to_back[59-500]	Pass	–
48	t3_C_reference	test_back_to_back[6-500]	Pass	–

#	Suite	Test case (inputs)	Correct build	Faults exposed
49	t3_C_reference	test_back_to_back[60-500]	Pass	F1
50	t3_C_reference	test_back_to_back[62-333]	Pass	R1
51	t3_C_reference	test_back_to_back[70-333]	Pass	R1
52	t3_C_reference	test_back_to_back[90-501]	Pass	R1
53	t3_D8	test_tc52_wrong_oracle	Fail (false report)	–
54	t4_B_experience	test_active_limit[0]	Pass	A1
55	t4_B_experience	test_active_limit[1]	Pass	A1
56	t4_B_experience	test_active_limit[2]	Pass	A1
57	t4_B_experience	test_active_limit[3]	Pass	A1
58	t4_B_experience	test_active_limit[4]	Pass	A1
59	t4_B_experience	test_cancel_fee[0.5-100]	Pass	–
60	t4_B_experience	test_cancel_fee[1-100]	Pass	–
61	t4_B_experience	test_cancel_fee[10-0]	Pass	–
62	t4_B_experience	test_cancel_fee[3-0]	Pass	A3
63	t4_B_experience	test_cancel_fee[6-0]	Pass	–
64	t4_B_experience	test_normal_booking[0]	Pass	–
65	t4_B_experience	test_normal_booking[1]	Pass	–
66	t4_B_experience	test_normal_booking[2]	Pass	–
67	t4_B_experience	test_normal_booking[3]	Pass	–
68	t4_B_experience	test_normal_booking[4]	Pass	–
69	t4_B_experience	test_normal_booking[5]	Pass	–
70	t4_B_experience	test_normal_booking[6]	Pass	–
71	t4_B_experience	test_normal_booking[7]	Pass	–
72	t4_B_experience	test_normal_booking[8]	Pass	–
73	t4_B_experience	test_normal_booking[9]	Pass	–
74	t4_B_experience	test_taken_slot_rejected[0]	Pass	–
75	t4_B_experience	test_taken_slot_rejected[1]	Pass	–
76	t4_B_experience	test_taken_slot_rejected[2]	Pass	–
77	t4_B_experience	test_taken_slot_rejected[3]	Pass	–
78	t4_B_experience	test_taken_slot_rejected[4]	Pass	–
79	t4_C_risk_v1	test_booking_core[0]	Pass	–
80	t4_C_risk_v1	test_booking_core[1]	Pass	–
81	t4_C_risk_v1	test_booking_core[2]	Pass	–
82	t4_C_risk_v1	test_booking_core[3]	Pass	–
83	t4_C_risk_v1	test_booking_core[4]	Pass	–
84	t4_C_risk_v1	test_booking_core[5]	Pass	–
85	t4_C_risk_v1	test_booking_core[6]	Pass	–
86	t4_C_risk_v1	test_booking_core[7]	Pass	–
87	t4_C_risk_v1	test_booking_core[8]	Pass	–
88	t4_C_risk_v1	test_cancel_boundary[0.25-100]	Pass	–
89	t4_C_risk_v1	test_cancel_boundary[1.99-100]	Pass	–
90	t4_C_risk_v1	test_cancel_boundary[2.0-0]	Pass	A3
91	t4_C_risk_v1	test_cancel_boundary[2.01-0]	Pass	A3
92	t4_C_risk_v1	test_cancel_boundary[3-0]	Pass	A3
93	t4_C_risk_v1	test_cancel_boundary[4-0]	Pass	–
94	t4_C_risk_v1	test_same_second_race[0]	Pass	A2
95	t4_C_risk_v1	test_same_second_race[1]	Pass	A2
96	t4_C_risk_v1	test_same_second_race[2]	Pass	A2
97	t4_C_risk_v1	test_same_second_race[3]	Pass	A2
98	t4_C_risk_v1	test_same_second_race[4]	Pass	A2
99	t4_C_risk_v1	test_third_active_rejected[11]	Pass	A1
100	t4_C_risk_v1	test_third_active_rejected[12]	Pass	A1
101	t4_C_risk_v1	test_third_active_rejected[13]	Pass	A1

#	Suite	Test case (inputs)	Correct build	Faults exposed
102	t4_C_risk_v1	test_third_active_rejected[14]	Pass	A1
103	t4_C_risk_v1	test_third_active_rejected[15]	Pass	A1
104	t4_C_risk_v2	test_booking_core[0]	Pass	–
105	t4_C_risk_v2	test_booking_core[1]	Pass	–
106	t4_C_risk_v2	test_booking_core[2]	Pass	–
107	t4_C_risk_v2	test_booking_core[3]	Pass	–
108	t4_C_risk_v2	test_booking_core[4]	Pass	–
109	t4_C_risk_v2	test_cancel_boundary[0.25-100]	Pass	–
110	t4_C_risk_v2	test_cancel_boundary[1.99-100]	Pass	–
111	t4_C_risk_v2	test_cancel_boundary[2.0-0]	Pass	A3
112	t4_C_risk_v2	test_cancel_boundary[2.01-0]	Pass	A3
113	t4_C_risk_v2	test_cancel_boundary[3-0]	Pass	A3
114	t4_C_risk_v2	test_cancel_boundary[4-0]	Pass	–
115	t4_C_risk_v2	test_cancelled_slot_is_freed[0]	Pass	A4
116	t4_C_risk_v2	test_cancelled_slot_is_freed[1]	Pass	A4
117	t4_C_risk_v2	test_past_slot_rejected[1]	Pass	A5
118	t4_C_risk_v2	test_past_slot_rejected[24]	Pass	A5
119	t4_C_risk_v2	test_same_second_race[0]	Pass	A2
120	t4_C_risk_v2	test_same_second_race[1]	Pass	A2
121	t4_C_risk_v2	test_same_second_race[2]	Pass	A2
122	t4_C_risk_v2	test_same_second_race[3]	Pass	A2
123	t4_C_risk_v2	test_same_second_race[4]	Pass	A2
124	t4_C_risk_v2	test_third_active_rejected[11]	Pass	A1
125	t4_C_risk_v2	test_third_active_rejected[12]	Pass	A1
126	t4_C_risk_v2	test_third_active_rejected[13]	Pass	A1
127	t4_C_risk_v2	test_third_active_rejected[14]	Pass	A1
128	t4_C_risk_v2	test_third_active_rejected[15]	Pass	A1
129	t5_A_peerdev	test_big_qty	Pass	–
130	t5_A_peerdev	test_day7_rejected_misread	Fail (false report)	–
131	t5_A_peerdev	test_duplicate_same_pharmacist	Pass	–
132	t5_A_peerdev	test_expired_batch	Pass	–
133	t5_A_peerdev	test_qty30_rejected_misread	Fail (false report)	–
134	t5_A_peerdev	test_recent_rx	Pass	–
135	t5_A_peerdev	test_ten_day_old_rx	Pass	P1
136	t5_B_independent	test_batch_expired_yesterday	Pass	–
137	t5_B_independent	test_day7_valid	Pass	–
138	t5_B_independent	test_day8_invalid	Pass	P1
139	t5_B_independent	test_duplicate_same_pharmacist	Pass	–
140	t5_B_independent	test_qty30_ok	Pass	–
141	t5_B_independent	test_qty31_rejected	Pass	P2
142	t5_C_domain	test_batch_expiring_today_blocked	Pass	P3
143	t5_C_domain	test_day7_valid	Pass	–
144	t5_C_domain	test_day8_invalid	Pass	P1
145	t5_C_domain	test_duplicate_other_pharmacist	Pass	P4
146	t5_C_domain	test_qty31_rejected	Pass	P2
147	t9_A_lean	test_adult	Pass	–
148	t9_A_lean	test_cancel_1h	Pass	–
149	t9_A_lean	test_cancel_5h	Pass	–
150	t9_A_lean	test_child	Pass	F2
151	t9_A_lean	test_rx_recent	Pass	–
152	t9_A_lean	test_senior	Pass	–
153	t9_B_structured	test_age[17-400]	Pass	F2

#	Suite	Test case (inputs)	Correct build	Faults exposed
154	t9_B_structured	test_age[18-500]	Pass	–
155	t9_B_structured	test_age[4-0]	Pass	F4
156	t9_B_structured	test_age[5-400]	Pass	F2
157	t9_B_structured	test_age[59-500]	Pass	–
158	t9_B_structured	test_age[60-300]	Pass	F1
159	t9_B_structured	test_cancel[1.99-100]	Pass	–
160	t9_B_structured	test_cancel[2.0-0]	Pass	F3
161	t9_B_structured	test_cancel[3-0]	Pass	F3
162	t9_B_structured	test_qty30	Pass	–
163	t9_B_structured	test_rx_day7	Pass	–
164	t9_B_structured	test_rx_day8	Pass	P1
165	t9_C_assured	test_age[0-0]	Pass	–
166	t9_C_assured	test_age[17-400]	Pass	F2
167	t9_C_assured	test_age[18-500]	Pass	–
168	t9_C_assured	test_age[4-0]	Pass	F4
169	t9_C_assured	test_age[5-400]	Pass	F2
170	t9_C_assured	test_age[59-500]	Pass	–
171	t9_C_assured	test_age[60-300]	Pass	F1
172	t9_C_assured	test_age[61-300]	Pass	–
173	t9_C_assured	test_cancel[0.5-100]	Pass	–
174	t9_C_assured	test_cancel[1.99-100]	Pass	–
175	t9_C_assured	test_cancel[2.0-0]	Pass	F3
176	t9_C_assured	test_cancel[2.01-0]	Pass	F3
177	t9_C_assured	test_qty[30-DISPENSE]	Pass	–
178	t9_C_assured	test_qty[31-REJECT_QTY]	Pass	P2
179	t9_C_assured	test_rx[rx0-DISPENSE]	Pass	–
180	t9_C_assured	test_rx[rx1-REJECT_EXPIRED_RX]	Pass	P1
181	t9_C_assured	test_rx[rx2-REJECT_EXPIRED_RX]	Pass	–
182	t10_A_trim	test_rx_day8	Pass	P1
183	t10_A_trim	test_senior_boundary	Pass	F1
184	t10_A_trim	test_video_smoke	Pass	E1
185	t10_B_auto_synth	test_erx_unsigned_rejected	Pass	E2
186	t10_B_auto_synth	test_health_id_synthetic[Patient000-dob0]	Pass	–
187	t10_B_auto_synth	test_health_id_synthetic[Patient001-dob1]	Pass	–
188	t10_B_auto_synth	test_health_id_synthetic[Patient002-dob2]	Pass	–
189	t10_B_auto_synth	test_health_id_synthetic[Patient003-dob3]	Pass	–
190	t10_B_auto_synth	test_health_id_synthetic[Patient004-dob4]	Pass	–
191	t10_B_auto_synth	test_health_id_synthetic[Patient005-dob5]	Pass	–
192	t10_B_auto_synth	test_health_id_synthetic[Patient006-dob6]	Pass	–
193	t10_B_auto_synth	test_health_id_synthetic[Patient007-dob7]	Pass	–
194	t10_B_auto_synth	test_health_id_synthetic[Patient008-dob8]	Pass	–
195	t10_B_auto_synth	test_health_id_synthetic[Patient009-dob9]	Pass	–
196	t10_B_auto_synth	test_health_id_synthetic[Patient010-dob10]	Pass	–
197	t10_B_auto_synth	test_health_id_synthetic[Patient011-dob11]	Pass	–
198	t10_B_auto_synth	test_health_id_synthetic[Patient012-dob12]	Pass	–
199	t10_B_auto_synth	test_health_id_synthetic[Patient013-dob13]	Pass	–
200	t10_B_auto_synth	test_health_id_synthetic[Patient014-dob14]	Pass	–
201	t10_B_auto_synth	test_health_id_synthetic[Patient015-dob15]	Pass	–
202	t10_B_auto_synth	test_health_id_synthetic[Patient016-dob16]	Pass	–
203	t10_B_auto_synth	test_health_id_synthetic[Patient017-dob17]	Pass	–
204	t10_B_auto_synth	test_health_id_synthetic[Patient018-dob18]	Pass	–
205	t10_B_auto_synth	test_health_id_synthetic[Patient019-dob19]	Pass	–
206	t10_B_auto_synth	test_rx_day8	Pass	P1
207	t10_B_auto_synth	test_senior_boundary	Pass	F1

#	Suite	Test case (inputs)	Correct build	Faults exposed
208	t10_B_auto_synth	test_video_timeout[29-True]	Pass	E1
209	t10_B_auto_synth	test_video_timeout[30-False]	Pass	–
210	t10_B_auto_synth	test_video_timeout[5-True]	Pass	E1
211	t10_C_shiftleft	test_audit_entry_written	Pass	E4
212	t10_C_shiftleft	test_contract_curly_apostrophe	Pass	E3
213	t10_C_shiftleft	test_contract_dob_mismatch_blocked	Pass	–
214	t10_C_shiftleft	test_erx_unsigned_rejected	Pass	E2
215	t10_C_shiftleft	test_rx_day8	Pass	P1
216	t10_C_shiftleft	test_senior_boundary	Pass	F1
217	t10_C_shiftleft	test_video_timeout_unit_minutes	Pass	E1

ANNEXURE J – Execution Log of the Correct Build (pytest -v)

Verbatim output of `pytest -v tests/` on the correct build (no mutant), produced while preparing this answer.

```
============================= test session starts ==============================
test_t10_A_trim.py::test_senior_boundary PASSED                     [  0%]
test_t10_A_trim.py::test_rx_day8 PASSED                             [  0%]
test_t10_A_trim.py::test_video_smoke PASSED                         [  1%]
test_t10_B_auto_synth.py::test_senior_boundary PASSED               [  1%]
test_t10_B_auto_synth.py::test_rx_day8 PASSED                       [  2%]
test_t10_B_auto_synth.py::test_video_timeout[5-True] PASSED         [  2%]
test_t10_B_auto_synth.py::test_video_timeout[29-True] PASSED        [  3%]
test_t10_B_auto_synth.py::test_video_timeout[30-False] PASSED       [  3%]
test_t10_B_auto_synth.py::test_erx_unsigned_rejected PASSED         [  4%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient000-dob0] PASSED [  4%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient001-dob1] PASSED [  5%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient002-dob2] PASSED [  5%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient003-dob3] PASSED [  5%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient004-dob4] PASSED [  6%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient005-dob5] PASSED [  6%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient006-dob6] PASSED [  7%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient007-dob7] PASSED [  7%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient008-dob8] PASSED [  8%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient009-dob9] PASSED [  8%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient010-dob10] PASSED [  9%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient011-dob11] PASSED [  9%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient012-dob12] PASSED [ 10%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient013-dob13] PASSED [ 10%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient014-dob14] PASSED [ 11%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient015-dob15] PASSED [ 11%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient016-dob16] PASSED [ 11%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient017-dob17] PASSED [ 12%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient018-dob18] PASSED [ 12%]
test_t10_B_auto_synth.py::test_health_id_synthetic[Patient019-dob19] PASSED [ 13%]
test_t10_C_shiftleft.py::test_senior_boundary PASSED                [ 13%]
test_t10_C_shiftleft.py::test_rx_day8 PASSED                        [ 14%]
test_t10_C_shiftleft.py::test_video_timeout_unit_minutes PASSED     [ 14%]
test_t10_C_shiftleft.py::test_erx_unsigned_rejected PASSED          [ 15%]
test_t10_C_shiftleft.py::test_contract_curly_apostrophe PASSED      [ 15%]
test_t10_C_shiftleft.py::test_contract_dob_mismatch_blocked PASSED [ 16%]
test_t10_C_shiftleft.py::test_audit_entry_written PASSED            [ 16%]
test_t1_A_testlast.py::test_adult_fee PASSED                        [ 17%]
test_t1_A_testlast.py::test_senior_fee PASSED                       [ 17%]
test_t1_A_testlast.py::test_child_fee PASSED                        [ 17%]
test_t1_A_testlast.py::test_cancel_late PASSED                      [ 18%]
test_t1_A_testlast.py::test_cancel_early PASSED                     [ 18%]
test_t1_B_review.py::test_adult PASSED                              [ 19%]
test_t1_B_review.py::test_senior PASSED                             [ 19%]
test_t1_B_review.py::test_child PASSED                              [ 20%]
test_t1_B_review.py::test_infant PASSED                             [ 20%]
test_t1_B_review.py::test_cancel_1h PASSED                          [ 21%]
test_t1_B_review.py::test_cancel_3h_no_fee PASSED                   [ 21%]
test_t1_C_continuous.py::test_age_boundaries[4-0] PASSED            [ 22%]
test_t1_C_continuous.py::test_age_boundaries[5-400] PASSED          [ 22%]
test_t1_C_continuous.py::test_age_boundaries[17-400] PASSED         [ 23%]
test_t1_C_continuous.py::test_age_boundaries[18-500] PASSED         [ 23%]
test_t1_C_continuous.py::test_age_boundaries[59-500] PASSED         [ 23%]
test_t1_C_continuous.py::test_age_boundaries[60-300] PASSED         [ 24%]
test_t1_C_continuous.py::test_age_boundaries[61-300] PASSED         [ 24%]
test_t1_C_continuous.py::test_cancel_boundaries[0.5-100] PASSED     [ 25%]
test_t1_C_continuous.py::test_cancel_boundaries[1.99-100] PASSED    [ 25%]
test_t1_C_continuous.py::test_cancel_boundaries[2.0-0] PASSED       [ 26%]
test_t1_C_continuous.py::test_cancel_boundaries[2.01-0] PASSED      [ 26%]
test_t3_A_mental.py::test_mental[10-500-400] PASSED                 [ 27%]
test_t3_A_mental.py::test_mental[30-500-500] PASSED                 [ 27%]
test_t3_A_mental.py::test_mental[65-500-300] PASSED                 [ 28%]
test_t3_A_mental.py::test_mental[3-500-0] PASSED                    [ 28%]
test_t3_A_mental.py::test_mental[45-500-500] PASSED                 [ 29%]
test_t3_A_mental.py::test_mental[8-500-400] PASSED                  [ 29%]
test_t3_A_mental.py::test_mental[70-500-300] PASSED                 [ 29%]
test_t3_A_mental.py::test_mental[25-500-500] PASSED                 [ 30%]
```

```
test_t3_A_mental.py::test_mental[15-500-400] PASSED              [ 30%]
test_t3_A_mental.py::test_mental[80-500-300] PASSED              [ 31%]
test_t3_B_spectable.py::test_table[0-500-0] PASSED               [ 31%]
test_t3_B_spectable.py::test_table[4-500-0] PASSED               [ 32%]
test_t3_B_spectable.py::test_table[5-500-400] PASSED             [ 32%]
test_t3_B_spectable.py::test_table[17-500-400] PASSED            [ 33%]
test_t3_B_spectable.py::test_table[18-500-500] PASSED            [ 33%]
test_t3_B_spectable.py::test_table[59-500-500] PASSED            [ 34%]
test_t3_B_spectable.py::test_table[60-500-300] PASSED            [ 34%]
test_t3_B_spectable.py::test_table[61-500-300] PASSED            [ 35%]
test_t3_B_spectable.py::test_table[10-333-266] PASSED            [ 35%]
test_t3_B_spectable.py::test_table[65-333-200] PASSED            [ 35%]
test_t3_C_reference.py::test_back_to_back[2-500] PASSED          [ 36%]
test_t3_C_reference.py::test_back_to_back[6-500] PASSED          [ 36%]
test_t3_C_reference.py::test_back_to_back[16-333] PASSED         [ 37%]
test_t3_C_reference.py::test_back_to_back[30-500] PASSED         [ 37%]
test_t3_C_reference.py::test_back_to_back[59-500] PASSED         [ 38%]
test_t3_C_reference.py::test_back_to_back[60-500] PASSED         [ 38%]
test_t3_C_reference.py::test_back_to_back[62-333] PASSED         [ 39%]
test_t3_C_reference.py::test_back_to_back[90-501] PASSED         [ 39%]
test_t3_C_reference.py::test_back_to_back[12-250] PASSED         [ 40%]
test_t3_C_reference.py::test_back_to_back[70-333] PASSED         [ 40%]
test_t3_D8.py::test_tc52_wrong_oracle FAILED                     [ 41%]
test_t4_B_experience.py::test_normal_booking[0] PASSED           [ 41%]
test_t4_B_experience.py::test_normal_booking[1] PASSED           [ 41%]
test_t4_B_experience.py::test_normal_booking[2] PASSED           [ 42%]
test_t4_B_experience.py::test_normal_booking[3] PASSED           [ 42%]
test_t4_B_experience.py::test_normal_booking[4] PASSED           [ 43%]
test_t4_B_experience.py::test_normal_booking[5] PASSED           [ 43%]
test_t4_B_experience.py::test_normal_booking[6] PASSED           [ 44%]
test_t4_B_experience.py::test_normal_booking[7] PASSED           [ 44%]
test_t4_B_experience.py::test_normal_booking[8] PASSED           [ 45%]
test_t4_B_experience.py::test_normal_booking[9] PASSED           [ 45%]
test_t4_B_experience.py::test_taken_slot_rejected[0] PASSED      [ 46%]
test_t4_B_experience.py::test_taken_slot_rejected[1] PASSED      [ 46%]
test_t4_B_experience.py::test_taken_slot_rejected[2] PASSED      [ 47%]
test_t4_B_experience.py::test_taken_slot_rejected[3] PASSED      [ 47%]
test_t4_B_experience.py::test_taken_slot_rejected[4] PASSED      [ 47%]
test_t4_B_experience.py::test_cancel_fee[6-0] PASSED             [ 48%]
test_t4_B_experience.py::test_cancel_fee[1-100] PASSED           [ 48%]
test_t4_B_experience.py::test_cancel_fee[10-0] PASSED            [ 49%]
test_t4_B_experience.py::test_cancel_fee[0.5-100] PASSED         [ 49%]
test_t4_B_experience.py::test_cancel_fee[3-0] PASSED             [ 50%]
test_t4_B_experience.py::test_active_limit[0] PASSED             [ 50%]
test_t4_B_experience.py::test_active_limit[1] PASSED             [ 51%]
test_t4_B_experience.py::test_active_limit[2] PASSED             [ 51%]
test_t4_B_experience.py::test_active_limit[3] PASSED             [ 52%]
test_t4_B_experience.py::test_active_limit[4] PASSED             [ 52%]
test_t4_C_risk_v1.py::test_same_second_race[0] PASSED            [ 52%]
test_t4_C_risk_v1.py::test_same_second_race[1] PASSED            [ 53%]
test_t4_C_risk_v1.py::test_same_second_race[2] PASSED            [ 53%]
test_t4_C_risk_v1.py::test_same_second_race[3] PASSED            [ 54%]
test_t4_C_risk_v1.py::test_same_second_race[4] PASSED            [ 54%]
test_t4_C_risk_v1.py::test_third_active_rejected[11] PASSED      [ 55%]
test_t4_C_risk_v1.py::test_third_active_rejected[12] PASSED      [ 55%]
test_t4_C_risk_v1.py::test_third_active_rejected[13] PASSED      [ 56%]
test_t4_C_risk_v1.py::test_third_active_rejected[14] PASSED      [ 56%]
test_t4_C_risk_v1.py::test_third_active_rejected[15] PASSED      [ 57%]
test_t4_C_risk_v1.py::test_cancel_boundary[1.99-100] PASSED      [ 57%]
test_t4_C_risk_v1.py::test_cancel_boundary[2.0-0] PASSED         [ 58%]
test_t4_C_risk_v1.py::test_cancel_boundary[2.01-0] PASSED        [ 58%]
test_t4_C_risk_v1.py::test_cancel_boundary[0.25-100] PASSED      [ 58%]
test_t4_C_risk_v1.py::test_cancel_boundary[3-0] PASSED           [ 59%]
test_t4_C_risk_v1.py::test_cancel_boundary[4-0] PASSED           [ 59%]
test_t4_C_risk_v1.py::test_booking_core[0] PASSED                [ 60%]
test_t4_C_risk_v1.py::test_booking_core[1] PASSED                [ 60%]
test_t4_C_risk_v1.py::test_booking_core[2] PASSED                [ 61%]
test_t4_C_risk_v1.py::test_booking_core[3] PASSED                [ 61%]
test_t4_C_risk_v1.py::test_booking_core[4] PASSED                [ 62%]
test_t4_C_risk_v1.py::test_booking_core[5] PASSED                [ 62%]
test_t4_C_risk_v1.py::test_booking_core[6] PASSED                [ 63%]
test_t4_C_risk_v1.py::test_booking_core[7] PASSED                [ 63%]
test_t4_C_risk_v1.py::test_booking_core[8] PASSED                [ 64%]
```

```
test_t4_C_risk_v2.py::test_same_second_race[0] PASSED             [ 64%]
test_t4_C_risk_v2.py::test_same_second_race[1] PASSED             [ 64%]
test_t4_C_risk_v2.py::test_same_second_race[2] PASSED             [ 65%]
test_t4_C_risk_v2.py::test_same_second_race[3] PASSED             [ 65%]
test_t4_C_risk_v2.py::test_same_second_race[4] PASSED             [ 66%]
test_t4_C_risk_v2.py::test_third_active_rejected[11] PASSED       [ 66%]
test_t4_C_risk_v2.py::test_third_active_rejected[12] PASSED       [ 67%]
test_t4_C_risk_v2.py::test_third_active_rejected[13] PASSED       [ 67%]
test_t4_C_risk_v2.py::test_third_active_rejected[14] PASSED       [ 68%]
test_t4_C_risk_v2.py::test_third_active_rejected[15] PASSED       [ 68%]
test_t4_C_risk_v2.py::test_cancel_boundary[1.99-100] PASSED       [ 69%]
test_t4_C_risk_v2.py::test_cancel_boundary[2.0-0] PASSED          [ 69%]
test_t4_C_risk_v2.py::test_cancel_boundary[2.01-0] PASSED         [ 70%]
test_t4_C_risk_v2.py::test_cancel_boundary[0.25-100] PASSED       [ 70%]
test_t4_C_risk_v2.py::test_cancel_boundary[3-0] PASSED            [ 70%]
test_t4_C_risk_v2.py::test_cancel_boundary[4-0] PASSED            [ 71%]
test_t4_C_risk_v2.py::test_booking_core[0] PASSED                 [ 71%]
test_t4_C_risk_v2.py::test_booking_core[1] PASSED                 [ 72%]
test_t4_C_risk_v2.py::test_booking_core[2] PASSED                 [ 72%]
test_t4_C_risk_v2.py::test_booking_core[3] PASSED                 [ 73%]
test_t4_C_risk_v2.py::test_booking_core[4] PASSED                 [ 73%]
test_t4_C_risk_v2.py::test_cancelled_slot_is_freed[0] PASSED      [ 74%]
test_t4_C_risk_v2.py::test_cancelled_slot_is_freed[1] PASSED      [ 74%]
test_t4_C_risk_v2.py::test_past_slot_rejected[1] PASSED           [ 75%]
test_t4_C_risk_v2.py::test_past_slot_rejected[24] PASSED          [ 75%]
test_t5_A_peerdev.py::test_recent_rx PASSED                       [ 76%]
test_t5_A_peerdev.py::test_ten_day_old_rx PASSED                  [ 76%]
test_t5_A_peerdev.py::test_big_qty PASSED                         [ 76%]
test_t5_A_peerdev.py::test_expired_batch PASSED                   [ 77%]
test_t5_A_peerdev.py::test_duplicate_same_pharmacist PASSED       [ 77%]
test_t5_A_peerdev.py::test_day7_rejected_misread FAILED           [ 78%]
test_t5_A_peerdev.py::test_qty30_rejected_misread FAILED          [ 78%]
test_t5_B_independent.py::test_day7_valid PASSED                  [ 79%]
test_t5_B_independent.py::test_day8_invalid PASSED                [ 79%]
test_t5_B_independent.py::test_qty30_ok PASSED                    [ 80%]
test_t5_B_independent.py::test_qty31_rejected PASSED              [ 80%]
test_t5_B_independent.py::test_batch_expired_yesterday PASSED     [ 81%]
test_t5_B_independent.py::test_duplicate_same_pharmacist PASSED   [ 81%]
test_t5_C_domain.py::test_day7_valid PASSED                       [ 82%]
test_t5_C_domain.py::test_day8_invalid PASSED                     [ 82%]
test_t5_C_domain.py::test_qty31_rejected PASSED                   [ 82%]
test_t5_C_domain.py::test_batch_expiring_today_blocked PASSED     [ 83%]
test_t5_C_domain.py::test_duplicate_other_pharmacist PASSED       [ 83%]
test_t9_A_lean.py::test_adult PASSED                              [ 84%]
test_t9_A_lean.py::test_senior PASSED                             [ 84%]
test_t9_A_lean.py::test_child PASSED                              [ 85%]
test_t9_A_lean.py::test_cancel_1h PASSED                          [ 85%]
test_t9_A_lean.py::test_cancel_5h PASSED                          [ 86%]
test_t9_A_lean.py::test_rx_recent PASSED                          [ 86%]
test_t9_B_structured.py::test_age[4-0] PASSED                     [ 87%]
test_t9_B_structured.py::test_age[5-400] PASSED                   [ 87%]
test_t9_B_structured.py::test_age[17-400] PASSED                  [ 88%]
test_t9_B_structured.py::test_age[18-500] PASSED                  [ 88%]
test_t9_B_structured.py::test_age[59-500] PASSED                  [ 88%]
test_t9_B_structured.py::test_age[60-300] PASSED                  [ 89%]
test_t9_B_structured.py::test_cancel[1.99-100] PASSED             [ 89%]
test_t9_B_structured.py::test_cancel[2.0-0] PASSED                [ 90%]
test_t9_B_structured.py::test_cancel[3-0] PASSED                  [ 90%]
test_t9_B_structured.py::test_rx_day7 PASSED                      [ 91%]
test_t9_B_structured.py::test_rx_day8 PASSED                      [ 91%]
test_t9_B_structured.py::test_qty30 PASSED                        [ 92%]
test_t9_C_assured.py::test_age[0-0] PASSED                        [ 92%]
test_t9_C_assured.py::test_age[4-0] PASSED                        [ 93%]
test_t9_C_assured.py::test_age[5-400] PASSED                      [ 93%]
test_t9_C_assured.py::test_age[17-400] PASSED                     [ 94%]
test_t9_C_assured.py::test_age[18-500] PASSED                     [ 94%]
test_t9_C_assured.py::test_age[59-500] PASSED                     [ 94%]
test_t9_C_assured.py::test_age[60-300] PASSED                     [ 95%]
test_t9_C_assured.py::test_age[61-300] PASSED                     [ 95%]
test_t9_C_assured.py::test_cancel[0.5-100] PASSED                 [ 96%]
test_t9_C_assured.py::test_cancel[1.99-100] PASSED                [ 96%]
test_t9_C_assured.py::test_cancel[2.0-0] PASSED                   [ 97%]
test_t9_C_assured.py::test_cancel[2.01-0] PASSED                  [ 97%]
```

```
test_t9_C_assured.py::test_rx[rx0-DISPENSE] PASSED                [ 98%]
test_t9_C_assured.py::test_rx[rx1-REJECT_EXPIRED_RX] PASSED       [ 98%]
test_t9_C_assured.py::test_rx[rx2-REJECT_EXPIRED_RX] PASSED       [ 99%]
test_t9_C_assured.py::test_qty[30-DISPENSE] PASSED                [ 99%]
test_t9_C_assured.py::test_qty[31-REJECT_QTY] PASSED              [100%]
================================== FAILURES ===================================
=========================== short test summary info ============================
FAILED tests/test_t3_D8.py::test_tc52_wrong_oracle - assert 400 == 450
FAILED tests/test_t5_A_peerdev.py::test_day7_rejected_misread - AssertionErro...
FAILED tests/test_t5_A_peerdev.py::test_qty30_rejected_misread - AssertionErr...
======================== 3 failed, 214 passed in 0.29s =========================
```

ANNEXURE K – Master Test Plan for the Next MediQueue Release (IEEE 829 structure)

This plan puts the decisions of Tasks 1–9 (configuration C) into the document a QA lead would circulate before the release starts.

1. Test plan identifier and introduction

MQ-MTP-3.5. Release 3.5 of MediQueue (Registration, Appointment, Billing, Pharmacy, Lab Reports, Reminders; 47 KLOC). Purpose: show, with measured evidence, that the release meets the business rules, protects patient safety and revenue, and does not repeat the ten defects D1–D10.

2. Test items

Angular front end build 3.5.x, Node.js services, PostgreSQL 15 schema, SMS gateway adapter, payment gateway adapter, the defect log, the specification tables for concession, cancellation, active appointments and prescription validity.

3. Features to be tested

Concession by age with half-up rounding; late-cancellation fee; maximum two active appointments and one patient per slot (including same-second confirmations); payment idempotency on refresh; prescription validity, quantity, batch expiry and duplicate dispensing; lab-report dates and response time at 800 users; reminder SMS after reschedule; registration of names with apostrophes.

4. Features not to be tested

Hospital HR and inventory interfaces (unchanged since 3.3, covered by smoke tests only); browser versions older than two years.

5. Approach

Requirement review with example tables (10 h); risk-based boundary design from the specification tables (18 h); automated regression and static analysis on every build (20 h); system and domain tests with the pharmacist and lab technician (30 h); load test of the morning peak (800 users, 2 s target); defect logging in JIRA with mandatory fields; retest and regression of fixes (10 h); metrics (DRE, defects per KLOC, Pareto) after release (3 h).

6. Item pass/fail criteria

A test passes when the actual result equals the value in the specification table (to the rupee, or to the second for times) or the stated behaviour (rejection, single bill, single booking). A feature passes when all its critical and major tests pass.

7. Suspension and resumption criteria

Suspend system testing if the smoke suite fails, if more than 3 critical defects are open, or if the test environment differs from the agreed configuration. Resume after the blocking defects are fixed and the smoke suite passes.

8. Test deliverables

This plan; specification tables; test cases in the repository; CI run reports; JIRA defect reports with links to tests and requirements; load-test report; release metrics; sign-off record of the QA lead and the medical officer.

9. Testing tasks

Review requirements and update example tables → design and automate boundary tests → configure quality gate (all tests pass, branch coverage ≥ 80 % on Billing, Appointment, Pharmacy) → system and domain tests → load test → defect triage → retest → release decision → metrics.

10. Environmental needs

Test environment with anonymised data of 9,000 patients; staging with production-like database size; load generator for 800 virtual users; SMS and payment sandboxes; CI server.

11. Responsibilities

QA lead: plan, gate, release decision. Testers T1–T3: modules as allocated in Task 5. Pharmacist and lab technician: domain examples and Pharmacy/Lab acceptance. Developers: unit tests, fixes. Medical officer: co-signs Pharmacy and Lab Reports.

12. Staffing and training

3 testers, 1 QA lead; half-day training on boundary-value design and on the JIRA workflow for developers and reception staff who report defects.

13. Schedule (3-week release)

Week 1: reviews, test design, pipeline update. Week 2: automated regression on every build, system and domain tests, load test. Week 3: retest, regression, release decision on day 15.

14. Risks and contingencies

Requirement changes late in the cycle → change goes through review and the example tables; pharmacist unavailable → pre-agreed examples from Task 5 reused; load environment late → run the load test on the first stable build of week 2 and repeat in week 3.

15. Approvals

QA lead, development lead, medical officer, Medical Superintendent.

ANNEXURE L – Analysis of the 164-Record Defect Log

Cross-tabulations computed directly from defect_log_last_release.csv. They support the module, severity and cause statements made in Tasks 4, 6 and 8.

L1. Module × severity

Module	Critical	Major	Minor	Trivial	Total
Billing	6	15	29	8	58
Appointment	5	18	14	11	48
Pharmacy	5	11	4	2	22
Lab Reports	3	1	8	4	16
Registration	2	3	5	1	11
Reminders	1	2	4	2	9
Total	**22**	**50**	**64**	**28**	**164**

L2. Module × detecting activity

Module	Requirement review	Code review	Unit test	System test	Production
Billing	8	8	9	25	8
Appointment	2	4	17	16	9
Pharmacy	1	5	2	5	9
Lab Reports	3	4	4	3	2
Registration	2	1	2	6	0
Reminders	2	0	1	5	1

L3. Cause category × severity

Cause	Critical	Major	Minor	Trivial	Total
Ambiguous/incomplete requirement	10	14	22	11	57
Boundary/off-by-one logic	1	11	12	4	28
Wrong field/data mapping	2	7	9	4	22
Concurrency/idempotency	5	7	5	4	21
Input validation	1	4	3	3	11
Test-case error	1	5	3	1	10
Configuration/environment	1	2	6	1	10
Performance	1	0	4	0	5

L4. Escaped defects (detected in production) by module and cause

Module	Ambiguous/incomplete requirement	Boundary/off-by-one logic	Wrong field/data mapping	Concurrency/idempotency	Input validation	Test-case error	Configuration/environment	Performance
Billing	4	0	0	1	0	0	2	1
Appointment	2	2	1	3	0	1	0	0
Pharmacy	5	0	1	2	1	0	0	0
Lab Reports	1	0	1	0	0	0	0	0
Registration	0	0	0	0	0	0	0	0
Reminders	1	0	0	0	0	0	0	0

Of the 29 escaped defects, 9 are critical. Ambiguous or incomplete requirements cause 13 escapes – the largest single cause – which is why requirement review with example tables is the first decision of the practice. Pharmacy has the most escapes for its size (Task 8), and every Pharmacy escape is a candidate for the pharmacist's acceptance examples.

L5. Severity by origin artefact

Origin	Critical	Major	Minor	Trivial	Critical share
Requirement	6	10	19	11	13%
Design	7	7	7	1	32%
Code	7	25	26	12	10%
Test case	1	5	3	1	10%
Configuration	1	2	6	0	11%
Data	0	1	3	3	0%

ANNEXURE M – Defect Repository Configuration (Bugzilla and JIRA)

Task 8 asks for screenshots from the real tools. The configuration below is what was set up in both tools so that the screenshots can be reproduced exactly.

M1. Field definitions

Field	Type	Values / rule	Compulsory at
Summary	Text (≤ 120 chars)	Symptom + where, e.g. "Age 60 charged ₹500 not ₹300"	New
Module	Select	Registration, Appointment, Billing, Pharmacy, Lab Reports, Reminders	New
Build / environment	Text	Build number, browser, database version	New
Steps to reproduce	Text	Numbered steps with data	New
Expected / actual	Text	Expected from the specification table; actual observed	New
Severity	Select	Critical, Major, Minor, Trivial (effect on patient / revenue)	Open
Priority	Select	P1–P4 (order of fixing)	Open
Class (scheme B)	Select	Logic, Data, Interface, Concurrency, Validation, Performance, Requirement gap, Test oracle, Data mapping	Open
Origin artefact	Select	Requirement, Design, Code, Test case, Configuration, Data	Fixed
Safety category	Select	Safety-critical, Data privacy, Financial, Availability, Usability, Cosmetic	Open
Linked test case	Link	Test ID in the repository (JIRA issue link "is tested by")	Retest
Linked requirement	Link	Requirement ID (JIRA issue link "relates to")	Open
Resolution	Select	Fixed, Duplicate, Won't fix (needs reason), Not a defect	Fixed / Rejected

M2. Workflow transitions (JIRA)

From	Transition	To	Who	Condition / validator
New	Triage	Open	QA lead	Severity, priority, class, safety category set
New	Reject	Rejected	QA lead	Reason comment required
Open	Start fix	In progress	Developer	Assignee set
In progress	Resolve	Fixed	Developer	Origin artefact and commit link set
Fixed	Deploy to test	Retest	CI	Build number recorded
Retest	Pass	Verified	Tester	Linked test case passed
Retest	Fail	Reopened	Tester	Failing evidence attached
Reopened	Start fix	In progress	Developer	–
Verified	Release	Closed	QA lead	Release version set

M3. Saved filters and dashboard gadgets (JQL)

```
-- open defects by module and severity (dashboard: two-dimensional filter statistics)
project = MQ AND issuetype = Bug AND status not in (Closed, Rejected)
-- escaped defects of the last release
project = MQ AND issuetype = Bug AND "Detected in" = Production AND fixVersion = 3.4
-- safety-critical defects still open (release blocker)
project = MQ AND "Safety category" = "Safety-critical" AND status not in (Verified, Closed)
-- reopened in the last 30 days (reopen rate)
project = MQ AND status CHANGED TO Reopened AFTER -30d
-- mean time to fix per module (control chart gadget on transition Open -> Fixed)
```

```
project = MQ AND issuetype = Bug AND resolution = Fixed AND resolved >= -90d
```

M4. Bugzilla equivalents

JIRA element	Bugzilla setting
Module field	Component (one per module)
Class, origin, safety category	Custom fields (drop-down)
Workflow New → Open → Fixed → Retest → Verified → Closed	Status workflow edited: UNCONFIRMED → CONFIRMED → RESOLVED/FIXED → (custom) RETEST → VERIFIED
Mandatory fields per transition	Not supported per transition; set required fields on entry only
Dashboard by module × severity	Reports → Tabular report: Component × Severity
Link to test case	URL field or See Also with the test ID

ANNEXURE N – Test-Case Specifications of the Chosen Suites

Specification form (identifier, objective, precondition, input, expected result, requirement) for the test cases of the techniques chosen in Tasks 1, 3, 5 and 9. Expected results are taken from the specification tables, not from the code.

ID	Area	Objective	Precondition	Input	Expected	Req.
TS-01	Billing concession	Concession for age 4	Patient registered; base fee ₹500	age = 4	₹0	REQ-BIL-01
TS-02	Billing concession	Concession for age 5	Patient registered; base fee ₹500	age = 5	₹400	REQ-BIL-01
TS-03	Billing concession	Concession for age 17	Patient registered; base fee ₹500	age = 17	₹400	REQ-BIL-01
TS-04	Billing concession	Concession for age 18	Patient registered; base fee ₹500	age = 18	₹500	REQ-BIL-01
TS-05	Billing concession	Concession for age 59	Patient registered; base fee ₹500	age = 59	₹500	REQ-BIL-01
TS-06	Billing concession	Concession for age 60	Patient registered; base fee ₹500	age = 60	₹300	REQ-BIL-01
TS-07	Billing concession	Concession for age 61	Patient registered; base fee ₹500	age = 61	₹300	REQ-BIL-01
TS-08	Late cancellation	Fee when cancelled 0.5 h before	Booked appointment	hours_before = 0.5	₹100	REQ-APT-01
TS-09	Late cancellation	Fee when cancelled 1.99 h before	Booked appointment	hours_before = 1.99	₹100	REQ-APT-01
TS-10	Late cancellation	Fee when cancelled 2.0 h before	Booked appointment	hours_before = 2.0	₹0	REQ-APT-01
TS-11	Late cancellation	Fee when cancelled 2.01 h before	Booked appointment	hours_before = 2.01	₹0	REQ-APT-01
TS-12	Oracle table	Rounding/band check age 0, base ₹500	Decision table row	age = 0, base = 500	₹0	REQ-BIL-01/02
TS-13	Oracle table	Rounding/band check age 60, base ₹500	Decision table row	age = 60, base = 500	₹300	REQ-BIL-01/02
TS-14	Oracle table	Rounding/band check age 10, base ₹333	Decision table row	age = 10, base = 333	₹266	REQ-BIL-01/02
TS-15	Oracle table	Rounding/band check age 65, base ₹333	Decision table row	age = 65, base = 333	₹200	REQ-BIL-01/02
TS-16	Pharmacy	Rx exactly 7 days old	Valid prescription; stock available	Rx 05-03-2026, today 12-03-2026	Dispense	REQ-PHA
TS-17	Pharmacy	Rx 8 days old	Valid prescription; stock available	Rx 04-03-2026	Reject – expired	REQ-PHA
TS-18	Pharmacy	Quantity 31	Valid prescription; stock available	qty = 31	Reject – quantity	REQ-PHA
TS-19	Pharmacy	Batch expiring today	Valid prescription; stock available	expiry = 12-03-2026	Reject – batch	REQ-PHA
TS-20	Pharmacy	Second pharmacist repeats dispense	Valid prescription; stock available	same Rx, ph1 then ph2	Reject – duplicate	REQ-PHA
TS-21	Appointment	Same-second double booking	Doctor schedule loaded	PA and PB confirm D1 10:15 in the same second	Second rejected	REQ-APT

ID	Area	Objective	Precondition	Input	Expected	Req.
TS-22	Appointment	Third active appointment	Doctor schedule loaded	patient holds 2 active appointments	Rejected	REQ-APT
TS-23	Appointment	Cancelled slot freed	Doctor schedule loaded	cancel 12:00 then rebook by another patient	Booked	REQ-APT
TS-24	Appointment	Past slot	Doctor schedule loaded	slot 1 h before now	Rejected	REQ-APT
TS-25	Tele-consultation (Task 10)	Video idle 29 min	Extended build	idle_minutes = 29	Session active	REQ-TEL/HID/AUD
TS-26	Tele-consultation (Task 10)	Video idle 30 min	Extended build	idle_minutes = 30	Session ended	REQ-TEL/HID/AUD
TS-27	Tele-consultation (Task 10)	Unsigned e-Rx	Extended build	signed = False	Rejected	REQ-TEL/HID/AUD
TS-28	Tele-consultation (Task 10)	Health-ID curly apostrophe	Extended build	local “Anita D'Souza”, ID “ANITA D’SOUZA”, same DOB	Linked	REQ-TEL/HID/AUD
TS-29	Tele-consultation (Task 10)	Health-ID DOB mismatch	Extended build	DOB differs by one day	Not linked	REQ-TEL/HID/AUD
TS-30	Tele-consultation (Task 10)	Audit entry written	Extended build	link attempt by qa1	Audit row with user qa1	REQ-TEL/HID/AUD

ANNEXURE O – Procedure Followed for Each Task

The steps below are what was actually done to produce each task's results, in order, so that the work can be repeated and explained in the viva. All commands run from the folder that contains mediqueue.py.

Task 1

- Step 1: Wrote the concession and cancellation rules in mediqueue.py with switchable faults F1–F5.
- Step 2: Wrote the requirement text with AMB-1 (2-hour limit) and AMB-2 (age on which date).
- Step 3: Wrote three suites: test_t1_A_testlast.py (5 cases), test_t1_B_review.py (6 cases after review), test_t1_C_continuous.py (11 boundary cases).
- Step 4: Ran python run_all.py: every suite on the correct build and on each single-fault build (MUTANT=F1 … F5).
- Step 5: Read the fault counts with python analyze.py and computed detection %, issues per ph and escape cost.
- Step 6: Asked an LLM for a test plan and checked it against the business rules (procedure in section 6).

Task 2

- Step 1: Generated the 164-record defect log with python make_log.py (seed 42).
- Step 2: Computed DRE = 135 ÷ 164 and the 15-defect cost table.
- Step 3: Split the 29 escapes by origin and estimated what each process catches from the Task 1 evidence.
- Step 4: Drew the activity timeline (Figure 3) and the DRE chart (Figure 4).

Task 3

- Step 1: Seeded F1 and R1 in concession_fee.
- Step 2: Wrote 10 tests per oracle strategy (mental values, decision table, reference implementation).
- Step 3: Ran the suites on the correct build and on F1 and R1; recorded the failing test names.
- Step 4: Added test_t3_D8.py with the wrong expected value ₹450 and ran it on the correct and the F2 builds.

Task 4

- Step 1: Seeded A1–A3 (release N) and A4–A5 (release N+1) in the booking code.
- Step 2: Wrote the experience-driven set (25), the risk-based release-N set (25) and the refreshed set (25).
- Step 3: Ran all three on every mutant; counted faults per release.
- Step 4: Computed module shares and defects per KLOC from the defect log; drew the Pareto chart.

Task 5

- Step 1: Seeded P1–P4 in can_dispense.

- Step 2: Wrote three passes: peer developer (7 tests), independent tester (6), tester + pharmacist (5).
- Step 3: Ran them on the correct build (to find false reports) and on each fault.
- Step 4: Wrote the two defect reports for P1 and the allocation table.

Task 6

- Step 1: Tagged D1–D10 and the 164 log records with origin and detecting activity.
- Step 2: Built the origin × activity matrix, computed containment effectiveness.
- Step 3: Drew fault trees for D2 and D4 and the Pareto chart of causes.

Task 7

- Step 1: Classified D1–D10 under schemes A, B and C with reasons.
- Step 2: Assigned severity and priority; prepared the agreement sheet for two classmates.

Task 8

- Step 1: Configured Bugzilla and JIRA as in Annexure M and logged D1–D10.
- Step 2: Moved D2 and D5 through the workflow; recorded timestamps from the history tab.
- Step 3: Normalised defects and escapes per KLOC.

Task 9

- Step 1: Seeded F1, F2, F3, F4, P1, P2 in one build.
- Step 2: Ran the lean (6), structured (12) and assured (17) suites on each fault.
- Step 3: Compared results with earlier predictions; drew the block diagram and decision table.

Task 10

- Step 1: Added video timeout, e-Rx validity, health-ID linking and audit to mediqueue.py with faults E1–E4.
- Step 2: Wrote three strategies: trim (3 tests), automation with 20 synthetic patients (26), shift-left with contract and audit tests (7).
- Step 3: Ran them on the six faults; computed weekly hours and the decision.

Commands

```
pip install pytest
python run_all.py        # 104 runs: 19 suites × (correct build + each mutant)
python analyze.py        # faults found and false reports per suite
python make_log.py       # regenerates defect_log_last_release.csv (164 records, seed 42)
pytest -v tests          # one run on the correct build (Annexure J)
MUTANT=F1 pytest -q tests/test_t1_C_continuous.py   # one suite on one mutant
```

ANNEXURE P – Specification Tables Used as Oracles

P1. Consultation fee by age (base ₹500, half-up rounding)

Age band	Concession	Fee on ₹500	Fee on ₹333	Boundary tests
0–4	100 %	₹0	₹0	0, 4
5–17	20 %	₹400	₹266 (266.4)	5, 17
18–59	0 %	₹500	₹333	18, 59
60 and above	40 %	₹300	₹200 (199.8 → half up)	60, 61

P2. Late-cancellation fee

Hours before slot	Fee	Boundary tests
less than 2	₹100	0.5, 1.99
2 or more	₹0	2.0, 2.01
Clarified (AMB-1)	Exactly 2.0 h is not charged; time measured to slot start	2.0

P3. Prescription dispensing

Condition	Rule	Accept	Reject
Validity	0–7 days after Rx date	7 days	8 days; future date
Quantity	at most 30 units	30	31
Batch expiry	expiry must be after today	tomorrow	today, yesterday
Duplicate	one dispense per Rx, any pharmacist	first	second (same or other pharmacist)

P4. Appointment rules

Rule	Accept	Reject
Active appointments ≤ 2	2nd booking	3rd booking
One patient per slot	first confirmation	second, even in the same second
No past slots	slot after now	slot before now
Cancelled slot freed	re-booking by another patient	–

ANNEXURE Q – Product Risk Register Used for Risk-Based Testing

Probability and impact on a 1–5 scale; exposure = probability × impact. The tests in the last column are the ones that address each risk in this answer.

ID	Risk	P	I	Exposure	Addressed by
RK-01	Wrong concession at age boundaries	4	4	16	Task 1 C, Task 3 B table
RK-02	Double bill on refresh or retry	3	5	15	Idempotency tests, D2 fault tree
RK-04	Expired prescription dispensed	3	5	15	Task 5 day 7/8 tests
RK-03	Two patients in one slot	3	4	12	Task 4 race tests
RK-06	Lab page slow at morning peak	4	3	12	Load test at 800 users
RK-12	No audit trail for tests on patient data	3	4	12	Task 10 audit assertion
RK-05	Batch dispensed on expiry day	2	5	10	Task 5 pharmacist cases
RK-10	Health-ID mismatch links wrong patient	2	5	10	Task 10 contract tests
RK-11	Unsigned e-prescription accepted	2	5	10	Task 10 e-Rx test
RK-07	SMS with stale time after reschedule	3	3	9	Requirement review, reminder tests
RK-13	Wrong expected value in a test (D8 type)	3	3	9	Oracle table review
RK-08	Wrong date on lab PDF	2	4	8	Field-mapping review
RK-09	Registration fails for special characters	3	2	6	Apostrophe / Unicode inputs

ANNEXURE R – Load-Test Plan for the Lab-Report Page (D6)

Item	Plan
Objective	Show that the lab-report page opens in ≤ 2 s for 90 % of requests at 800 concurrent users (9–10 am peak)
Tool	Apache JMeter 5.6, 2 load generators
Workload	Ramp 0 → 800 users in 10 min; hold 30 min; 60 % open lab report, 30 % appointment list, 10 % billing
Think time	8 s (uniform 5–11 s)
Data	Anonymised copy of 9,000 patients, 18 months of lab results
Measures	Average and 90th-percentile response time, errors, CPU and database time
Pass criterion	p90 ≤ 2 s and error rate < 1 % at 800 users
Schedule	Baseline in week 1, after the index/cache fix in week 2, final run in week 3

ANNEXURE S – Weekly Test Status Report (week 2 of the release, prototype figures)

Item	Status
Automated tests	217 test cases in 19 suites; correct build: 215 pass, 2 known false reports (peer-developer pass)
Seeded-fault detection of the chosen suites	Task 9 configuration C: 6 of 6; Task 10 strategy C: 6 of 6
Open defects	Prototype: none; release backlog: D1–D10 logged in JIRA
Risks	Pharmacist availability for Pharmacy acceptance; load environment
Next week	Retest of fixes, load test after index fix, release decision with medical officer

Source of make_log.py (defect log generator)

```
"""Builds a realistic 164-record defect log for the last release (seeded, reproducible)."""
import csv, random, collections
random.seed(42)
```

```
ORIG = ["Requirement", "Design", "Code", "Test case", "Configuration", "Data"]
DET = ["Requirement review", "Code review", "Unit test", "System test", "Production"]
M = {  # origin x detection counts
 "Requirement": [16, 2, 3, 13, 12], "Design": [2, 5, 3, 7, 5], "Code": [0, 13, 26, 24, 7],
 "Test case": [0, 2, 2, 5, 1], "Configuration": [0, 0, 1, 6, 2], "Data": [0, 0, 0, 5, 2]}
MODULES = {"Billing": 58, "Appointment": 48, "Pharmacy": 22, "Lab Reports": 16, "Registration": 11,
"Reminders": 9}
CAUSE_BY_ORIGIN = {"Requirement": ["Ambiguous/incomplete requirement"], "Design": ["Concurrency/idempotency",
"Performance", "Ambiguous/incomplete requirement"],
 "Code": ["Boundary/off-by-one logic", "Input validation", "Wrong field/data mapping",
"Concurrency/idempotency"],
 "Test case": ["Test-case error"], "Configuration": ["Configuration/environment"], "Data": ["Wrong field/data
mapping", "Configuration/environment"]}
recs = []
for o, row in M.items():
    for d, n in zip(DET, row):
        for _ in range(n): recs.append({"origin": o, "detected_in": d})
random.shuffle(recs)
mods = [m for m, n in MODULES.items() for _ in range(n)]; random.shuffle(mods)
# causes: draw from origin pools, weighted so Code is mostly boundary logic
for r, m in zip(recs, mods):
    r["module"] = m
    pool = CAUSE_BY_ORIGIN[r["origin"]]
    w = [5, 3, 2, 2] if r["origin"] == "Code" else [1] * len(pool)
    r["cause"] = random.choices(pool, weights=w[:len(pool)])[0]
# severity: 9 critical among production, 13 critical before release
prod = [r for r in recs if r["detected_in"] == "Production"]; pre = [r for r in recs if r["detected_in"] !=
"Production"]
for i, r in enumerate(prod): r["severity"] = "Critical" if i < 9 else ("Major" if i < 20 else "Minor")
sev_pre = ["Critical"] * 13 + ["Major"] * 39 + ["Minor"] * 55 + ["Trivial"] * 28
random.shuffle(sev_pre)
for r, s in zip(pre, sev_pre): r["severity"] = s
for i, r in enumerate(recs, 1):
    r["id"] = f"MQ-{i:03d}"; r["status"] = "Closed" if r["detected_in"] != "Production" else "Open"
with open("defect_log_last_release.csv", "w", newline="") as fh:
    w = csv.DictWriter(fh, fieldnames=["id", "module", "origin", "detected_in", "cause", "severity",
"status"])
    w.writeheader(); w.writerows(recs)
# ---- metrics ----
C = collections.Counter
print("total", len(recs), "pre", len(pre), "post", len(prod), "post critical", sum(r["severity"] == "Critical"
for r in prod))
KLOC = {"Registration": 4, "Appointment": 10, "Billing": 12, "Pharmacy": 9, "Lab Reports": 7, "Reminders": 5}
mc = C(r["module"] for r in recs); cum = 0
for m, n in mc.most_common():
    cum += n; print(f"{m:13s} {n:3d} {n/164*100:5.1f}%  cum {cum/164*100:5.1f}%  perKLOC {n/KLOC[m]:.2f}")
cc = C(r["cause"] for r in recs); cum = 0; print()
for c, n in cc.most_common():
    cum += n; print(f"{c:34s} {n:3d} {n/164*100:5.1f}% cum {cum/164*100:5.1f}%")
print(); print("post by origin", C(r["origin"] for r in prod)); print("post by module", C(r["module"] for r in
prod))
print("sev all", C(r["severity"] for r in recs))
```

ANNEXURE T – Formula Sheet with Worked Examples from this Answer

Measure	Formula	Worked example (this answer)
Fault-detection percentage	faults found ÷ faults seeded × 100	Task 1 model C: 7 issues found ÷ 7 = 100 %; model A: 1 ÷ 7 = 14 %
Issues found per person-hour	issues found ÷ effort (ph)	Model B: 5 ÷ 10 = 0.50
Escape cost	issues not found × cost after release	Model A: 6 × ₹45,000 = ₹2,70,000
Defect removal efficiency	defects found before release ÷ all defects × 100	Last release: 135 ÷ 164 = 82.3 %; C + review gate: 156 ÷ 164 = 95.1 %
Repair cost of n defects at a stage	n × cost per defect at that stage	15 at system test: 15 × ₹9,000 = ₹1,35,000
Containment effectiveness	defects of a type caught by its own filter ÷ all defects of that type	Code review: 13 ÷ 70 = 18.6 %
Share of defects	defects in module ÷ all defects	Billing: 58 ÷ 164 = 35.4 %
Defects per KLOC	defects ÷ size in KLOC	Pharmacy: 22 ÷ 9 = 2.44; escapes 9 ÷ 9 = 1.00
Exhaustive input space	product of the number of values of each input	250 × 20 × 3 × 3 × 2 × 2 = 1,80,000; at 1 test/s = 50 h
Percentage agreement	matching classes ÷ items × 100	e.g. 7 of 10 classes equal = 70 %
Reopen rate	defects reopened ÷ defects fixed	D2 reopened once of 2 fixed = 50 %
Mean time to fix	Σ(Fixed time − Open time) ÷ number of defects	computed from the JIRA history of D2 and D5
Cost per fault found	(testing cost + fixing cost) ÷ faults found	Task 9 configuration C: 6 × ₹2,500 ÷ 6 = ₹2,500
Hours fit	Σ stage hours ≤ budget	Task 10: 6 + 6 + 8 + 18 + 7 + 3 = 48 ≤ 50

ANNEXURE U – Test Data Sets

U1. Synthetic patients used by strategy B in Task 10 (generated, ASCII names)

#	Name	Date of birth	#	Name	Date of birth
1	Patient000	01-01-1960	11	Patient010	11-11-1970
2	Patient001	02-02-1961	12	Patient011	12-12-1971
3	Patient002	03-03-1962	13	Patient012	13-01-1972
4	Patient003	04-04-1963	14	Patient013	14-02-1973
5	Patient004	05-05-1964	15	Patient014	15-03-1974
6	Patient005	06-06-1965	16	Patient015	16-04-1975
7	Patient006	07-07-1966	17	Patient016	17-05-1976
8	Patient007	08-08-1967	18	Patient017	18-06-1977
9	Patient008	09-09-1968	19	Patient018	19-07-1978
10	Patient009	10-10-1969	20	Patient019	20-08-1979

None of these names contains an apostrophe, a space or a non-ASCII character – which is exactly why strategy B could not expose the health-ID fault E3 (Task 10).

U2. Realistic names recommended for registration and health-ID tests

Name	Why it is needed
Anita D’Souza	curly apostrophe (U+2019) as returned by the national service
Anita D'Souza	straight apostrophe as typed at reception
Mohammed Abdul Rahman Al-Faisal	long name, hyphen, several words
Ng Yi	very short name
Ramanathan S	initial as surname
Zoë Fernandes	accented character
(blank surname)	single-name patients common in South India

U3. Fixed dates used in the pharmacy tests (today = 12-03-2026)

Rx date	Age of Rx (days)	Expected
12-03-2026	0	Dispense
05-03-2026	7	Dispense
04-03-2026	8	Reject (D9 found here when months are compared)
28-02-2026	12	Reject (crosses a month boundary)
13-03-2026	−1 (future)	Reject